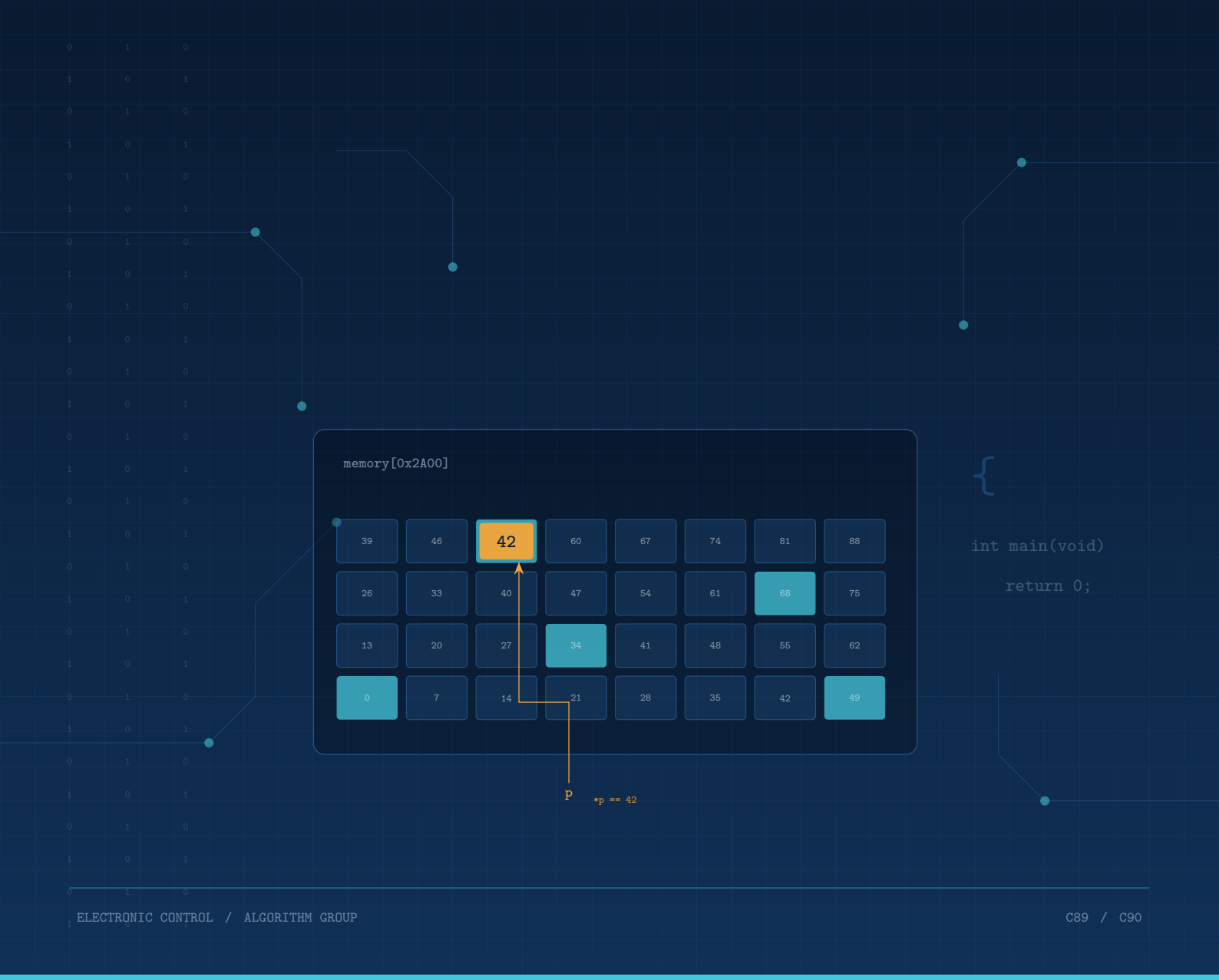


C 语言入门

从计算机组成到指针与算法

电控算法组新生培训教程

作者：电控算法组
组织：电控算法组 · 新生培训系列
时间：2026 年 10 月
版本：1.0

C
89

先想清楚「数据长什么样」，再想清楚「步骤怎么走」。

写在前面

这本书是写给谁的

这本书写给刚加入电控算法组的你。

也许你之前从未写过一行代码；也许你在高中信息课上看过 `print("hello")`，但不知道它为什么能运行；也许你用过 Python，却对“指针”“内存”这些词心存畏惧。都没关系。这本书假设你**只会用电脑，不会编程**，从“计算机里到底发生了什么”讲起，一步一步把你带到能独立写出一个像样程序的位置。

我们不追求“三天速成”，也不打算把你训练成“语法词典”。我们想让你建立三样东西：

1. **一个正确的心智模型**：知道变量住在内存的哪里、数组为什么从 0 开始、指针到底指向什么；
2. **一套可靠的编程习惯**：先想清楚再动手、先声明再使用、写完就编译、出错会调试；
3. **一种工程化的思维方式**：把大问题拆成小问题，把重复劳动交给循环和函数，把“能跑”变成“可读、可改、可维护”。

为什么电控算法组要学 C

你以后会接触电机、电池、传感器、嵌入式控制器，会写让无人机悬停、让小车循迹、让机械臂抓取的算法。这些场景有一个共同点：**程序要直接和硬件打交道，而且要在有限的时间、有限的内存里稳定运行**。

C 语言正好站在“硬件”和“算法”的交界处：

- 它**贴近硬件**：一个 `int` 占几个字节、数组在内存里怎么排布、指针里存的是哪个地址，你都能说得清；
- 它**足够快**：没有隐藏的运行时开销，适合实时控制；
- 它**足够小**：从 8 位单片机到服务器，几乎处处都有 C 编译器；
- 它**是很多语言的地基**：C++、Rust、Python 解释器、操作系统内核，都能看到它的影子。

学会 C，不只是学会一门语言，更是学会“站在机器的角度思考”。以后你再学 C++、学数据结构、学嵌入式，都会轻松很多。

关于“C98”和本书的代码标准

你可能会听到“C98”这个说法。严格来说，C 语言的标准版本叫 **C89**（1989 年 ANSI 标准，也叫 C90），之后还有 C99、C11、C17 等。本书所说的“C98 风格”，我们理解为：**按 C89/C90 的规矩写代码**。

这带来几条你必须从一开始就养成的习惯：

本书的 C89/C90 硬性约定

1. **先声明，后使用**：所有变量都写在语句块（`{`）的开头，声明之前不能执行任何语句。
2. **循环变量要在外面声明**：写成 `int i; for (i = 0; i < n; i++)`，**不能**写成 `for (int i = 0; ...)`——后者是 C99 才允许的。
3. **只用 `/* */` 注释**，不用 `//`（`//` 是 C99 引入的）。
4. **函数必须写返回类型**，`main` 结尾写 `return 0;`。
5. **不使用变长数组**（`int a[n];` 且 `n` 是变量），数组长度必须是编译期常量。

为什么这么“老派”？因为电控领域大量使用老编译器、老代码库和严格的编码规范（例如汽车电子的 MISRA C）。先把规矩学正，以后换到任何环境都不会被“惯坏”。而且，这些限制会逼你把“声明”和“逻辑”分开，代码反而更清楚。

如果你的编译器报错……

如果你的环境默认使用 C99 或更新标准，用 `for (int i = ...)` 可能不会报错，但这不代表它符合本书约定。请主动按 C89 的写法练习。附录 B 给出了推荐的编译命令，其中包含 `-std=c89 -pedantic` 选项，可以帮你检查出不合规的写法。

怎么读这本书

第一遍：顺着读，别跳

本书的章节顺序是精心安排的：先认识机器（第 1 章），再学“一条一条往下走”的顺序结构（第 2 章），然后才学判断（第 3、4 章）和循环（第 5 章）。数组、结构体、指针、字符串这四章是“数据怎么组织”的递进关系，排序算法则是前四章的综合运用，宏定义放在最后，讲的是“写代码的代码”。

请不要因为“这章好像很简单”就跳过。很多同学最后卡在指针，根源往往是前面数组和内存模型没学扎实。

第二遍：把代码敲一遍

本书所有完整程序都可以直接编译运行。请**亲手敲**，不要复制粘贴。敲代码时你会被迫注意分号、括号、大小写，这些“低级错误”是每个程序员都要经历的。

每敲完一个程序，做三件事：

1. 预测输出：先别运行，想一想它会打印什么；
2. 运行对照：和你的预测比较，不一样就追问“为什么”；
3. 改一改：改一个数字、删一个分号、把 `<` 改成 `<=`，看看会发生什么。

第三遍：做章末习题

每章末尾都有习题，附录 C 给出参考答案与提示。建议先自己写，卡住 20 分钟再看答案。看懂答案不等于会写，合上答案再默写一遍，才算真的会。

本书的排版约定

为了让你在阅读时快速抓住重点，本书用了几种固定的“视觉语言”：

版式	含义
蓝色标题的“定义”盒	重要概念的正式定义，第一次出现时务必读懂
绿色“小贴士”	经验、技巧、可以让你少走弯路的建议
橙色“小心有坑”	常见错误、易混淆点，考试和调试中的高频雷区
蓝灰色“C89 约定”	与代码标准相关的硬性规定
紫色“拔高阅读”	选读内容，第一次读不懂可以先跳过，不影响后续学习
带行号的浅色代码盒	完整或半完整的 C 程序，可直接编译
“运行结果”盒	程序在典型环境下的输出
“例题 / 解答”	带完整分析过程的示范题
“本章练习”	章末习题，答案见附录 C

正文中的行内代码用等宽字体表示，例如 `printf`、`&a`、`a[i]`。

学习地图

图 1 是全书的学习路线。可以把它贴在显示器旁边：每学完一章，就在心里点亮一个格子。

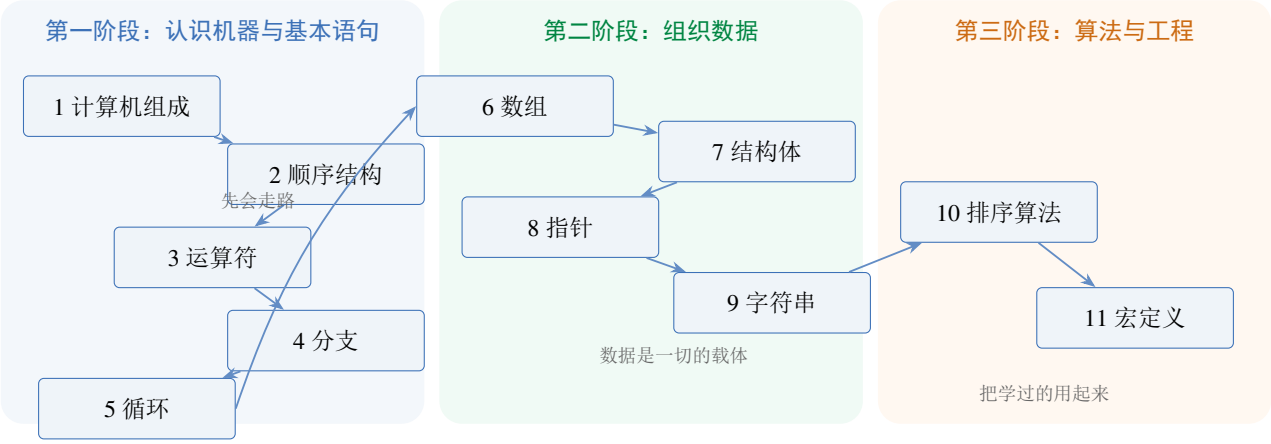


图 1: 全书学习路线：先建立机器模型，再学控制流程，然后组织数据，最后进入算法与工程

三个“不要”

不要只看不敲；不要一遇到报错就问人（先自己读错误信息）；不要背语法，要理解“这条语句让内存发生了什么”。

开发环境

本书不绑定任何编辑器。你只需要一个能编译 C89 的编译器（推荐 GCC/MinGW），以及一个顺手的编辑器（VS Code、Notepad++、Vim 都可以）。安装步骤、编译命令和常见报错见附录 B。

第一个程序永远是：

hello.c ——一切的开始

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

运行结果

```
Hello, world!
```

如果它能跑起来，恭喜你，你已经迈出了第一步。接下来，我们先把“计算机到底是怎么执行这行字的”弄清楚——这会让你后面少走很多弯路。

电控算法组

2026 年 10 月

目录

写在前面 **i**

第一章 计算机组成基础 **1**

1.1 计算机能做什么 . . . 1
1.2 计算机的四样东西 . . . 1
1.3 程序是什么 . . . 2
1.4 计算机为什么只认识 0 和 1 . . . 3
1.5 内存像一排小盒子 . . . 4
1.6 从写代码到程序运行 . . . 5
1.7 C 语言为什么适合电控 . . . 6
1.8 本章小结 . . . 6
第一章 练习 . . . 7

第二章 正式上路：顺序结构程序设计 **8**

2.1 第一个程序 . . . 8
2.2 变量：给数据准备一个盒子 . . . 10
2.3 输出：让计算机说话 . . . 12
2.4 输入：让计算机听我们说话 . . . 13
2.5 顺序结构：一步一步来 . . . 14
2.6 注释和好习惯 . . . 16
2.7 常见错误 . . . 17
2.8 本章小结 . . . 18
第二章 练习 . . . 18

第三章 写在分支结构之前：运算符 **20**

3.1 表达式与运算符总览 . . . 20
3.2 算术运算符 . . . 21
3.3 赋值运算符 . . . 23
3.4 自增与自减 . . . 24
3.5 关系运算符 . . . 25
3.6 逻辑运算符 . . . 26
3.7 位运算符 . . . 27
3.8 条件运算符 . . . 30
3.9 逗号、`sizeof` 与强制类型转换 . . . 31
3.10 优先级与结合性总表 . . . 33
3.11 类型转换 . . . 33
3.12 求值顺序与副作用 . . . 35
3.13 综合例题 . . . 36
3.14 本章小结 . . . 38
第三章 练习 . . . 38

第四章 分支：明辨是非 **40**

4.1 为什么需要分支 . . . 40

4.2 if 语句 . . . 40
4.3 if-else 语句 . . . 43
4.4 else if：多路分支 . . . 45
4.5 嵌套分支 . . . 47
4.6 switch 语句 . . . 50
4.7 分支的常见错误 . . . 55
4.8 综合例题 . . . 55
4.9 拔高：让分支更好读 . . . 59
4.10 本章小结 . . . 60
第四章 练习 . . . 60

第五章 循环：重复再重复 61
5.1 为什么需要循环 . . . 61
5.2 while 循环 . . . 62
5.3 for 循环 . . . 65
5.4 do-while 循环 . . . 67
5.5 break 与 continue . . . 68
5.6 嵌套循环 . . . 70
5.7 循环的经典算法 . . . 72
5.8 怎样写好一个循环 . . . 78
5.9 循环的常见错误 . . . 78
5.10 综合例题 . . . 79
5.11 拔高：循环的效率与递归 . . . 82
5.12 本章小结 . . . 83
第五章 练习 . . . 83

第六章 数组：优雅的数据存取方式 84
6.1 为什么需要数组 . . . 84
6.2 一维数组 . . . 85
6.3 数组的常见操作 . . . 88
6.4 二维数组 . . . 92
6.5 数组的常见错误 . . . 96
6.6 拔高：数组名、退化与栈空间 . . . 96
6.7 综合例题 . . . 97
6.8 本章小结 . . . 100
第六章 练习 . . . 100

第七章 结构体：古希腊掌管分类的神 101
7.1 为什么需要结构体 . . . 101
7.2 结构体的定义与使用 . . . 102
7.3 结构体在内存里长什么样 . . . 105
7.4 结构体数组 . . . 106
7.5 结构体与函数 . . . 107
7.6 综合例题 . . . 109
7.7 常见错误 . . . 113
7.8 拔高：位域与联合体 . . . 113

7.9 本章小结 . . . 115
第七章 练习 . . . 115

第八章 指针：会者不难 **116**
8.1 地址与指针 . . . 116
8.2 指针与函数 . . . 119
8.3 指针与数组 . . . 121
8.4 指针与结构体 . . . 123
8.5 二级指针与函数指针（拔高） . . . 124
8.6 动态内存 . . . 125
8.7 指针的常见错误 . . . 127
8.8 指针与 const . . . 127
8.9 综合例题 . . . 128
8.10 本章小结 . . . 130
第八章 练习 . . . 130

第九章 字符串 **131**
9.1 字符与字符串 . . . 131
9.2 字符数组与字符指针 . . . 133
9.3 字符串的输入与输出 . . . 134
9.4 常用字符串函数 . . . 136
9.5 字符串的处理 . . . 140
9.6 字符串与指针 . . . 143
9.7 中文与字符编码 . . . 144
9.8 常见错误 . . . 145
9.9 综合例题 . . . 145
9.10 本章小结 . . . 147
第九章 练习 . . . 148

第十章 排序算法 **149**
10.1 为什么要排序 . . . 149
10.2 冒泡排序 . . . 149
10.3 选择排序 . . . 151
10.4 插入排序 . . . 152
10.5 三种排序比一比 . . . 154
10.6 小应用：给成绩排序 . . . 154
10.7 本章小结 . . . 155
第十章 练习 . . . 155

第十一章 宏定义 **156**
11.1 预处理器 . . . 156
11.2 对象式宏 . . . 157
11.3 函数式宏 . . . 158
11.4 # 与 ## . . . 160
11.5 条件编译 . . . 161
11.6 预定义宏 . . . 162

11.7 调试宏 . . . 163
11.8 宏的常见陷阱 . . . 164
11.9 拔高：X-Macro 与代码生成 . . . 164
11.10 本章小结 . . . 165
第十一章 练习 . . . 165

附录 A C 语言速查表 **166**
A.1 关键字（C89） . . . 166
A.2 基本类型 . . . 166
A.3 常用格式符 . . . 167
A.4 转义字符 . . . 167
A.5 运算符优先级（高到低） . . . 167
A.6 常用标准库函数 . . . 168
A.7 常用代码片段 . . . 169

附录 B 编译、调试与常见错误 **170**
B.1 选择工具链 . . . 170
B.2 Windows：安装 MinGW-w64 . . . 170
B.3 第一个程序的编译与运行 . . . 170
B.4 读懂错误信息 . . . 171
B.5 常见编译错误对照 . . . 172
B.6 用 gdb 调试 . . . 172
B.7 VS Code 配置 . . . 172
B.8 多文件项目 . . . 173
B.9 编码问题 . . . 173
B.10 推荐的学习流程 . . . 173

附录 C 习题参考答案与提示 **174**
C.1 第 1 章 . . . 174
C.2 第 2 章 . . . 174
C.3 第 3 章 . . . 174
C.4 第 4 章 . . . 175
C.5 第 5 章 . . . 175
C.6 第 6 章 . . . 175
C.7 第 7 章 . . . 176
C.8 第 8 章 . . . 176
C.9 第 9 章 . . . 176
C.10 第 10 章 . . . 176
C.11 第 11 章 . . . 177

附录 D 术语中英对照 **178**

附录 E 函数速查（补充） **180**
E.1 函数的定义与调用 . . . 180
E.2 返回值与 `void` . . . 181
E.3 函数声明（原型） . . . 181
E.4 参数传递 . . . 182

E.5 递归 183
E.6 作用域与存储类别 183
E.7 多文件编程 184
E.8 函数的常见错误 185

第一章 计算机组成基础

内容提要

- 计算机由哪几个部分组成
- 程序到底是什么
- 计算机为什么只认识 0 和 1
- 内存、地址和变量
- 从写代码到程序运行

「计算机很笨，但它很听话。」

同学们，先别急着写代码。我们先用一节课的时间，认识一下计算机这个“大玩具”。

你可能会觉得计算机很聪明：它会算题、会放视频、会下棋。其实它一点都不聪明。它只会做很简单的事情：**算一算、比一比、把东西搬来搬去**。它之所以看起来聪明，是因为人把做事情的步骤一条一条写给了它。

这一章不写程序，只讲五件事：计算机由什么组成、程序是什么、为什么它只认识 0 和 1、内存是什么、以及你写的代码是怎么变成能运行的程序的。

1.1 计算机能做什么

我们用计算机做事情，通常都是这样一个过程：



举几个例子：

- 用计算器算 3 + 5：你按的“3、+、5”是**输入**；计算器在内部做加法是**处理**；屏幕上出现“8”是**输出**。
- 用手机拍照：镜头看到画面是输入；手机把画面变成照片是处理；屏幕上显示照片是输出。
- 让小车前进：传感器测到距离是输入；单片机判断“前面没有障碍”是处理；电机转动是输出。

输入 → 处理 → 输出，这就是计算机做事的三个步骤。我们写程序，主要就是在告诉计算机“怎么处理”。

1.2 计算机的四样东西

一台计算机看起来零件很多，但真正重要的只有四样：

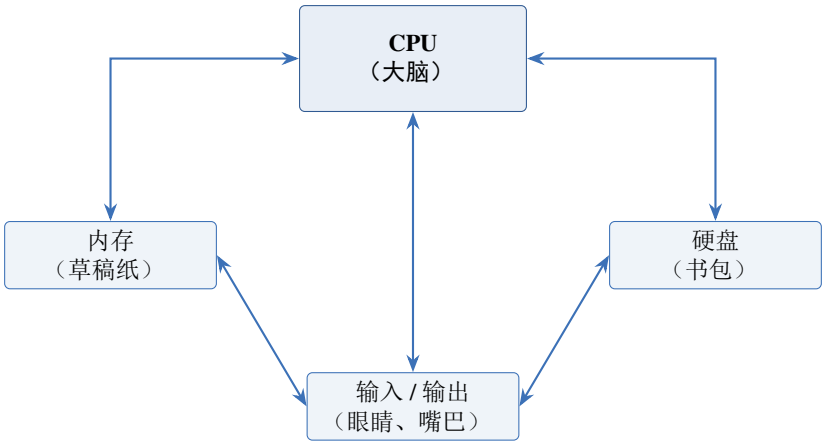


图 1.1: 计算机的四样核心部件：CPU、内存、硬盘、输入输出

1.2.1 CPU：计算机的大脑

CPU 是计算机的“大脑”，全名叫**中央处理器**。

它负责两件事：

1. **算**：做加减乘除、比较大小、判断真假；
2. **指挥**：告诉其他部件“现在该做什么”。

CPU 算得很快，一秒钟能做几十亿次运算。但它只会做很简单的运算，复杂的事情要拆成很多小步骤。

1.2.2 内存：计算机的草稿纸

内存是计算机的“草稿纸”，用来临时存放正在使用的数据。

它的特点是：

- 速度比较快；
- **一断电，里面的东西就全没了**。

所以内存只适合放“正在用”的东西。比如你正在写的文档、正在玩的游戏，数据都在内存里。突然断电，没保存的内容就丢了。

1.2.3 硬盘：计算机的书包

硬盘是计算机的“书包”，用来长期存放东西。

它的特点是：

- 速度比内存慢；
- **断电以后东西还在**。

你的照片、作业、程序文件，平时都放在硬盘里。用的时候，计算机会把它们从硬盘搬到内存里。

手机也是一样的

手机的“8 GB + 256 GB”里，8 GB 通常指运行内存（相当于电脑的内存），256 GB 是存储空间（相当于硬盘）。内存越大，能同时开的程序越多；存储越大，能存的东西越多。

1.2.4 输入和输出：眼睛、耳朵和嘴巴

输入设备让计算机“知道外面发生了什么”，例如键盘、鼠标、麦克风、摄像头、温度传感器。

输出设备让计算机“把结果告诉外面”，例如屏幕、音箱、打印机、电机、指示灯。

部件	像人的什么	作用
CPU	大脑	计算、指挥
内存	草稿纸	临时存放正在用的数据
硬盘	书包	长期存放数据
输入设备	眼睛、耳朵	把信息送进计算机
输出设备	嘴巴、手	把结果送出去

1.3 程序是什么

1.3.1 程序就是一张步骤清单

程序不是魔法，它只是一张**步骤清单**。就像泡面的做法：

1. 烧一壶水；

2. 把面饼放进碗里；
3. 倒入开水；
4. 等三分钟；
5. 放入调料，搅拌。

计算机做事情也是这样：你给它一张步骤清单，它就从第一条开始，一条一条做下去。

1.3.2 计算机不会猜

人听别人说话，可以猜意思。计算机不会猜。你写“把水烧开”，它不知道多少度算开、用什么烧、要等多久。你必须写清楚：

- 用哪个锅？
- 加热到多少度？
- 到了温度以后做什么？

写程序也是一样：**每一步都要明确，不能有歧义**。这就是为什么程序设计要学很多规则。

把计算机当成一个非常听话、但非常死板的同学

你说“往前走”，他就一直往前走，撞墙也不停，因为你没说“遇到墙就停”。写程序时，要把所有情况都想到。

1.4 计算机为什么只认识 0 和 1

1.4.1 电灯只有开和关

计算机里面是电子电路。电路最简单的两种状态是：

- 通电；
- 断电。

就像电灯：要么亮，要么灭。我们用数字表示这两种状态：

$$\text{通电} = 1, \qquad \text{断电} = 0.$$

只用两种状态，电路最简单、最不容易出错。所以计算机选择用 0 和 1 来表示所有信息。

1.4.2 位和字节

- 一个 0 或一个 1，叫一位（bit，比特）；
- 8 位合在一起，叫一个字节（byte）。

1	0	1	1	0	0	1	0

这是一个字节，一共 8 位。一个字节可以组合出 $2^8 = 256$ 种不同的样子（从 `00000000` 到 `11111111`）。

1.4.3 用 0 和 1 数数

我们平时用十进制：0、1、2、3、4、5、6、7、8、9，到 9 再往上就进位，变成 10。

计算机用二进制：只有 0 和 1，到 1 再往上就进位：

二进制	十进制
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8

规律是：**满二进一**。和我们数数“满十进一”是一样的，只是计算机只有两个数字。

例题 1.1 二进制 1011 表示十进制多少？

解从右往左，每一位的“份量”分别是 1、2、4、8：

$$1011 = 8 + 0 + 2 + 1 = 11.$$

所以二进制 1011 就是十进制的 11。

1.4.4 字符怎么表示

计算机里没有“字母”，只有数字。那怎么存字母呢？

人们约定了一张“密码表”，给每个字符编一个数字。最常用的是 **ASCII 码**。例如：

字符	ASCII 码	说明
'0'	48	数字字符 0
'A'	65	大写字母 A
'a'	97	小写字母 a
空格	32	空格
'\n'	10	换行

所以计算机里存的不是字母 A，而是数字 65。程序输出时，再按照密码表把 65 变回字母 A。

汉字也有编码表，例如 GBK、UTF-8。一个汉字通常要占 2 个或 3 个字节。这部分以后用到再了解。

负数和小数怎么办？

负数和小数在计算机里也有专门的表示方法，但比正数复杂。初学阶段先不用管，只要知道“计算机有办法表示”就可以了。等你把基本程序写熟，再回头看也不迟。

1.5 内存像一排小盒子

1.5.1 地址就是盒子编号

内存可以想象成一排小盒子，每个盒子装 1 个字节。每个盒子都有一个编号，这个编号叫**地址**。

地址就像酒店的房间号：房间号不是房间本身，但拿着房间号就能找到房间。

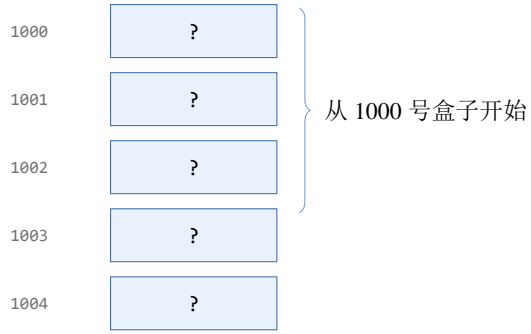


图 1.2: 内存像一排有编号的小盒子，每个盒子有地址

1.5.2 变量就是贴了标签的盒子

写程序时，我们不需要每次都记“1000”这样的地址。我们给盒子贴一个标签，起一个名字，例如：

variable-box.c ——变量就是一个盒子

```c
int a = 10;
```

这句话的意思是：

- 准备一个能装整数的盒子；
- 在盒子外面贴上标签 a；
- 往盒子里放 10。

以后写 a，就是在说“那个贴了 a 标签的盒子”。盒子里的东西可以换，但标签不变。

变量三件事

每个变量都有：**名字**（标签）、**类型**（盒子能装什么）、**值**（盒子里现在装什么）。地址是盒子的编号，通常由计算机管理，不需要你手动记。

1.6 从写代码到程序运行

1.6.1 人话和机器话

你写的 C 代码，长这样：

hello.c ——给人看的代码

```c
#include <stdio.h>

int main(void)
{
    printf("Hello!\n");
    return 0;
}
```

但 CPU 看不懂英文，它只认识 0 和 1 组成的机器指令。所以中间需要一个“翻译官”。

1.6.2 编译器就是翻译官

把 C 代码翻译成机器能懂的程序，这个工具叫**编译器**。



图 1.3: 写代码 → 编译 → 运行

过程是这样的：

1. 你在电脑上写一个 `hello.c` 文件；
2. 用编译器把它翻译成 `hello.exe`；
3. 双击 `hello.exe`，计算机就按里面的指令做事。

1.6.3 两种错误

写程序会遇到两种错误：

- **编译错误**：翻译官看不懂你写的话。比如少了一个分号、括号不配对。这时程序根本生成不了。
- **运行错误**：程序能生成，但运行的时候做错了事。比如让计算机除以 0，或者读到了不该读的地方。

报错不可怕

编译器报错时，会告诉你“第几行出了问题”。先看第一行错误，通常改掉它，后面的错误就消失了。

1.7 C 语言为什么适合电控

你以后要接触电机、传感器、单片机、机器人。这些设备有三个特点：

- 要**快**：电机控制一秒钟要算很多次；
- 要**省**：单片机的内存很小；
- 要**直接控制硬件**：读写寄存器、引脚、串口。

C 语言正好适合：

1. 它很**接近硬件**，能直接操作内存和寄存器；
2. 它**很快**，没有太多额外的开销；
3. 它**通用**，从单片机到电脑都能用；
4. 学好 C，再学 C++、Python、数据结构都会更容易。

1.8 本章小结

问题	一句话答案
计算机做什么	输入 → 处理 → 输出
有哪四样核心部件	CPU、内存、硬盘、输入输出
CPU 是什么	大脑，负责计算和指挥
内存和硬盘的区别	内存快但断电就丢；硬盘慢但能长期保存
程序是什么	给计算机的步骤清单，必须一步一步写清楚
计算机为什么用二进制	电路只有通/断两种稳定状态
位和字节	1 位是 0 或 1；8 位是 1 字节
内存是什么	一排有编号的小盒子，编号叫地址
变量是什么	贴了名字标签的盒子
代码怎么运行	写代码 → 编译器翻译 → 可执行程序 → 运行

下一章预告

认识完计算机，下一章我们就写第一个真正的 C 程序：让计算机在屏幕上说话，再让它听我们输入数字、做计算。

第一章 练习

1. 用自己的话说一说：计算机做一件事要经过哪三个步骤？
2. CPU、内存、硬盘各像什么？它们分别负责什么？
3. 内存和硬盘最大的区别是什么？为什么手机内存越大越贵？
4. 程序为什么必须一步一步写清楚？举一个“没说清楚”的例子。
5. 二进制 `1010`、`1100`、`11111` 分别等于十进制多少？
6. 1 个字节等于多少位？一个字节最多能表示多少种不同的组合？
7. 字符 `'A'` 在计算机里存的是什么？为什么不是直接存字母？
8. 把“写代码 → 运行”的过程画出来，并说明编译器做了什么。
9. 编译错误和运行错误有什么区别？各举一个例子。
10. 思考题：为什么计算机不直接用十进制，而是用二进制？

第二章 正式上路：顺序结构程序设计

内容提要

- 写第一个 C 程序，并逐行看懂它
- 变量：给数据准备一个盒子
- 用 printf 输出，用 scanf 输入
- 顺序结构：输入 → 计算 → 输出
- 注释、缩进和常见错误

「先把一句话说清楚，再写第一行代码。」

上一章我们认识了计算机：它很笨，只会按步骤做事。这一章，我们第一次给它下命令。

你不需要记住很多语法。先跟着做，把第一个程序跑起来。跑起来以后，我们再一行一行弄懂它。

2.1 第一个程序

2.1.1 写代码

打开你的编辑器，新建一个文件，名字叫 hello.c，把下面几行抄进去：

hello.c ——第一个 C 程序

```c
#include <stdio.h>

int main(void)
{
    printf("你好，计算机！\n");
    return 0;
}
```

编译并运行后，屏幕上会出现：

运行结果

```
你好，计算机！
```

恭喜，你已经写出第一个程序了！下面我们一行一行看，它到底在说什么。

2.1.2 逐行解释

第一行：#include <stdio.h>

这一行是“借工具箱”。

stdio 是“标准输入输出”的缩写。我们马上要用到的 printf（在屏幕上打印）就放在这个工具箱里。写这一行，相当于告诉计算机：“我要用这个工具箱里的东西。”

暂时不用深究

你现在只要记住：**只要用到 printf 或 scanf，开头就写这一行**。为什么、怎么找到工具箱的，以后会慢慢明白。

第三行：int main(void)

这一行是程序的“大门”。

计算机运行程序时，会先找到 main，从它后面的花括号开始，一句一句往下执行。

- main 是固定的名字，不能改；
- int 表示程序结束时会给计算机一个整数；
- (void) 表示这个程序不需要从外面接收参数。

花括号 { }：范围

花括号里的内容，是这个程序要做的所有事情。就像写作文的“正文部分”。

第五行：printf(" 你好，计算机！\n");

这一行让计算机“说话”。

- printf 是打印函数，负责把双引号里的内容显示到屏幕上；
- 双引号里的文字，会原样显示；
- \n 表示“换行”，不是两个普通字符；
- 最后的分号; 表示“这句话说完了”，相当于中文的句号。

分号不能少

C 语言的每条语句后面都要有分号。少一个分号，编译器就会报错。分号必须是英文的;，不能是中文的；。

第六行：return 0;

这一行告诉计算机：“程序正常结束了。”

0 表示“一切正常”。如果程序出了问题，可以返回别的数字，但对初学者来说，先固定写 return 0; 就好。

2.1.3 程序的基本骨架

把上面的程序缩成一幅图，就是：

```
#include <stdio.h>          借工具箱
int main(void)              程序大门
{ 要做的事情}               程序内容
```

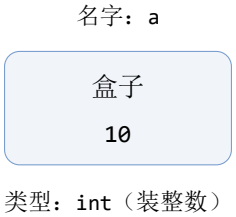
写代码的三个好习惯

1. 花括号单独占一行，里面的语句往右缩进 4 个空格；
2. 每条语句后面写英文分号；
3. main 写成 int main(void)，结尾写 return 0;。

2.2 变量：给数据准备一个盒子

2.2.1 什么是变量

变量就是一个“贴了名字的盒子”，里面可以放数据。



写程序时，我们这样使用它：

variable.c ——变量的声明和赋值

```c
int a;          /* 准备一个盒子，名字叫 a */
a = 10;         /* 把 10 放进盒子里 */
```

也可以把两步合成一步：

variable2.c ——声明时直接放值

```c
int a = 10;     /* 准备盒子 a，并放入 10 */
```

2.2.2 变量的类型

盒子有不同的大小，能装的东西也不一样。C 语言里常用的类型有三种：

类型	能装什么	例子
`int`	整数	0、5、-3、100
`double`	小数	3.14、0.5、-2.75
`char`	一个字符	’A’、’x’、’5’

- `int` 是 *integer*（整数）的缩写；
- `double` 表示双精度小数，可以理解为“更精确的小数”；
- `char` 是 *character*（字符）的缩写。

先只用这三种

初学阶段，整数用 `int`，小数用 `double`，单个字符用 `char`，足够用了。其他类型以后需要再学。

2.2.3 变量名怎么起

变量名要遵守几条规则：

- 只能用字母、数字、下划线；
- 不能以数字开头；
- 区分大小写，`a` 和 `A` 是两个不同的变量；
- 不能用 C 语言的关键字，例如 `int`、`return`。

起名字要让人一看就懂：

好名字	不好的名字
age、score、sum	a、b、x1
max_value、student_num	abc、tmp1

2.2.4 C89 的声明规则

C89 规定：**一个花括号里，所有变量都要先声明，再写其他语句。**

正确的写法：

declare-ok.c ——先声明，后使用

```c
#include <stdio.h>

int main(void)
{
    int a = 3;
    int b = 5;
    int sum;

    sum = a + b;
    printf("%d\n", sum);
    return 0;
}
```

错误的写法：

declare-bad.c ——声明写在了后面

```c
#include <stdio.h>

int main(void)
{
    int a = 3;

    printf("%d\n", a);

    int b = 5;      /* 错误：声明不能写在语句后面 */
    printf("%d\n", b);
    return 0;
}
```

记住这句话

变量声明全部写在花括号开头，后面才写执行语句。后面学 `for` 循环时，循环变量也要提前声明，不能写在 `for` 里面。

2.3 输出：让计算机说话

2.3.1 用 printf 输出

printf 可以输出文字，也可以输出变量的值。

printf1.c ——输出文字和数字

```c
#include <stdio.h>

int main(void)
{
    int age = 18;

    printf("我今年 %d 岁\n", age);
    return 0;
}
```

运行结果

```
我今年 18 岁
```

这里的%d 是一个“占位符”，意思是“这里要放一个整数”。后面的 age 会填到这个位置。

常用的占位符：

占位符	用来输出	例子
%d	整数	printf("%d", 10);
%f	小数	printf("%f", 3.14);
%.2f	保留两位小数	printf("%.2f", 3.14159); 输出 3.14
%c	一个字符	printf("%c", 'A');
%s	字符串	printf("%s", " 你好");

占位符和类型要对应

整数用%d，小数用%f。用错了可能输出奇怪的结果。比如用%d 输出小数，就是不对的。

2.3.2 换行

\n 表示换行。没有它，下一次输出会接在同一行后面。

newline.c ——换行的效果

```c
#include <stdio.h>

int main(void)
{
    printf("第一行\n");
    printf("第二行\n");
    return 0;
}
```

运行结果

```
第一行
第二行
```

2.4 输入：让计算机听我们说话

2.4.1 用 scanf 读入

如果想让用户输入数字，就用 scanf。

scanf1.c ——读入一个整数

```c
#include <stdio.h>

int main(void)
{
    int a;

    printf("请输入一个整数：");
    scanf("%d", &a);

    printf("你输入的是 %d\n", a);
    return 0;
}
```

运行示例

```
请输入一个整数：7
你输入的是 7
```

注意 scanf 里面写的是 &a，不是 a。这个 & 读作“取地址”。

为什么 scanf 要加 &？

scanf 要把读到的数字放进变量 a 的盒子里，所以它需要知道盒子在哪里。&a 就是“盒子 a 的地址”。现在先照写，等学指针时会彻底明白。

2.4.2 读入两个数

scanf2.c ——读入两个整数

```c
#include <stdio.h>

int main(void)
{
    int a, b;

    printf("请输入两个整数：");
    scanf("%d%d", &a, &b);

```

```c
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

运行示例

```
请输入两个整数：3 5
a = 3, b = 5
```

两个%d 连在一起，输入时用空格或回车隔开都可以。

2.5 顺序结构：一步一步来

顺序结构就是程序从上往下，一句一句执行。大部分小程序的套路都是：



下面用几个例子练一练。

2.5.1 例：两数之和

例题 2.1 输入两个整数，输出它们的和。

解

sum.c ——两数之和

```c
#include <stdio.h>

int main(void)
{
    int a, b, sum;

    printf("请输入两个整数：");
    scanf("%d%d", &a, &b);

    sum = a + b;
    printf("和是 %d\n", sum);
    return 0;
}
```

运行示例

```
请输入两个整数：3 5
和是 8
```

2.5.2 例：长方形的面积

例题 2.2 输入长和宽，输出长方形的面积。

解

rectangle.c ——长方形面积

```c
#include <stdio.h>

int main(void)
{
    double length, width, area;

    printf("请输入长和宽: ");
    scanf("%lf%lf", &length, &width);

    area = length * width;
    printf("面积是 %.2f\n", area);
    return 0;
}
```

运行示例

```
请输入长和宽: 3.5 2
面积是 7.00
```

scanf 读 double 时用%lf，输出时用%f 或%.2f。先记住这个区别。

2.5.3 例：求平均分

例题 2.3 输入三门课的成绩，输出平均分，保留两位小数。

解

average.c ——平均分

```c
#include <stdio.h>

int main(void)
{
    double a, b, c, average;

    printf("请输入三门课成绩: ");
    scanf("%lf%lf%lf", &a, &b, &c);

    average = (a + b + c) / 3.0;
    printf("平均分是 %.2f\n", average);
    return 0;
}
```

运行示例

```
请输入三门课成绩: 80 90 100
平均分是 90.00
```

2.5.4 例：交换两杯水

例题 2.4 有两个变量 a 和 b，怎样交换它们的值？

解 交换两杯水，需要第三个空杯子。交换两个变量，也需要一个临时变量。

swap.c ——交换两个变量

```c
#include <stdio.h>

int main(void)
{
    int a = 3;
    int b = 5;
    int temp;

    printf("交换前: a = %d, b = %d\n", a, b);

    temp = a;       /* 把 a 倒进空杯子 */
    a = b;          /* 把 b 倒进 a */
    b = temp;       /* 把空杯子里的 a 倒进 b */

    printf("交换后: a = %d, b = %d\n", a, b);
    return 0;
}
```

运行结果

```
交换前: a = 3, b = 5
交换后: a = 5, b = 3
```

这三步以后会经常用到，请记牢。

2.6 注释和好习惯

2.6.1 写注释

注释是写给人看的，计算机不会执行。C89 用 `/* */` 包住注释：

comment.c ——注释

```c
#include <stdio.h>

int main(void)
{
    /* 这是一个变量，用来存年龄 */
    int age = 18;

    printf("%d\n", age);    /* 输出年龄 */
    return 0;
}
```

- `/*` 和 `*/` 要成对出现；
- 注释可以写在一行里，也可以跨很多行；
- 注释是给自己和同学看的，写“为什么这样做”，不要写废话。

C89 只用块注释

C89 不支持 `//` 这种单行注释。本书所有代码都用 `/* */`。

2.6.2 排版好习惯

同样的程序，排版好会更容易看懂：

style.c ——好的排版

```c
#include <stdio.h>

int main(void)
{
    int a = 10;
    int b = 20;
    int sum;

    sum = a + b;

    printf("sum = %d\n", sum);
    return 0;
}
```

- 花括号单独占一行；
- 里面的语句缩进 4 个空格；
- 不同功能的代码之间空一行；
- 变量名起得有意义。

2.7 常见错误

初学阶段，90% 的错误都是下面几种：

错误	原因
少写分号	每条语句末尾要有;
用了中文标点	中文的;，（）计算机不认识
`printf` 拼错	大小写要一致，是 `printf`
`scanf` 忘了 `&`	应该写 `scanf("%d", &a);`
占位符用错	整数用`%d`，小数用`%f`
变量没赋值就使用	盒子里是垃圾数据
变量声明写在语句后面	C89 不允许，要全部提到开头

程序一闪而过

在 Windows 上双击运行控制台程序，程序执行完窗口就关了，看起来“一闪而过”。这是正常现象。可以在最后加一句 `getchar();` 让程序等一下，或者从命令行运行。

2.8 本章小结

主题	要点
程序骨架	头文件、`main`、花括号、`return 0;`
输出	`printf`，用`%d`、`%f` 等占位
输入	`scanf`，变量前加 `&`
变量	先声明，后使用；名字要有意义
类型	整数 `int`，小数 `double`，字符 `char`
顺序结构	输入 → 计算 → 输出
注释	`/* */`
常见错误	少分号、中文标点、忘 `&`、占位符不对

下一章预告

顺序结构只能一条路走到底。下一章我们先学运算符，再学“如果……就……”的分支结构，让程序学会判断。

第二章 练习

1. 写一个程序，在屏幕上输出你的姓名和一句座右铭，分两行显示。
2. 输入两个整数，输出它们的和、差、积。
3. 输入一个圆的半径，输出它的周长和面积，保留两位小数（π 取 3.14159）。
4. 输入一个长方形的长和宽，输出面积和周长。
5. 输入三个整数，输出它们的平均值，保留两位小数。
6. 输入两个整数，交换后输出，要求使用临时变量。
7. 下面程序有几处错误？找出来并改正。

find-errors.c ——找错误

```c
#include <stdio.h>

int main(void)
{
    int a = 1
    int b = 2;
    sum = a + b;
    printf("%d\n", sum)
    return 0;
}
```

8. 解释 `scanf("%d", &a);` 中 `&` 的作用。
9. 解释 `printf("%d", 3.14);` 为什么不对，应该怎么写。

10. 思考题：为什么变量要先声明再使用？

第三章 写在分支结构之前：运算符

内容提要

- 算术、赋值、自增自减：最常见的四类运算
- 关系与逻辑：为“判断”准备的两把尺子
- 位运算：直接操作二进制位
- 优先级、结合性、类型转换与求值顺序
- 副作用与未定义行为：那些“看起来对”的坑

「程序 = 数据 + 运算 + 控制。先把运算说清楚，控制才有着落。」

下一章要学分支，也就是“如果满足某个条件，就做某件事”。条件从哪来？从**关系运算**和**逻辑运算**来。所以这一章是分支的“前置技能包”。

除此之外，本章还会把 C 语言里所有常用运算符过一遍。内容看起来多，但核心只有三句话：

1. 每个运算符都有**优先级**和**结合性**，不确定就加括号；
2. 不同类型混算会发生**类型转换**，转换规则要心里有数；
3. 有些表达式有**副作用**，求值顺序不对结果就不可预测。

3.1 表达式与运算符总览

3.1.1 什么是运算符

运算符是告诉编译器“做什么运算”的符号，例如 `+`、`*`、`<`、`&&`。参与运算的数据叫**操作数**。

分类	运算符	说明
算术	`+ - * / %`	加减乘除、取余
赋值	`= += -= *= /= %=`	把值写进变量
自增自减	`++ --`	加 1 / 减 1
关系	`> < >= <= == !=`	比较大小、判断相等
逻辑	`&& || !`	与、或、非
位运算	`& | ^ ~ << >>`	按二进制位操作
条件	`?:`	三目运算符
逗号	`,`	顺序求值
其他	`sizeof`、强制类型转换 `(类型)`	求大小、转换类型

3.1.2 优先级与结合性：先算谁

一个表达式里出现多个运算符时，先算谁由**优先级**决定；优先级相同的，由**结合性**决定（从左往右还是从右往左）。

例如 `2 + 3 * 4`，因为 `*` 的优先级高于 `+`，所以先算 `3 * 4`，结果是 14。如果先算加法，结果会是 20。这就是优先级的差别。

记不住优先级怎么办

加括号。括号的优先级最高，而且能让读代码的人一眼看懂你的意图。经验法则是：**只要混合了三种以上运算符，就加括号**。

完整的优先级表（从高到低）放在本章后面，先学具体运算符，再回来看表。

3.2 算术运算符

3.2.1 加、减、乘、除

arith.c ——四则运算

```c
#include <stdio.h>

int main(void)
{
    int a = 7;
    int b = 2;

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    return 0;
}
```

运行结果

```
a + b = 9
a - b = 5
a * b = 14
a / b = 3
```

注意 `7 / 2` 的结果是 3，不是 3.5。因为两个操作数都是整数，C 执行的是**整数除法**：只保留商的整数部分，小数部分直接丢弃（不是四舍五入）。

整数除法向零截断

`7 / 2` 得 3，`-7 / 2` 得 −3（向零截断，不是向下取整 −4）。如果你需要小数结果，把至少一个操作数变成浮点：`7.0 / 2` 得 3.5。

3.2.2 取余运算符%

`%` 计算除法后的余数，只能用于整数：

mod.c ——取余

```c
#include <stdio.h>

int main(void)
{
    printf("7 %% 2 = %d\n", 7 % 2);
    printf("8 %% 2 = %d\n", 8 % 2);
    printf("7 %% 3 = %d\n", 7 % 3);
    return 0;
}
```

运行结果

```
7 % 2 = 1
8 % 2 = 0
7 % 3 = 1
```

取余的常见用途：

- **判断奇偶**：`n % 2` 为 0 是偶数，非 0 是奇数；
- **取某一位数字**：`n % 10` 取个位；
- **循环计数**：让计数器在 $0 \sim N-1$ 之间循环；
- **时间换算**：`seconds % 60` 得到秒数。

负数取余要小心

在多数现代编译器上，`-7 % 2` 的结果是 −1，不是 1。所以判断奇数不要写 `n % 2 == 1`，而要写 `n % 2 != 0`，否则负数会被误判为偶数。C89 对负数除法和取余的符号没有强制规定，跨平台代码更要注意。

3.2.3 除零与溢出

整数除以 0 是**未定义行为**，程序通常会直接崩溃：

divzero.c ——除零

```c
#include <stdio.h>

int main(void)
{
    int a = 10;
    int b = 0;

    printf("%d\n", a / b);   /* 运行时错误 */
    return 0;
}
```

浮点数除以 0 不会崩溃，而是得到“无穷大”或“非数”（NaN），但同样不应该依赖。

整数运算还可能溢出。有符号整数溢出属于未定义行为。看一个例子：

int-overflow.c ——整数运算溢出

```c
#include <stdio.h>

int main(void)
{
    int a = 100000;
    int b = 100000;
    int c;

    c = a * b;       /* 10^10 超出了 int 的范围 */
    printf("c = %d\n", c);
    return 0;
}
```

运行结果（常见实现，不可依赖）

```
c = 1410065408
```

电控中的溢出

电机编码器计数、定时器计数、ADC 采样值累加都可能溢出。做乘加运算前，先估算最大值是否会超过类型范围；必要时用 `long`、`double` 或先做除法。

3.3 赋值运算符

3.3.1 基本赋值

`=` 是赋值，不是“等于”。它把右边的值写进左边的变量：

assign.c ——赋值

```c
#include <stdio.h>

int main(void)
{
    int a;
    int b;

    a = 10;         /* 把 10 写进 a */
    b = a;          /* 把 a 的值复制给 b */
    a = 20;         /* 再改 a，b 不受影响 */

    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

运行结果

```
a = 20, b = 10
```

赋值表达式的值就是被赋的值，所以可以连续赋值：

chain.c ——连续赋值

```c
#include <stdio.h>

int main(void)
{
    int a, b, c;

    a = b = c = 5;      /* 从右往左：c=5，b=5，a=5 */
    printf("a=%d b=%d c=%d\n", a, b, c);
    return 0;
}
```

3.3.2 复合赋值

`a += b` 等价于 `a = a + b`。常见的复合赋值有：

写法	等价于
`a += b`	`a = a + b`
`a -= b`	`a = a - b`
`a *= b`	`a = a * b`
`a /= b`	`a = a / b`
`a %= b`	`a = a % b`
`a <<= 2`	`a = a << 2`
`a &= mask`	`a = a & mask`
`a |= flag`	`a = a | flag`

复合赋值的好处是简洁，而且左边只求值一次（对指针和数组下标有意义，后面会讲）。但不要为了简洁牺牲可读性。

`a += b * c` 不是 `(a + b) * c`

`a += b * c` 等价于 `a = a + (b * c)`，因为 `*` 的优先级高于 `+=`。复合赋值运算符的优先级很低，仅高于逗号。

3.4 自增与自减

`++` 让变量加 1，`--` 让变量减 1。它们有两种写法：

写法	名称	含义
`++a`	前置	先把 a 加 1，再使用 a 的新值
`a++`	后置	先使用 a 的旧值，再把 a 加 1
`--a`	前置	先减 1，再使用
`a--`	后置	先使用，再减 1

inc.c ——前置与后置

```c
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b;

    b = ++a;        /* a 先变 6，再把 6 赋给 b */
    printf("a=%d b=%d\n", a, b);

    a = 5;
    b = a++;        /* 先把 5 赋给 b，a 再变 6 */
    printf("a=%d b=%d\n", a, b);

    return 0;
```

```
}
```

运行结果

```
a=6 b=6
a=6 b=5
```

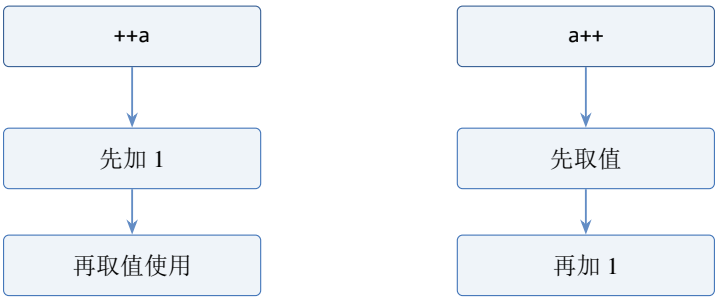


图 3.1: 前置 ++ 与后置 ++ 的执行顺序

单独使用 `a++;` 时，前置和后置效果一样，都是让 a 加 1。区别只在“参与其他运算”时体现。

不要在同一个表达式里对同一变量多次自增

`i = i++;`、`a = i++ + ++i;` 这类写法在 C 标准里属于**未定义行为**，不同编译器可能给出不同结果。原因是 C 没有规定“先算左边还是先算右边”。记住：**一个变量在一个表达式里最多被修改一次**。

拔高：为什么 `i = i++` 是未定义的

`i++` 包含两个动作：读取 i 的旧值、把 i 加 1 写回。`i = ...` 又要把结果写回 i。两次写入之间没有“顺序点”（sequence point），标准无法规定谁先谁后，于是干脆规定这是未定义行为。现代编译器打开优化后，结果可能和你想的完全不同。

3.5 关系运算符

关系运算符用来比较两个值，结果只有两种：**真（1）**或**假（0）**。

运算符	含义	例子（a=3, b=5）
>	大于	a > b 为 0
<	小于	a < b 为 1
>=	大于等于	a >= 3 为 1
<=	小于等于	a <= 2 为 0
==	等于	a == 3 为 1
!=	不等于	a != b 为 1

relational.c ——关系运算的结果

```c
#include <stdio.h>

int main(void)
{
    int a = 3;
```

```c
    int b = 5;

    printf("a < b  = %d\n", a < b);
    printf("a > b  = %d\n", a > b);
    printf("a == b = %d\n", a == b);
    printf("a != b = %d\n", a != b);
    return 0;
}
```

运行结果

```
a < b  = 1
a > b  = 0
a == b = 0
a != b = 1
```

= 和 == 是两回事

`if (a = 5)` 是赋值，把 5 写进 a，然后判断 5 是否为真（永远为真）；`if (a == 5)` 才是比较。这个错误编译器通常只会给警告，一定要看清。

不要连写比较

数学里可以写 $3 < x < 10$，C 语言里 `3 < x < 10` 的含义完全不同：先算 `3 < x` 得到 0 或 1，再拿这个 0/1 和 10 比较，结果永远为真。正确写法是 `3 < x && x < 10`。

3.6 逻辑运算符

逻辑运算符把多个条件组合起来：

运算符	名称	含义
`&&`	逻辑与	两个都为真，结果才为真
`||`	逻辑或	有一个为真，结果就为真
`!`	逻辑非	真变假，假变真

真值表：

a	b	a && b	a || b	!a
0	0	0	0	1
0	非 0	0	1	1
非 0	0	0	1	0
非 0	非 0	1	1	0

在 C 语言里，**0 为假，非 0 为真**。任何非零值（包括负数、浮点数）都算真。

3.6.1 短路求值

`&&` 和 `||` 有一个非常重要的特性：**短路**。

- `a && b`：如果 a 为假，就不再计算 b，直接得 0；
- `a || b`：如果 a 为真，就不再计算 b，直接得 1。

这个特性有两个用途：一是提高效率，二是**避免危险操作**。

short-circuit.c ——短路求值避免除零

```c
#include <stdio.h>

int main(void)
{
    int a = 10;
    int b = 0;

    if (b != 0 && a / b > 2) {
        printf("商大于 2\n");
    } else {
        printf("不满足条件\n");
    }
    return 0;
}
```

运行结果

```
不满足条件
```

因为 `b != 0` 为假，`&&` 后面的 `a / b` 根本不会被计算，所以不会除零崩溃。如果写成 `a / b > 2 && b != 0`，就会先除零，程序崩溃。

短路只保证“不被求值”，不保证“顺序好”

虽然 `&&` 和 `||` 保证从左到右求值，但其余运算符（如 `+`、`*`、函数参数）的求值顺序是未指定的。不要把有副作用的表达式塞进复杂表达式里。

3.6.2 德·摩根定律

逻辑表达式可以互相转换：

$$!(a\&\&b) \equiv (!a)||(!b), \qquad !(a||b) \equiv (!a)\&\&(!b).$$

例如“不是（年龄大于 18 且身高大于 170）”等价于“年龄不大于 18 或身高不大于 170”。化简逻辑表达式时很有用。

3.7 位运算符

位运算符直接操作整数的二进制位。它在电控里非常常用：寄存器配置、标志位、掩码、协议解析，都离不开它。

运算符	名称	作用
&	按位与	两位都为 1，结果才为 1
|	按位或	有一位为 1，结果就为 1
^	按位异或	两位不同，结果为 1
~	按位取反	0 变 1，1 变 0
<<	左移	整体左移，右边补 0
>>	右移	整体右移

3.7.1 按位与、或、异或、取反

以 8 位为例：

运算	例子	结果	说明
&	1010 1100 & 0000 1111	0000 1100	取低 4 位
|	1010 0000 | 0000 1111	1010 1111	置低 4 位为 1
^	1010 1100 ^ 1111 0000	0101 1100	翻转高 4 位
~	~1010 1100	0101 0011	按位取反

3.7.2 移位

`x << n` 把 `x` 的二进制位整体左移 `n` 位，右边补 0；`x >> n` 右移。

shift.c ——移位

```c
#include <stdio.h>

int main(void)
{
    unsigned int x = 1;

    printf("1 << 0 = %u\n", x << 0);
    printf("1 << 1 = %u\n", x << 1);
    printf("1 << 4 = %u\n", x << 4);
    printf("1 << 8 = %u\n", x << 8);
    return 0;
}
```

运行结果

```
1 << 0 = 1
1 << 1 = 2
1 << 4 = 16
1 << 8 = 256
```

左移 n 位相当于乘以 2^n，右移 n 位相当于除以 2^n（对无符号数）。但**不要用移位代替乘除来“优化”**，现代编译器比你聪明，而且移位遇到负数或溢出时会出问题。

移位位数不能超过类型宽度

对 32 位 int 左移 32 位是未定义行为。移位位数必须小于类型宽度。另外，有符号数右移时高位补 0 还是补 1 由实现决定（算术右移通常补符号位），所以位操作用 unsigned 类型最安全。

3.7.3 位操作的经典用法

位操作最常用的四个动作是：**判断某位、置位、清位、取反某位**。

bits.c ——位操作的四个动作

```c
#include <stdio.h>

int main(void)
{
    unsigned int flags = 0;

    /* 置位：把第 3 位置 1 */
    flags = flags | (1u << 3);
    printf("置位后：%u\n", flags);

    /* 判断：第 3 位是否为 1 */
    if (flags & (1u << 3)) {
        printf("第 3 位是 1\n");
    }

    /* 清位：把第 3 位置 0 */
    flags = flags & ~(1u << 3);
    printf("清位后：%u\n", flags);

    /* 翻转：把第 0 位取反 */
    flags = flags ^ (1u << 0);
    printf("翻转后：%u\n", flags);

    return 0;
}
```

运行结果

```
置位后：8
第 3 位是 1
清位后：0
翻转后：1
```

记住这四个式子

- 置位：flags |= (1u << n);
- 清位：flags &= ~(1u << n);
- 判断：if (flags & (1u << n))
- 翻转：flags ^= (1u << n);

其中 1u 表示无符号的 1，避免有符号移位的坑。

拔高：寄存器配置就是这么干的

假设某个 32 位寄存器里，第 0~3 位控制电机使能，第 4~7 位控制方向，第 8~15 位是速度。你可能会写：

register.c ——寄存器位域操作（示意）

```c
unsigned int reg = 0;

reg |= (1u << 0);                 /* 使能 */
reg = (reg & ~(0xFu << 4)) | (3u << 4);   /* 方向设为 3 */
reg = (reg & ~(0xFFu << 8)) | (200u << 8);/* 速度设为 200 */
```

这段代码不需要访问真实硬件，但思想是通用的：先清掉要改的位，再用“或”写入新值。直接赋值 reg = 3 << 4 会把其他位清零，是初学者常犯的错误。

3.8 条件运算符

?: 是 C 语言里唯一的三目运算符，形式是：

条件? 表达式 1 : 表达式 2

条件为真时取表达式 1 的值，否则取表达式 2 的值。

ternary.c ——条件运算符

```c
#include <stdio.h>

int main(void)
{
    int a = 7;
    int b = 3;
    int max;

    max = (a > b) ? a : b;
    printf("较大的数是 %d\n", max);

    printf("a 是%s\n", (a % 2 == 0) ? "偶数" : "奇数");
    return 0;
}
```

运行结果

```
较大的数是 7
a 是奇数
```

条件运算符适合“二选一”的简单场景。如果逻辑复杂，还是用 if-else 更清楚。

条件运算符的优先级很低

`a > b ? a : b + 1` 会被解析成 `a > b ? a : (b + 1)`，而不是 `(a > b ? a : b) + 1`。不确定就加括号。

3.9 逗号、sizeof 与强制类型转换

3.9.1 逗号运算符

逗号表达式依次计算每个子表达式，整个表达式的值是**最后一个子表达式的值**：

comma.c ——逗号运算符

```c
#include <stdio.h>

int main(void)
{
    int a;
    int b;

    a = (1, 2, 3);      /* 依次计算 1、2、3，a 得 3 */
    b = (a++, a + 10);  /* 先 a 加 1，再算 a+10 */

    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

运行结果

```
a = 4, b = 14
```

逗号运算符主要用于 `for` 循环的初始化和更新部分（第 5 章会看到），平时不要滥用。

3.9.2 sizeof

`sizeof` 不是函数，是运算符。它返回某个类型或变量占用的字节数，结果类型是 `size_t`（无符号整数）。

sizeof2.c ——sizeof 的用法

```c
#include <stdio.h>

int main(void)
{
    int a;
    double b;
    int arr[10];

    printf("sizeof(int)    = %d\n", (int)sizeof(int));
    printf("sizeof(a)      = %d\n", (int)sizeof(a));
    printf("sizeof(double) = %d\n", (int)sizeof(double));
    printf("sizeof(b)      = %d\n", (int)sizeof(b));
    printf("sizeof(arr)    = %d\n", (int)sizeof(arr));
```

```
    printf("数组元素个数   = %d\n", (int)(sizeof(arr) / sizeof(arr[0])));
    return 0;
}
```

运行结果（典型 32 位 int 环境）

```
sizeof(int)    = 4
sizeof(a)      = 4
sizeof(double) = 8
sizeof(b)      = 8
sizeof(arr)    = 40
数组元素个数   = 10
```

`sizeof(arr) / sizeof(arr[0])` 是求数组元素个数的经典写法，第 6 章会详细讲。

3.9.3 强制类型转换

强制类型转换的语法是 `(类型) 表达式`：

cast.c ——强制类型转换

```c
#include <stdio.h>

int main(void)
{
    int a = 7;
    int b = 2;
    double c;

    c = a / b;              /* 整数除法，得 3，再转 double */
    printf("c = %f\n", c);

    c = (double)a / b;      /* 先把 a 转成 double，再除 */
    printf("c = %f\n", c);

    printf("(int)3.99 = %d\n", (int)3.99);  /* 截断小数部分 */
    return 0;
}
```

运行结果

```
c = 3.000000
c = 3.500000
(int)3.99 = 3
```

强制转换可能丢失精度

`(int)3.99` 得 3，不是 4；`(char)300` 会溢出，结果由实现决定。强制转换是“我确定要这样”，不是“帮我修好”。

3.10 优先级与结合性总表

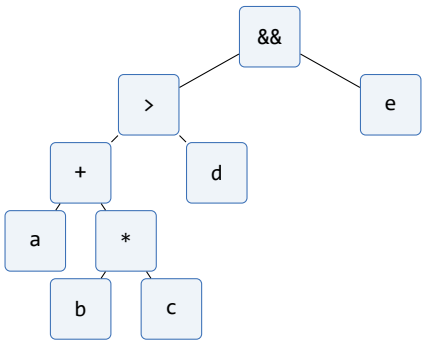
下面这张表从高到低排列。同一行的运算符优先级相同。

优先级	运算符	含义	结合性
1	`() [] -> .`	函数调用、下标、成员	左
2	`! ~ ++ -- + - * & (类型) sizeof`	单目运算	右
3	`* / %`	乘除、取余	左
4	`+ -`	加减	左
5	`<< >>`	移位	左
6	`< <= > >=`	关系	左
7	`== !=`	相等/不等	左
8	`&`	按位与	左
9	`^`	按位异或	左
10	`|`	按位或	左
11	`&&`	逻辑与	左
12	`||`	逻辑或	左
13	`?:`	条件	右
14	`= += -= *= /= %= <<= >>= &= ^= |=`	赋值	右
15	`,`	逗号	左

例题 3.1 分析表达式 `a + b * c > d && e` 的求值顺序。

解 按优先级从高到低：

1. `b * c`（乘法优先级 3）；
2. `a + (b * c)`（加法优先级 4）；
3. `(a + b * c) > d`（关系优先级 6，结果是 0 或 1）；
4. `((a + b * c) > d) && e`（逻辑与优先级 11）。



3.11 类型转换

不同类型的数据一起运算时，C 会自动做类型转换。规则可以概括为“向精度更高、范围更大的类型看齐”。

3.11.1 整型提升

`char`、`short` 在参与运算前，会先被提升为 `int`：

promote.c ——整型提升

```c
#include <stdio.h>

int main(void)
{
    char a = 100;
    char b = 100;
    int c;

    c = a + b;      /* a、b 先提升为 int，再相加，不会溢出 */
    printf("c = %d\n", c);
    return 0;
}
```

运行结果

```
c = 200
```

如果直接用一个 char 存 200，会溢出（char 范围通常是 −128 ~ 127）。这里因为先提升成 int 再相加，所以结果正确。

3.11.2 通常算术转换

当 int 和 double 运算时，int 会被转成 double；当 int 和 unsigned int 运算时，int 会被转成 unsigned int。后者是很多隐蔽 bug 的来源：

unsigned-trap.c ——有符号与无符号混算

```c
#include <stdio.h>

int main(void)
{
    int a = -1;
    unsigned int b = 1;

    if (a < b) {
        printf("a < b\n");
    } else {
        printf("a >= b\n");
    }
    return 0;
}
```

运行结果（常见实现）

```
a >= b
```

为什么？因为 a 被转换成了 unsigned int，−1 变成 4294967295，当然大于 1。这类问题在比较 sizeof 的结果时特别常见：

sizeof-trap.c ——sizeof 比较陷阱

```c
#include <stdio.h>

int main(void)
{
    int i;

    for (i = 10; i >= 0; i--) {
        if (i < sizeof(int)) {     /* i 被转成无符号，比较结果可能反了 */
            printf("%d\n", i);
        }
    }
    return 0;
}
```

避免有符号/无符号混算

尽量让参与比较的两个变量类型一致；如果必须混用，先显式转换，或者把 `sizeof` 的结果强制转成 `int`。

3.12 求值顺序与副作用

3.12.1 什么是副作用

表达式除了产生值，还可能修改环境，这叫**副作用**。赋值、自增自减、函数调用都可能有副作用。

C 标准只规定了少数几个“顺序点”（sequence point），在顺序点之前的所有副作用必须完成。顺序点包括：

- 每条完整语句结束的分号；
- `&&`、`||`、`?:`、逗号运算符的求值处；
- 函数调用前（参数求值完成后）。

函数参数的求值顺序不确定

`printf("%d %d", i++, i++);` 的结果在不同编译器上可能不同。不要写这种代码。同理，`f(i++, i)` 也不可预测。

3.12.2 一个经典例子

sequence.c ——未定义行为示例

```c
#include <stdio.h>

int main(void)
{
    int i = 1;
    int a;

    a = i++ + i++;      /* 未定义行为！不要这样写 */
    printf("a = %d, i = %d\n", a, i);
    return 0;
}
```

这段代码在不同编译器、不同优化级别下可能输出不同结果。正确的写法是拆成多条语句：

sequence-ok.c ——拆开写

```c
#include <stdio.h>

int main(void)
{
    int i = 1;
    int a;

    a = i;
    i++;
    a = a + i;
    i++;
    printf("a = %d, i = %d\n", a, i);
    return 0;
}
```

一条语句只做一件事

这不是语法要求，而是可靠性要求。一条语句里修改多个变量、或者对同一变量多次修改，都会让代码难以理解和调试。

3.13 综合例题

例题 3.2 不用 `if`，用表达式判断一个整数 `n` 是否为偶数。

解 `n % 2 == 0`。结果为 1 表示偶数，0 表示奇数。注意负数的情况，用 `== 0` 判断比 `== 1` 安全。

例题 3.3 用位运算判断整数 `n` 的第 5 位是否为 1。

解 `(n >> 5) & 1` 或 `n & (1 << 5)`。前者得到 0 或 1，后者得到 0 或 32（非零即真）。

例题 3.4 把秒数 `total` 转换成“时: 分: 秒”的形式。

解 思路如下：

time.c ——秒数转换

```c
#include <stdio.h>

int main(void)
{
    int total = 3725;
    int h, m, s;

    h = total / 3600;
    m = total % 3600 / 60;
    s = total % 60;

    printf("%d:%02d:%02d\n", h, m, s);
    return 0;
}
```

运行结果

```
1:02:05
```

total % 3600 先去掉整小时，再除以 60 得到分钟；total % 60 直接得到秒。

例题 3.5 用异或运算交换两个整数（不使用临时变量）。

解思路如下：

xor-swap.c ——异或交换

```c
#include <stdio.h>

int main(void)
{
    int a = 3;
    int b = 5;

    a = a ^ b;
    b = a ^ b;      /* 此时 b 变成原来的 a */
    a = a ^ b;      /* 此时 a 变成原来的 b */

    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

运行结果

```
a = 5, b = 3
```

异或交换只适合炫技

这个技巧在面试题里很常见，但实际工程中不推荐：可读性差，而且如果 a 和 b 是同一个变量，会把值清零。老老实实用临时变量。

例题 3.6 写出判断闰年的逻辑表达式（先不考虑 `if` 的写法）。

解 闰年的规则是：能被 4 整除但不能被 100 整除，或者能被 400 整除。表达式为：

```
(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)
```

下一章我们就用这个表达式写完整的判断程序。

3.14 本章小结

主题	要点
算术	整数除法会截断；`%` 只能用于整数；除零危险；溢出危险
赋值	`=` 是赋值，`==` 是比较；复合赋值优先级低
自增自减	前置先加后用，后置先用后加；同一表达式不要多次修改同一变量
关系	结果是 0 或 1；不要连写比较；浮点不要直接用 `==`
逻辑	非 0 即真；`&&`、`||` 短路；德·摩根定律
位运算	与、或、异或、取反、移位；用 `unsigned` 更安全
条件运算符	二选一的简写；优先级低，注意括号
类型转换	整型提升、有符号/无符号混算的坑
求值顺序	只有顺序点保证顺序；避免副作用叠加

下一章预告

有了关系运算和逻辑运算，我们就可以写“如果……就……”了。下一章学习 `if`、`if-else`、`else if` 和 `switch`，让程序学会“明辨是非”。

第三章 练习

1. 计算 `7 / 2`、`7 % 2`、`-7 / 2`、`-7 % 2` 的值，并解释。
2. 写出判断整数 `n` 是奇数的表达式（要求对负数也正确）。
3. 写出判断整数 `n` 能被 3 整除但不能被 5 整除的表达式。
4. 用位运算实现：判断 `n` 的第 0 位是否为 1；把第 2 位置 1；把第 3 位清零。
5. 分析表达式 `a = b = c + 1` 的求值顺序。
6. 下面代码有什么问题？

q6.c ——找问题

```c
#include <stdio.h>

int main(void)
{
    int i = 0;
    int a;
    a = i++ + ++i;
    printf("%d %d\n", a, i);
    return 0;
}
```

7. 写一个表达式，判断字符 c 是不是小写字母。
8. 解释为什么 `-1 < 1U` 的结果是假。
9. 用条件运算符求三个整数中的最大值。
10. 思考题：为什么 `x << 2` 不一定等于 `x * 4`？举一个反例。

第四章　分支：明辨是非

内容提要

- if：满足条件才执行
- if-else：二选一
- else if：多路分支与分段函数
- 嵌套分支与“就近匹配”的 else
- switch：按整数常量分情况处理
- 分支的常见错误与综合例题

「计算机的“聪明”，来自你替它写好的每一个“如果”。」

顺序结构只能一条路走到黑。可现实中的程序要会看情况：温度过高就报警，按键按下就启动，成绩及格就通过，除数不为零才做除法。这就需要**分支结构**。

C 语言的分支工具只有两类：`if` 语句和 `switch` 语句。这一章把它们的语法、套路、坑和例题一次讲透。

4.1　为什么需要分支

先看一个没有分支的程序：

no-branch.c ——只会往前走的程序

```c
#include <stdio.h>

int main(void)
{
    int score = 85;

    printf("你的成绩是 %d 分\n", score);
    printf("恭喜你，通过了！\n");
    return 0;
}
```

无论 score 是多少，它都会打印“恭喜你，通过了”。这显然不合理。我们希望程序这样思考：

如果成绩大于等于 60，就打印“通过”；否则打印“未通过”。

用流程图表示：

流程图里的**菱形**表示判断，有“是/否”两条出口；**矩形**表示处理；**圆角矩形**表示开始/结束；**平行四边形**表示输入输出。后面我们还会用这套符号。

4.2　if 语句

4.2.1　基本语法

```
if (条件) {
    语句;
}
```

执行逻辑：计算括号里的条件，如果结果为**真**（非 0），就执行花括号里的语句；否则跳过。

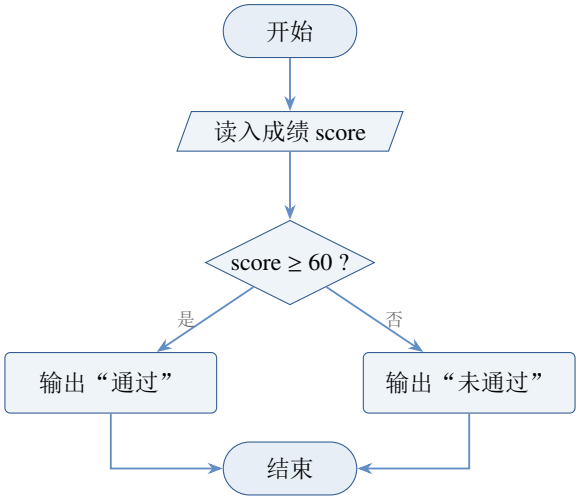


图 4.1: 带分支的流程图：条件成立走一条路，不成立走另一条路

if-basic.c ——最简单的 if

```c
#include <stdio.h>

int main(void)
{
    int score = 85;

    if (score >= 60) {
        printf("恭喜你，通过了！\n");
    }

    printf("程序结束\n");
    return 0;
}
```

运行结果

```
恭喜你，通过了！
程序结束
```

如果把 score 改成 50，第一行就不会打印，但“程序结束”仍然会打印——因为 if 只控制它后面那一组花括号。

4.2.2 条件可以是任意表达式

if 只关心“是不是 0”：非 0 为真，0 为假。所以下面这些写法都合法：

if-truth.c ——条件的真真假假

```c
#include <stdio.h>

int main(void)
{
    int a = 5;
```

```c

    if (a) {
        printf("a 非零，条件为真\n");
    }

    if (a - 5) {
        printf("这一行不会打印\n");
    }

    if (a > 0 && a < 10) {
        printf("a 在 0 到 10 之间\n");
    }

    if (!a) {
        printf("这一行也不会打印\n");
    }

    return 0;
}
```

运行结果

```
a 非零，条件为真
a 在 0 到 10 之间
```

条件写清楚，别炫技

`if (a)` 虽然合法，但读者要猜“a 是什么含义”。更好的写法是 `if (a != 0)` 或 `if (count > 0)`，把意图写出来。

4.2.3 单条语句可以省略花括号

如果 `if` 后面只有一条语句，花括号可以省略：

if-nobrace.c ——省略花括号

```c
#include <stdio.h>

int main(void)
{
    int a = 10;

    if (a > 0)
        printf("a 是正数\n");

    return 0;
}
```

省略花括号是 bug 的温床

if-bug.c ——少写花括号的后果

```c
#include <stdio.h>

int main(void)
{
    int a = -5;

    if (a > 0)
        printf("a 是正数\n");
        printf("这一行无论 a 正负都会打印\n");   /* 缩进骗了你 */

    return 0;
}
```

第二行看起来在 if 里面，实际上不在。**本书约定：即使只有一条语句，也写花括号**。多打两个字符，少一个通宵调试。

4.3 if-else 语句

4.3.1 二选一

```
if (条件) {
    条件为真时执行;
} else {
    条件为假时执行;
}
```

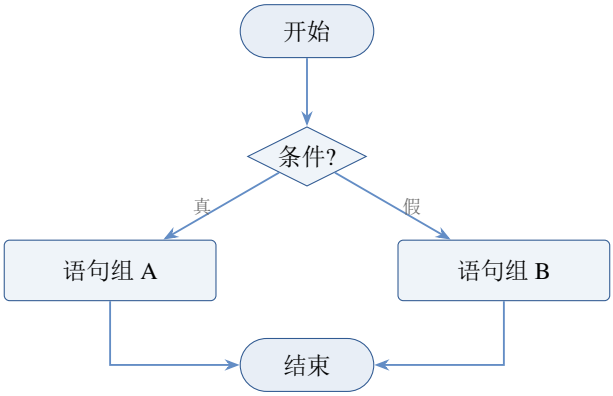


图 4.2: if-else 的执行路径

if-else.c ——判断正负数

```c
#include <stdio.h>

int main(void)
{
```

```c
    int n;

    printf("请输入一个整数: ");
    scanf("%d", &n);

    if (n > 0) {
        printf("%d 是正数\n", n);
    } else {
        printf("%d 不是正数\n", n);
    }

    return 0;
}
```

运行示例

```
请输入一个整数: -7
-7 不是正数
```

注意“不是正数”包括 0 和负数。如果想区分 0、正数、负数，就需要下一节的多分支。

4.3.2 例：求绝对值

例题 4.1 输入一个整数，输出它的绝对值。

解思路如下：

abs.c ——求绝对值

```c
#include <stdio.h>

int main(void)
{
    int n;
    int abs_n;

    printf("请输入一个整数: ");
    scanf("%d", &n);

    if (n < 0) {
        abs_n = -n;
    } else {
        abs_n = n;
    }

    printf("|%d| = %d\n", n, abs_n);
    return 0;
}
```

运行示例

```
请输入一个整数：-9
|-9| = 9
```

也可以用条件运算符写成一行：`abs_n = (n < 0) ? -n : n;`。

4.4 else if：多路分支

4.4.1 语法与执行顺序

当情况不止两种时，可以用 `else if` 串成一条“阶梯”：

```
if (条件 1) { ... }
else if (条件 2) { ... }
else if (条件 3) { ... }
else { ... }
```

执行逻辑：从上往下依次判断，**一旦某个条件成立，就执行对应的语句组，然后跳出整个阶梯**；如果所有条件都不成立，执行最后的 `else`。

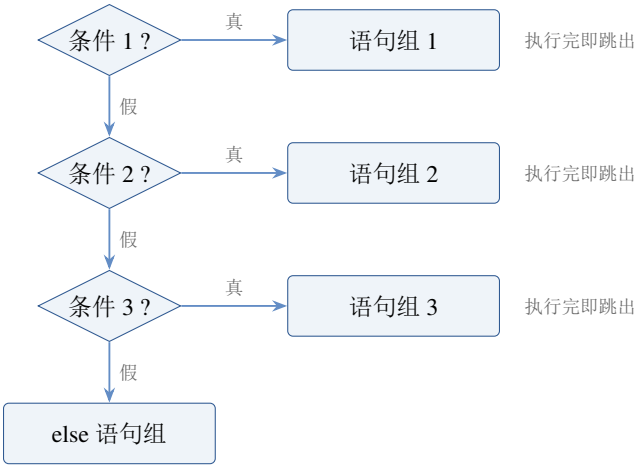


图 4.3: `else if` 阶梯：从上到下，命中即停

4.4.2 例：成绩等级

例题 4.2 输入一个 0~100 的成绩，输出等级：90 及以上为 A，80~89 为 B，70~79 为 C，60~69 为 D，60 以下为 E。非法成绩给出提示。

解思路如下：

grade.c ——成绩等级

```c
#include <stdio.h>

int main(void)
{
    int score;

    printf("请输入成绩（0-100）：");
```

```c
    scanf("%d", &score);

    if (score < 0 || score > 100) {
        printf("成绩不合法\n");
    } else if (score >= 90) {
        printf("等级 A\n");
    } else if (score >= 80) {
        printf("等级 B\n");
    } else if (score >= 70) {
        printf("等级 C\n");
    } else if (score >= 60) {
        printf("等级 D\n");
    } else {
        printf("等级 E\n");
    }

    return 0;
}
```

运行示例

```
请输入成绩（0-100）：85
等级 B
```

注意判断顺序：先排除非法值，再从高到低判断。如果把 `score >= 60` 放在最前面，85 会先命中 60 这一档，直接输出 D，后面的判断永远不会执行。

else if 的顺序很重要

多分支是“命中即停”的。范围判断要从宽到窄或从窄到宽，保证先命中的是更具体的条件。写完后用边界值（59、60、89、90）测试一遍。

4.4.3 例：分段函数

例题 4.3 输入 x，计算

$$y=\begin{cases}-1, & x<0,\\ 0, & x=0,\\ 1, & x>0.\end{cases}$$

解思路如下：

sign.c ——符号函数

```c
#include <stdio.h>

int main(void)
{
    double x;
    int y;
```

```c

    printf("请输入 x: ");
    scanf("%lf", &x);

    if (x < 0) {
        y = -1;
    } else if (x == 0) {
        y = 0;
    } else {
        y = 1;
    }

    printf("y = %d\n", y);
    return 0;
}
```

运行示例

```
请输入 x: -2.5
y = -1
```

浮点数不要直接用 == 判断

`x == 0` 对精确输入的 0 可以工作，但如果是计算出来的值（例如 `0.1 + 0.2 - 0.3`），它可能是一个极小的非零数，判断就会失败。更稳妥的写法是 `fabs(x) < 1e-9`（需要 `math.h`）。

4.5 嵌套分支

4.5.1 一个分支里再套一个分支

分支可以嵌套。例如判断一个年份是否是闰年，再判断这一年的二月有多少天：

nested.c ——嵌套 if

```c
#include <stdio.h>

int main(void)
{
    int year;
    int is_leap;

    printf("请输入年份: ");
    scanf("%d", &year);

    if (year % 4 == 0) {
        if (year % 100 != 0) {
            is_leap = 1;
        } else {
            if (year % 400 == 0) {
```

```
                is_leap = 1;
            } else {
                is_leap = 0;
            }
        }
    } else {
        is_leap = 0;
    }

    if (is_leap) {
        printf("%d 年是闰年，二月有 29 天\n", year);
    } else {
        printf("%d 年不是闰年，二月有 28 天\n", year);
    }

    return 0;
}
```

嵌套让逻辑更接近人的思考方式，但层数太多会难以阅读。一般建议嵌套不超过三层；超过时，考虑用“提前返回”或拆成函数（第 9 章之后会讲）。

4.5.2 悬空的 else

看下面这段代码，else 到底和哪个 if 配对？

dangling.c ——悬空的 else

```
#include <stdio.h>

int main(void)
{
    int a = 1;
    int b = -1;

    if (a > 0)
        if (b > 0)
            printf("a、b 都是正数\n");
    else
        printf("b 不是正数\n");

    return 0;
}
```

运行结果

```
b 不是正数
```

C 语言的规则是：**else 与离它最近的、还没有配对的 if 配对**。所以这里的 else 属于内层 if (b > 0)，而不是外层。缩进在这件事上完全不起作用——编译器只看语法。

用花括号消灭歧义

上面的代码如果本意是“a 不大于 0 时打印”，应该写成：

dangling-fixed.c ——加花括号

```c
#include <stdio.h>

int main(void)
{
    int a = 1;
    int b = -1;

    if (a > 0) {
        if (b > 0) {
            printf("a、b 都是正数\n");
        }
    } else {
        printf("a 不是正数\n");
    }

    return 0;
}
```

只要 **if** 或 **else** 后面有嵌套，就一定加花括号。

4.5.3 例：三个数中的最大值

例题 4.4 输入三个整数，输出最大值。

解思路如下：

max3.c ——三个数的最大值

```c
#include <stdio.h>

int main(void)
{
    int a, b, c, max;

    printf("请输入三个整数：");
    scanf("%d%d%d", &a, &b, &c);

    if (a > b) {
        if (a > c) {
            max = a;
        } else {
            max = c;
        }
    } else {
        if (b > c) {
            max = b;
```

```
        } else {
            max = c;
        }
    }

    printf("最大值是 %d\n", max);
    return 0;
}
```

也可以先令 max = a，再依次比较：

max3-simple.c ——更简洁的写法

```
#include <stdio.h>

int main(void)
{
    int a, b, c, max;

    printf("请输入三个整数: ");
    scanf("%d%d%d", &a, &b, &c);

    max = a;
    if (b > max) {
        max = b;
    }
    if (c > max) {
        max = c;
    }

    printf("最大值是 %d\n", max);
    return 0;
}
```

后一种写法更容易扩展到 n 个数，也更不容易出错。第 5 章学循环后，我们会用循环处理任意多个数。

4.6 switch 语句

4.6.1 基本语法

当分支依据是“一个整数表达式的若干个确定取值”时，switch 比一长串 else if 更清楚。

```
switch (表达式) {
    case 常量 1: 语句; break;
    case 常量 2: 语句; break;
    default: 语句; break;
}
```

- switch 后面的表达式必须是**整数类型**（int、char、enum 等），不能是 double 或字符串；

- `case` 后面必须是**编译期常量**，不能是变量；
- `break` 表示“跳出 `switch`”，没有 `break` 会继续往下执行（称为“贯穿”，fall-through）；
- `default` 处理所有没有匹配到的情况，位置可以放在任意处，但习惯上放最后。

switch-basic.c ——根据数字输出星期

```c
#include <stdio.h>

int main(void)
{
    int day;

    printf("请输入 1-7: ");
    scanf("%d", &day);

    switch (day) {
    case 1:
        printf("星期一\n");
        break;
    case 2:
        printf("星期二\n");
        break;
    case 3:
        printf("星期三\n");
        break;
    case 4:
        printf("星期四\n");
        break;
    case 5:
        printf("星期五\n");
        break;
    case 6:
        printf("星期六\n");
        break;
    case 7:
        printf("星期日\n");
        break;
    default:
        printf("输入不合法\n");
        break;
    }

    return 0;
}
```

运行示例

```
请输入 1-7: 3
星期三
```

4.6.2 贯穿（fall-through）

如果某个 case 后面没有 break，程序会继续执行下一个 case 的语句，直到遇到 break 或 switch 结束。

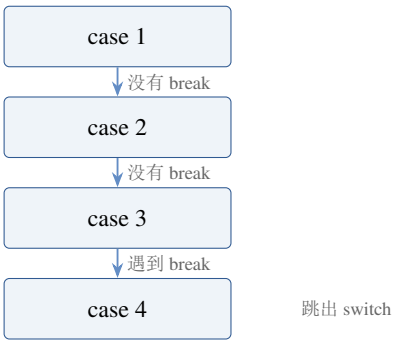


图 4.4: 没有 break 时会“贯穿”到下一个 case

漏写 break 是最常见的 switch 错误

switch-bug.c ——漏写 break

```c
#include <stdio.h>

int main(void)
{
    int n = 1;

    switch (n) {
    case 1:
        printf("one\n");
    case 2:
        printf("two\n");
    case 3:
        printf("three\n");
        break;
    default:
        printf("other\n");
        break;
    }
    return 0;
}
```

输出是三行：one、two、three，而不是只有 one。

4.6.3 故意利用贯穿

有些场景下贯穿反而能简化代码，例如判断某个月有多少天：

days.c ——月份天数

```c
#include <stdio.h>

int main(void)
```

```c
{
    int month;
    int year;
    int days;

    printf("请输入年份和月份：");
    scanf("%d%d", &year, &month);

    switch (month) {
    case 1: case 3: case 5: case 7:
    case 8: case 10: case 12:
        days = 31;
        break;
    case 4: case 6: case 9: case 11:
        days = 30;
        break;
    case 2:
        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
            days = 29;
        } else {
            days = 28;
        }
        break;
    default:
        printf("月份不合法\n");
        return 1;
    }

    printf("%d 年 %d 月有 %d 天\n", year, month, days);
    return 0;
}
```

运行示例

```
请输入年份和月份：2024 2
2024 年 2 月有 29 天
```

把多个 case 写在一起、共享同一段处理代码，是贯穿的正当用法。但要在代码里写注释说明“这是故意的”，避免后来的人误以为你漏了 break。

4.6.4 例：简易计算器

例题 4.5 输入两个数和四则运算符（+、-、*、/），输出运算结果。除法要判断除数是否为 0。

解思路如下：

calc.c ——简易计算器

```c
#include <stdio.h>

int main(void)
{
    double a, b;
    char op;

    printf("请输入表达式（如 3.5 + 2）: ");
    scanf("%lf %c %lf", &a, &op, &b);

    switch (op) {
    case '+':
        printf("%.2f\n", a + b);
        break;
    case '-':
        printf("%.2f\n", a - b);
        break;
    case '*':
        printf("%.2f\n", a * b);
        break;
    case '/':
        if (b == 0.0) {
            printf("除数不能为 0\n");
        } else {
            printf("%.2f\n", a / b);
        }
        break;
    default:
        printf("不支持的运算符: %c\n", op);
        break;
    }

    return 0;
}
```

运行示例

```
请输入表达式（如 3.5 + 2）: 3.5 * 2
7.00
```

格式串"%lf %c %lf" 中，%c 前面的空格很重要：它会跳过前面数字后面的空白字符（包括空格和换行），否则%c 可能读到一个空格。

4.7 分支的常见错误

错误写法	问题
`if (a = 5)`	赋值，不是比较；条件永远为真
`if (a == 5);`	多了一个分号，`if` 后面变成了空语句
`if (a > 0)` 后接多条语句但不加花括号	只有第一条受 `if` 控制
`if (0 < x < 10)`	连写比较，结果永远为真
`if (a & b)`	按位与，不是逻辑与；应写 `&&`
`if (x == 0.1)`	浮点数直接比较，可能不相等
`switch (x)` 且 `x` 是 `double`	`switch` 只接受整数
`case 1.5:`	`case` 必须是整数常量
`default` 后忘记 `break`	通常无害，但不规范

`if` 后面的分号

if-semicolon.c ——致命的空语句

```c
if (a > 0);          /* 分号让 if 提前结束 */
{
    printf("a 是正数\n");   /* 这段代码永远执行 */
}
```

编译器通常不会报错，甚至不会警告。看到 `if (...)` 后面紧跟分号，立刻警觉。

条件里的“范围”怎么写

“`x` 在 0 到 10 之间”要写成 `x > 0 && x < 10`；“`x` 不在 0 到 10 之间”写成 `x <= 0 || x >= 10`。注意逻辑或不是“与”的简单取反，写完后用边界值测试。

4.8 综合例题

4.8.1 例：闰年判断

例题 4.6 输入年份，判断是否为闰年。

解 规则：能被 4 整除且不能被 100 整除，或者能被 400 整除。

leap.c ——闰年判断

```c
#include <stdio.h>

int main(void)
{
    int year;

    printf("请输入年份：");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
        printf("%d 年是闰年\n", year);
```

```
    } else {
        printf("%d 年不是闰年\n", year);
    }

    return 0;
}
```

运行示例

```
请输入年份：2000
2000 年是闰年
```

测试用例建议覆盖：1900（不是）、2000（是）、2024（是）、2023（不是）。

4.8.2 例：一元二次方程

例题 4.7 输入 a、b、c，求方程 $ax^2+bx+c=0$ 的实根。要考虑 $a=0$、判别式大于 0、等于 0、小于 0 四种情况。

解思路如下：

quadratic.c ——一元二次方程

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double a, b, c;
    double delta;
    double x1, x2;

    printf("请输入 a b c: ");
    scanf("%lf%lf%lf", &a, &b, &c);

    if (a == 0.0) {
        if (b == 0.0) {
            printf("不是二次方程\n");
        } else {
            printf("x = %.2f\n", -c / b);
        }
    } else {
        delta = b * b - 4 * a * c;
        if (delta > 0.0) {
            x1 = (-b + sqrt(delta)) / (2 * a);
            x2 = (-b - sqrt(delta)) / (2 * a);
            printf("两个实根: x1 = %.2f, x2 = %.2f\n", x1, x2);
        } else if (delta == 0.0) {
            x1 = -b / (2 * a);
            printf("两个相等实根: x = %.2f\n", x1);
        } else {
```

```
            printf("没有实根\n");
        }
    }

    return 0;
}
```

运行示例

```
请输入 a b c: 1 -3 2
两个实根: x1 = 2.00, x2 = 1.00
```

笔记 在 Linux 上用 GCC 编译含 math.h 的程序时，需要在命令末尾加 -lm 链接数学库：gcc quadratic.c -o quadratic -lm。

4.8.3 例：三角形判断

例题 4.8 输入三条边长，判断能否构成三角形；如果能，再判断是等边、等腰还是普通三角形。

解 思路如下：

triangle.c ——三角形判断

```c
#include <stdio.h>

int main(void)
{
    double a, b, c;

    printf("请输入三条边长: ");
    scanf("%lf%lf%lf", &a, &b, &c);

    if (a + b > c && a + c > b && b + c > a) {
        if (a == b && b == c) {
            printf("等边三角形\n");
        } else if (a == b || a == c || b == c) {
            printf("等腰三角形\n");
        } else {
            printf("普通三角形\n");
        }
    } else {
        printf("不能构成三角形\n");
    }

    return 0;
}
```

运行示例

```
请输入三条边长：3 3 5
等腰三角形
```

浮点数判断相等的隐患

这里用 `==` 判断边长相等，对整数输入没问题；如果边长是计算出来的小数，应改用 `fabs(a - b) < 1e-9`。

4.8.4 例：出租车计费

例题 4.9 某城市出租车计费规则：3 公里以内（含）收费 13 元；超过 3 公里的部分，每公里 2.5 元；超过 10 公里的部分，每公里加收 50% 空驶费（即 3.75 元/公里）。输入行驶公里数，输出费用。

解思路如下：

taxi.c ——分段计费

```c
#include <stdio.h>

int main(void)
{
    double km;
    double fee;

    printf("请输入公里数：");
    scanf("%lf", &km);

    if (km <= 3.0) {
        fee = 13.0;
    } else if (km <= 10.0) {
        fee = 13.0 + (km - 3.0) * 2.5;
    } else {
        fee = 13.0 + 7.0 * 2.5 + (km - 10.0) * 2.5 * 1.5;
    }

    printf("费用 = %.2f 元\n", fee);
    return 0;
}
```

运行示例

```
请输入公里数：12
费用 = 38.00
```

分段计费是 `else if` 的典型场景：每一段都只计算“超出部分”，千万不要重复计算前面的里程。

4.9 拔高：让分支更好读

4.9.1 卫语句：先处理异常，再处理正常

当嵌套很深时，可以把“不满足条件就提前结束”的情况放在前面：

guard.c ——卫语句

```c
#include <stdio.h>

int main(void)
{
    int score;

    printf("请输入成绩：");
    scanf("%d", &score);

    if (score < 0 || score > 100) {
        printf("成绩不合法\n");
        return 1;
    }

    if (score >= 60) {
        printf("通过\n");
    } else {
        printf("未通过\n");
    }

    return 0;
}
```

先排除非法输入，后面的逻辑就不用再考虑它，代码更扁平。

4.9.2 表驱动：用数组代替一长串 `if`

如果分支是“输入一个编号，输出一个固定结果”，可以把结果放进数组，用下标直接取。第 6 章学完数组后，你会看到这种写法有多简洁。

4.9.3 分支预测

CPU 遇到 `if` 时会“猜”走哪条路，猜对了就快，猜错了要清空流水线。所以**把最可能发生的情况放在前面**，有时能提升性能。但对初学者来说，先保证正确和可读，性能是后话。

4.10 本章小结

主题	要点
`if`	条件非 0 则执行；建议总是加花括号
`if-else`	二选一；两个分支必有一个执行
`else if`	多路分支，从上到下命中即停；注意判断顺序
嵌套	分支里套分支；`else` 与最近未配对的 `if` 匹配
`switch`	整数表达式 + 常量 `case`；`break` 跳出；可故意贯穿
常见错误	`=` 与 `==`、多余分号、漏花括号、连写比较
综合应用	闰年、方程、三角形、计费、计算器

下一章预告

分支让程序会“选”，但还不会“重复”。如果要计算 1 到 100 的和，你总不能写一百行加法。下一章学习循环，让程序学会“重复再重复”。

第四章 练习

1. 输入一个整数，判断它是正数、负数还是零。
2. 输入一个年份，判断是否为闰年（分别用 `if` 和逻辑表达式两种写法）。
3. 输入一个字符，判断它是大写字母、小写字母、数字还是其他字符。
4. 输入三个整数，按从小到大的顺序输出。
5. 输入学生成绩，输出等级（A/B/C/D/E），并处理非法输入。
6. 用 `switch` 实现：输入月份，输出该月所属的季节（春/夏/秋/冬）。
7. 输入一个一元二次方程的系数，输出实根（考虑各种情况）。
8. 输入三条边，判断能否构成三角形，并判断类型。
9. 解释 `if (a = 5)` 和 `if (a == 5)` 的区别。
10. 思考题：为什么 `switch` 的 `case` 后面不能是变量？如果一定要根据变量分支，应该怎么写？

第五章 循环：重复再重复

内容提要

- 为什么需要循环：从“写一百行”到“写三行”
- while、do-while、for 三种循环
- C89 的循环变量声明规矩
- break 与 continue
- 嵌套循环与经典算法：累加、素数、斐波那契、辗转相除
- 循环的写法、边界与常见错误

「聪明人把重复交给机器，把思考留给自己。」

如果要计算 1 + 2 + 3 + ··· + 100，你可以写一百行 `sum = sum + i;`，但没人会这么做。循环就是为“重复做同一件事”而生的。

本章会学三种循环：`while`、`do-while`、`for`。它们能力相同，只是适合的场景不同。学完后，你会用循环解决求和、阶乘、素数、斐波那契、最大公约数等一大批问题。

5.1 为什么需要循环

先看一个笨办法：

sum100-hard.c ——笨办法：写一百行

```c
#include <stdio.h>

int main(void)
{
    int sum = 0;

    sum = sum + 1;
    sum = sum + 2;
    sum = sum + 3;
    /* ... 中间省略 96 行 ... */
    sum = sum + 100;

    printf("sum = %d\n", sum);
    return 0;
}
```

这段代码有 100 行几乎一样的语句。如果要求和到 10000，就得写一万行；如果要求到 1 亿，根本没法写。更糟糕的是，只要需求变一下（比如只加偶数），就要改一百处。

循环的思路是：把“重复的动作”写一次，让机器重复执行。

sum100.c ——用循环求和

```c
#include <stdio.h>

int main(void)
{
    int i;
```

```c
    int sum = 0;

    for (i = 1; i <= 100; i++) {
        sum = sum + i;
    }

    printf("1 + 2 + ... + 100 = %d\n", sum);
    return 0;
}
```

运行结果

```
1 + 2 + ... + 100 = 5050
```

三行循环，替代了一百行。这就是循环的威力。

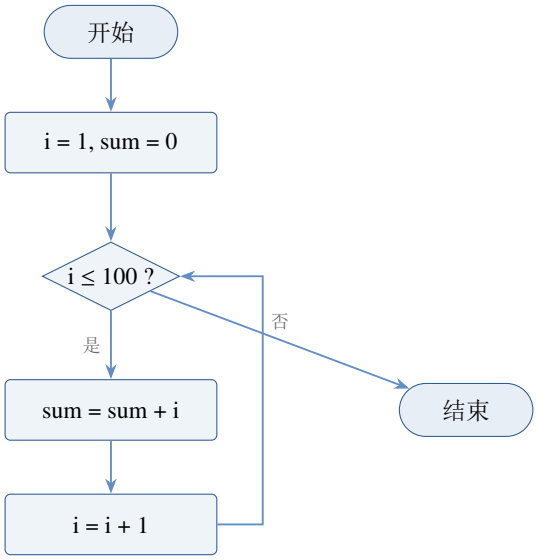


图 5.1: 累加循环的流程图：条件成立就重复“加一自增一再判断”

5.2 while 循环

5.2.1 基本语法

```
while (条件) {
    循环体;
}
```

执行逻辑：先判断条件，为真就执行循环体，执行完再判断条件……直到条件为假，循环结束。

while.c ——while 倒计时

```c
#include <stdio.h>

int main(void)
{
```

```
    int i = 5;

    while (i > 0) {
        printf("%d\n", i);
        i--;            /* 千万别忘了改变循环变量 */
    }

    printf("发射！\n");
    return 0;
}
```

运行结果

```
5
4
3
2
1
发射！
```

死循环

如果循环体里没有让条件最终变假的语句，循环会永远执行下去：

infinite.c ——死循环

```c
#include <stdio.h>

int main(void)
{
    int i = 1;

    while (i <= 5) {
        printf("%d\n", i);
        /* 忘了 i++，条件永远为真 */
    }
    return 0;
}
```

运行后终端会无限刷 1，只能按 Ctrl+C 强制结束。

5.2.2 用 while 实现累加

while-sum.c ——while 求和

```c
#include <stdio.h>

int main(void)
{
    int i = 1;
```

```c
    int sum = 0;

    while (i <= 100) {
        sum = sum + i;
        i++;
    }

    printf("sum = %d\n", sum);
    return 0;
}
```

运行结果

```
sum = 5050
```

写 while 循环时，按“四步法”检查：

1. **初始化**：循环变量从几开始？（`i = 1`）
2. **条件**：什么时候继续？（`i <= 100`）
3. **循环体**：每次重复做什么？（`sum += i`）
4. **更新**：怎么让循环走向结束？（`i++`）

少任何一步，循环都可能不工作或停不下来。

5.2.3 循环次数不确定时用 while

while 最适合“不知道要循环多少次，只知道什么时候停”的场景。例如不断读入数字，直到读到 0 为止：

read-until-zero.c ——读到 0 为止

```c
#include <stdio.h>

int main(void)
{
    int n;
    int sum = 0;

    printf("请输入整数（输入 0 结束）: ");

    scanf("%d", &n);
    while (n != 0) {
        sum = sum + n;
        scanf("%d", &n);
    }

    printf("总和 = %d\n", sum);
    return 0;
}
```

运行示例

```
请输入整数（输入 0 结束）：3 5 7 0
总和 = 15
```

注意 scanf 出现了两次：循环前读一次，循环体里再读一次。这是“读取—判断—处理—再读取”的经典模式。

5.3 for 循环

5.3.1 基本语法

```
for (初始化; 条件; 更新) {
    循环体;
}
```

for 把 while 的四步压缩到一行，最适合“次数确定”的循环。

for.c ——for 求和

```c
#include <stdio.h>

int main(void)
{
    int i;
    int sum = 0;

    for (i = 1; i <= 100; i++) {
        sum = sum + i;
    }

    printf("sum = %d\n", sum);
    return 0;
}
```

5.3.2 执行顺序

for 的执行顺序是：

1. 执行**初始化**（只执行一次）；
2. 判断**条件**，为假则结束；
3. 执行**循环体**；
4. 执行**更新**；
5. 回到第 2 步。

C89：循环变量必须提前声明

C89 不允许在 for 的初始化部分声明变量。下面两段代码，左边是 C99 写法，右边才是本书要求的 C89 写法：

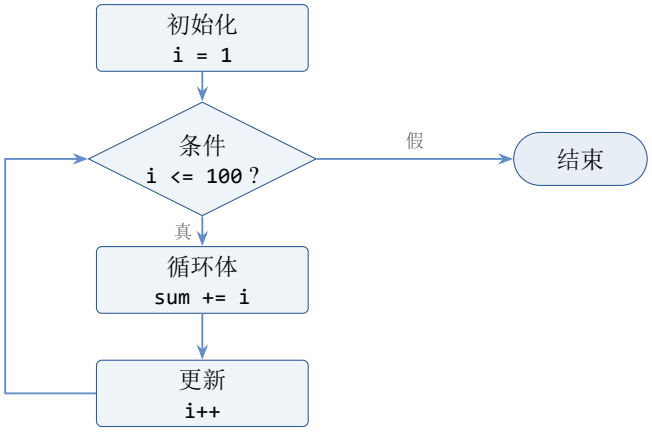


图 5.2: for 的执行顺序：初始化一次，然后“条件—循环体—更新”循环

wrong-for.c / right-for.c ——两种写法的对比

```c
/* C99: 不允许（本书不使用） */
for (int i = 0; i < n; i++) {
    ...
}

/* C89: 正确写法 */
int i;
for (i = 0; i < n; i++) {
    ...
}
```

把 int i; 写在函数（或语句块）开头，这是本书所有代码的写法。

5.3.3 for 的灵活写法

for 的三个部分都可以为空或使用逗号表达式：

for-flex.c ——for 的灵活用法

```c
#include <stdio.h>

int main(void)
{
    int i;
    int j;

    /* 从 10 倒数到 1 */
    for (i = 10; i >= 1; i--) {
        printf("%d ", i);
    }
    printf("\n");

    /* 每次加 2: 打印 0 2 4 6 8 */
```

```c
    for (i = 0; i < 10; i += 2) {
        printf("%d ", i);
    }
    printf("\n");

    /* 逗号表达式：同时控制两个变量 */
    for (i = 0, j = 10; i < j; i++, j--) {
        printf("(%d,%d) ", i, j);
    }
    printf("\n");

    return 0;
}
```

运行结果

```
10 9 8 7 6 5 4 3 2 1
0 2 4 6 8
(0,10) (1,9) (2,8) (3,7) (4,6)
```

分号陷阱

`for (i = 0; i < 10; i++);` 末尾的分号会让循环体变成**空语句**，循环空转十次，后面的代码只执行一次。这是很常见的错误。

5.4 do-while 循环

do-while 的特点是：**先执行一次循环体，再判断条件**。所以循环体至少执行一次。

```
do {
    循环体;
} while (条件);
```

do-while.c ——至少执行一次

```c
#include <stdio.h>

int main(void)
{
    int n;

    do {
        printf("请输入 1-10 之间的数：");
        scanf("%d", &n);
    } while (n < 1 || n > 10);

    printf("你输入了 %d\n", n);
    return 0;
}
```

运行示例

```
请输入 1-10 之间的数：0
请输入 1-10 之间的数：11
请输入 1-10 之间的数：5
你输入了 5
```

这种“至少问一次，不合法就再问”的场景，do-while 比 while 更自然。

digits.c ——数一数有几位

```c
#include <stdio.h>

int main(void)
{
    int n;
    int count = 0;

    printf("请输入一个非负整数：");
    scanf("%d", &n);

    do {
        count++;
        n = n / 10;
    } while (n != 0);

    printf("共有 %d 位\n", count);
    return 0;
}
```

运行示例

```
请输入一个非负整数：2026
共有 4 位
```

如果 n 是 0，do-while 也能正确输出 1 位；换成 while (n != 0) 则一次都不会执行，结果为 0。

5.5 break 与 continue

5.5.1 break：提前跳出循环

break 立即结束当前循环，跳到循环后面的语句。

break.c ——找到就停

```c
#include <stdio.h>

int main(void)
{
    int i;
```

```c

    for (i = 1; i <= 100; i++) {
        if (i * i > 50) {
            printf("第一个平方大于 50 的数是 %d\n", i);
            break;
        }
    }

    return 0;
}
```

运行结果

```
第一个平方大于 50 的数是 8
```

break 也常用于“搜索”：找到目标就停止，不必把剩下的都检查一遍。

5.5.2 continue：跳过本次，继续下一次

continue 跳过循环体剩下的部分，直接进入下一次循环（对 for 来说，会先执行更新部分）。

continue.c ——只打印奇数

```c
#include <stdio.h>

int main(void)
{
    int i;

    for (i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;       /* 偶数跳过 */
        }
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

运行结果

```
1 3 5 7 9
```

break 只跳出最内层循环

在嵌套循环里，break 只结束它所在的那一层，外层循环继续。如果要一次跳出多层，需要额外的标志变量或 goto（不推荐）。

表 5.1: break 与 continue 的区别

语句	作用	之后执行什么
break	立即结束当前循环	循环后面的语句
continue	跳过本次剩余部分	下一次循环（for 先更新）

5.6 嵌套循环

循环体里再写一个循环，叫**嵌套循环**。外层循环每执行一次，内层循环完整地执行一轮。

nested-loop.c ——嵌套循环

```c
#include <stdio.h>

int main(void)
{
    int i, j;

    for (i = 1; i <= 3; i++) {
        for (j = 1; j <= 4; j++) {
            printf("(%d,%d) ", i, j);
        }
        printf("\n");
    }

    return 0;
}
```

运行结果

```
(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)
```

外层 i 每取一个值，内层 j 就从 1 到 4 走一遍，所以共执行 $3 \times 4 = 12$ 次。

5.6.1 例：九九乘法表

例题 5.1 输出九九乘法表（下三角）。

解思路如下：

multiply.c ——九九乘法表

```c
#include <stdio.h>

int main(void)
{
    int i, j;

    for (i = 1; i <= 9; i++) {
```

```
        for (j = 1; j <= i; j++) {
            printf("%d*%d=%-2d ", j, i, i * j);
        }
        printf("\n");
    }

    return 0;
}
```

运行结果（节选）

```
1*1=1
1*2=2  2*2=4
1*3=3  2*3=6  3*3=9
...
1*9=9  2*9=18 3*9=27 ... 9*9=81
```

内层循环的上限是 i，所以每行输出 i 项，形成下三角。%-2d 表示左对齐、占两格，让各列大致对齐。

5.6.2 例：星号三角形

例题 5.2 输入行数 n，输出一个由星号组成的直角三角形。

解思路如下：

stars.c ——星号三角形

```
#include <stdio.h>

int main(void)
{
    int n;
    int i, j;

    printf("请输入行数：");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

运行示例（n = 5）

```
*
**
***
****
*****
```

如果要做“金字塔”（居中对齐），需要在星号前打印空格：第 i 行先打印 n - i 个空格，再打印 2*i - 1 个星号。这个练习留给你自己完成。

5.7 循环的经典算法

5.7.1 累加与累乘

累加的模式是 sum = sum + 项，累乘的模式是 product = product * 项。注意累乘的初始值必须是 1，累加的初始值必须是 0。

factorial.c ——阶乘

```c
#include <stdio.h>

int main(void)
{
    int n;
    int i;
    int fact = 1;

    printf("请输入 n: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %d\n", n, fact);
    return 0;
}
```

运行示例

```
请输入 n: 10
10! = 3628800
```

阶乘增长极快

13! = 6227020800，已经超过 32 位 int 的范围。计算更大的阶乘需要 long long（C99）或自行实现大整数。不要假设 int 能装下所有结果。

5.7.2 最大值与最小值

max-min.c ——求最大值和最小值

```c
#include <stdio.h>

int main(void)
{
    int n, i;
    int x;
    int max, min;

    printf("请输入数据个数：");
    scanf("%d", &n);

    printf("请输入 %d 个整数：", n);
    scanf("%d", &x);
    max = x;
    min = x;

    for (i = 2; i <= n; i++) {
        scanf("%d", &x);
        if (x > max) {
            max = x;
        }
        if (x < min) {
            min = x;
        }
    }

    printf("最大值 = %d，最小值 = %d\n", max, min);
    return 0;
}
```

运行示例

```
请输入数据个数：5
请输入 5 个整数：3 9 1 7 5
最大值 = 9，最小值 = 1
```

注意第一个数要先读进来作为 `max` 和 `min` 的初始值，而不是把初值设成 0——否则全是负数时最大值会错。

5.7.3 判断素数

素数是只能被 1 和自身整除的大于 1 的整数。最直接的判断方法是试除：

prime.c ——判断素数

```c
#include <stdio.h>

```

```c
int main(void)
{
    int n;
    int i;
    int is_prime = 1;

    printf("请输入一个整数：");
    scanf("%d", &n);

    if (n <= 1) {
        is_prime = 0;
    } else {
        for (i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                is_prime = 0;
                break;
            }
        }
    }

    if (is_prime) {
        printf("%d 是素数\n", n);
    } else {
        printf("%d 不是素数\n", n);
    }

    return 0;
}
```

运行示例

```
请输入一个整数：97
97 是素数
```

为什么循环条件是 `i * i <= n`？因为如果 n 有大于 $\sqrt{n}$ 的因子，那么它一定还有一个小于 $\sqrt{n}$ 的因子。检查到 $\sqrt{n}$ 就够了。

5.7.4 打印 100 以内的素数

primes100.c ——100 以内的素数

```c
#include <stdio.h>

int main(void)
{
    int n, i, is_prime;

    for (n = 2; n <= 100; n++) {
```

```
        is_prime = 1;
        for (i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                is_prime = 0;
                break;
            }
        }
        if (is_prime) {
            printf("%d ", n);
        }
    }
    printf("\n");

    return 0;
}
```

运行结果

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

这是嵌套循环的典型用法：外层枚举候选数，内层判断素数。

5.7.5 斐波那契数列

斐波那契数列：$F_1 = 1$，$F_2 = 1$，$F_n = F_{n-1} + F_{n-2}$。

fib.c ——前 20 项斐波那契数

```c
#include <stdio.h>

int main(void)
{
    int i;
    int a = 1;
    int b = 1;
    int c;

    printf("%d %d ", a, b);
    for (i = 3; i <= 20; i++) {
        c = a + b;
        printf("%d ", c);
        a = b;
        b = c;
    }
    printf("\n");

    return 0;
}
```

运行结果

```
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765
```

这个“滚动变量”的技巧很常用：只保留最近两个值，不断往前推。

5.7.6 辗转相除法求最大公约数

gcd.c ——最大公约数

```c
#include <stdio.h>

int main(void)
{
    int a, b;
    int t;

    printf("请输入两个正整数: ");
    scanf("%d%d", &a, &b);

    while (b != 0) {
        t = a % b;
        a = b;
        b = t;
    }

    printf("最大公约数 = %d\n", a);
    return 0;
}
```

运行示例

```
请输入两个正整数: 48 18
最大公约数 = 6
```

辗转相除法（欧几里得算法）的思想是：$\gcd(a, b) = \gcd(b, a \bmod b)$，直到余数为 0，此时的除数就是最大公约数。

5.7.7 水仙花数

水仙花数是指一个三位数，其各位数字的立方和等于它本身，例如 $153 = 1^3 + 5^3 + 3^3$。

narcissus.c ——水仙花数

```c
#include <stdio.h>

int main(void)
{
    int n;
    int a, b, c;
```

```
    for (n = 100; n <= 999; n++) {
        a = n / 100;
        b = n / 10 % 10;
        c = n % 10;
        if (a * a * a + b * b * b + c * c * c == n) {
            printf("%d ", n);
        }
    }
    printf("\n");

    return 0;
}
```

运行结果

```
153 370 371 407
```

5.7.8 反转整数

reverse-int.c ——反转整数

```
#include <stdio.h>

int main(void)
{
    int n;
    int rev = 0;

    printf("请输入一个整数: ");
    scanf("%d", &n);

    while (n != 0) {
        rev = rev * 10 + n % 10;
        n = n / 10;
    }

    printf("反转后 = %d\n", rev);
    return 0;
}
```

运行示例

```
请输入一个整数: 12345
反转后 = 54321
```

这个算法对负数也能工作，但反转后的值可能溢出（例如反转 2147483647），实际工程中要检查。

5.8 怎样写好一个循环

5.8.1 四步检查法

写循环前，先回答四个问题：

1. **从哪里开始**？（初始化）
2. **做到什么时候停**？（条件）
3. **每次做什么**？（循环体）
4. **怎么走向结束**？（更新）

写完后，用三个值测试：**最小规模**（0 次或 1 次）、**典型值**、**边界值**。例如求和 1 到 100，要测试 `n = 0`、`n = 1`、`n = 100`。

5.8.2 边界：<= 还是 <

写法	循环次数	说明
`for (i = 1; i <= n; i++)`	n 次	包含 n
`for (i = 0; i < n; i++)`	n 次	不包含 n，适合数组下标
`for (i = 1; i < n; i++)`	$n-1$ 次	少一次，常犯的 off-by-one 错误

数组下标从 0 开始时，`for (i = 0; i < n; i++)` 是最标准的写法（第 6 章会反复用）。

5.8.3 循环不变量

循环不变量是每次循环开始时都成立的性质。例如求和循环的不变量是：

$$\text{在每轮循环开始时，sum 等于 } 1+2+\cdots+(i-1)。$$

验证一下：初始 `i = 1` 时，`sum = 0`，空和确实为 0；每轮把 `i` 加进 `sum`，然后 `i` 加 1，不变量继续成立；循环结束时 `i = 101`，`sum` 就是 $1+\cdots+100$。

养成想“不变量”的习惯，能帮你快速检查循环写得对不对。

5.9 循环的常见错误

错误	后果
忘了更新循环变量	死循环
`for (...);` 多写分号	循环体为空，后面的语句只执行一次
`while (i <= n)` 里 `n` 被修改	循环次数不可预测
用浮点数做循环变量	累积误差可能让循环多一次或少一次
`unsigned` 变量写 `i >= 0`	永远为真，死循环
循环体内修改循环变量	次数混乱
嵌套循环里 `break`	只跳出内层

无符号变量的倒序循环

unsigned-loop.c ——死循环陷阱

```c
#include <stdio.h>

int main(void)
{
    unsigned int i;

    for (i = 10; i >= 0; i--) {
        printf("%u ", i);   /* 0 之后再减 1 会变成 4294967295 */
    }
    return 0;
}
```

这个循环停不下来。正确写法是用 `int`，或者写成 `for (i = 10; i > 0; i--)` 并单独处理 0。

浮点数循环变量

float-loop.c ——浮点循环的隐患

```c
for (x = 0.0; x != 1.0; x += 0.1) {
    ...
}
```

0.1 在二进制里不能精确表示，累加十次后 x 可能不等于 1.0，循环停不下来。循环次数应该用整数控制。

5.10 综合例题

例题 5.3 输入 n，计算 $1+\frac{1}{2}+\frac{1}{3}+\cdots+\frac{1}{n}$，保留 4 位小数。

解思路如下：

harmonic.c ——调和级数

```c
#include <stdio.h>

int main(void)
{
    int n, i;
    double sum = 0.0;

    printf("请输入 n: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        sum = sum + 1.0 / i;
    }

    printf("sum = %.4f\n", sum);
```

```
    return 0;
}
```

注意 `1.0 / i` 而不是 `1 / i`。后者是整数除法，除了 `i = 1` 以外全是 0。

例题 5.4 输入若干个整数（以 0 结束），统计正数、负数的个数，并计算平均值。

解思路如下：

stat.c ——统计正负数

```c
#include <stdio.h>

int main(void)
{
    int n;
    int positive = 0;
    int negative = 0;
    int sum = 0;
    int count = 0;

    printf("请输入整数（0 结束）: ");
    scanf("%d", &n);

    while (n != 0) {
        if (n > 0) {
            positive++;
        } else {
            negative++;
        }
        sum = sum + n;
        count++;
        scanf("%d", &n);
    }

    printf("正数 %d 个，负数 %d 个\n", positive, negative);
    if (count > 0) {
        printf("平均值 = %.2f\n", (double)sum / count);
    } else {
        printf("没有输入有效数据\n");
    }

    return 0;
}
```

这里用 `(double)sum / count` 做浮点除法；同时处理了“一个数都没输入”的情况，避免除以 0。

例题 5.5 输出 1000 以内的所有完数。完数是指一个数恰好等于它的真因子之和，例如 $6 = 1 + 2 + 3$。

解思路如下：

perfect.c ——完数

```c
#include <stdio.h>

int main(void)
{
    int n, i, sum;

    for (n = 2; n <= 1000; n++) {
        sum = 0;
        for (i = 1; i < n; i++) {
            if (n % i == 0) {
                sum = sum + i;
            }
        }
        if (sum == n) {
            printf("%d ", n);
        }
    }
    printf("\n");

    return 0;
}
```

运行结果

```
6 28 496
```

例题 5.6 输入一个整数，输出它的二进制表示（从最高位到最低位）。

解 可以用“掩码”从高位往低位检查。以 32 位 unsigned int 为例：

to-binary.c ——输出二进制

```c
#include <stdio.h>

int main(void)
{
    unsigned int n;
    int i;

    printf("请输入一个非负整数：");
    scanf("%u", &n);

    for (i = 31; i >= 0; i--) {
        if (n & (1u << i)) {
            printf("1");
        } else {
            printf("0");
        }
        if (i % 8 == 0) {
```

```
            printf(" ");
        }
    }
    printf("\n");

    return 0;
}
```

运行示例

```
请输入一个非负整数：182
00000000 00000000 00000000 10110110
```

`1u << i` 生成只有第 `i` 位为 1 的掩码，`n &` 掩码判断该位是否为 1。这是位运算和循环的经典结合。

5.11 拔高：循环的效率与递归

5.11.1 时间复杂度直觉

循环执行次数决定了程序要花多少时间。求 1 到 n 的和需要循环 n 次，记作 $O(n)$；判断一个数是否为素数只需要试除到 $\sqrt{n}$，记作 $O(\sqrt{n})$。数据量增大时，$O(\sqrt{n})$ 比 $O(n)$ 快得多。

n	$O(n)$ 次操作	$O(\sqrt{n})$ 次操作
100	100	10
10000	10000	100
1000000	1000000	1000

5.11.2 递归：循环的另一种写法

有些问题用“自己调用自己”描述更自然，例如阶乘：

fact-rec.c ——递归求阶乘

```c
#include <stdio.h>

int factorial(int n)
{
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main(void)
{
    printf("%d\n", factorial(5));
    return 0;
}
```

递归和循环可以互相转换。初学阶段先把循环写熟，递归等学完函数（后续章节）再深入。

5.11.3 goto：不推荐，但要知道

`goto` 可以跳到同一函数内的标号处，常用于“多层循环一次跳出”或“统一清理资源”。但它会破坏代码结构，现代工程中很少使用。知道有这个东西即可，不要主动用。

5.12 本章小结

主题	要点
`while`	先判断后执行；适合次数不确定的循环
`for`	初始化、条件、更新集中在一行；适合次数确定的循环
`do-while`	先执行一次再判断；适合“至少做一次”的场景
C89 循环变量	必须在循环外声明
`break`	立即结束当前循环
`continue`	跳过本次，进入下一次
嵌套循环	外层每走一步，内层走完整一轮
写循环	初始化、条件、循环体、更新，四步缺一不可
常见错误	死循环、多余分号、边界差一、无符号倒序

下一章预告

有了循环，我们可以处理大量数据了。但数据放哪里？如果每读一个数就丢了，循环再厉害也没用。下一章学习**数组**——把一堆同类型的数据整齐地存在一起。

第五章 练习

1. 用 `for`、`while`、`do-while` 三种循环分别求 $1 + 2 + \cdots + 100$。
2. 输入 n，求 $n!$，并测试 $n = 10$、$n = 13$ 的结果，解释差异。
3. 输入 10 个整数，求最大值、最小值和平均值。
4. 输出 1~100 中所有能被 3 或 5 整除的数，每行 5 个。
5. 判断一个数是否为素数，输出 100 以内的所有素数。
6. 输出前 30 项斐波那契数列。
7. 输入两个正整数，用辗转相除法求最大公约数和最小公倍数。
8. 输出所有三位水仙花数。
9. 输入一个整数，输出它的位数、各位数字之和、逆序数。
10. 思考题：为什么 `for (i = 0; i <= n; i++)` 在处理长度为 `n` 的数组时会越界？应该怎么改？

第六章 数组：优雅的数据存取方式

内容提要

- 为什么需要数组：一批同类型数据的家
- 一维数组的定义、初始化、下标与遍历
- 数组在内存中的连续排布
- 求最值、逆序、统计、查找等经典算法
- 二维数组与矩阵
- 数组的边界、退化与常见错误

「把一百个数据排好队，你只需要记住队首的位置。」

前面的程序一次只能处理一两个数。可现实中我们经常要处理一批数据：全班 50 个成绩、一帧 128 个采样点、一张 3×4 的矩阵。如果每个数据都用一个变量，代码会变得无法维护。

数组就是为“一批同类型数据”准备的容器。它让数据排成连续的一队，用一个名字和下标来访问。这一章会把数组的“形”和“神”都讲清楚：既会用它，也知道它在内存里长什么样。

6.1 为什么需要数组

假设要读入 10 个成绩并求平均分。不用数组，你可能写成这样：

scores-ugly.c ——没有数组的笨办法

```c
#include <stdio.h>

int main(void)
{
    int s1, s2, s3, s4, s5, s6, s7, s8, s9, s10;
    int sum;

    printf("请输入 10 个成绩：");
    scanf("%d%d%d%d%d%d%d%d%d%d",
          &s1, &s2, &s3, &s4, &s5,
          &s6, &s7, &s8, &s9, &s10);

    sum = s1 + s2 + s3 + s4 + s5 + s6 + s7 + s8 + s9 + s10;
    printf("平均分 = %.2f\n", sum / 10.0);
    return 0;
}
```

这段代码能跑，但问题很明显：

- 变量太多，读起来累；
- 想求最大值、排序、找某个分数，几乎没法写；
- 如果成绩变成 100 个，代码就没法维护了。

用数组，只需要一个名字 `scores` 和一个下标：

scores-array.c ——用数组读入 10 个成绩

```c
#include <stdio.h>

int main(void)
```

```
{
    int scores[10];
    int i;
    int sum = 0;

    printf("请输入 10 个成绩: ");
    for (i = 0; i < 10; i++) {
        scanf("%d", &scores[i]);
    }

    for (i = 0; i < 10; i++) {
        sum = sum + scores[i];
    }

    printf("平均分 = %.2f\n", sum / 10.0);
    return 0;
}
```

运行示例

```
请输入 10 个成绩: 90 85 77 92 88 60 95 73 81 100
平均分 = 84.10
```

> 数组 = 一段连续的内存 + 一个名字 + 一组从 0 开始的下标。

6.2 一维数组

6.2.1 定义

```
类型数组名 [元素个数];
```

例如：

define-array.c ——数组的定义

```
int a[10];          /* 10 个 int，下标 0 到 9 */
double b[5];        /* 5 个 double */
char name[20];      /* 20 个 char，第 9 章会用作字符串 */
```

几个要点：

- 元素个数必须是**编译期常量**，不能是变量（C89 不支持变长数组）；
- 下标从 **0** 开始，到**元素个数 – 1** 结束；
- 数组名是一个整体，代表这段连续内存的首地址（第 8 章会展开）。

C89：数组长度必须是常量

vla.c ——错误示范（C99 才允许）

```c
int n = 10;
int a[n];       /* C89 不允许：n 不是编译期常量 */
```

C89 里请写成 `int a[10];`。如果确实需要运行时决定长度，应该用 `malloc` 动态分配（第 8 章讲指针时介绍），或者定义一个足够大的数组。

6.2.2 内存布局：连续排布

数组最重要的性质是：**所有元素在内存中连续存放**。下图是一个 `int a[5]` 的内存示意（假设 `int` 占 4 字节，首地址为 0x1000）：

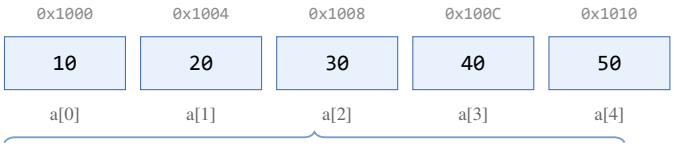


图 6.1: `int a[5]` 的内存布局：元素一个挨一个

因为连续，所以第 `i` 个元素的地址可以由首地址算出来：

$$\&a[i] = \&a[0] + i \times \mathrm{sizeof(int)}.$$

这个公式是后面理解指针的关键。现在你只需要记住：**下标就是“第几个”，地址是“在哪里”，两者可以互相换算**。

6.2.3 初始化

init-array.c ——数组的初始化

```c
int a[5] = {10, 20, 30, 40, 50};  /* 全部初始化 */
int b[5] = {1, 2};                /* 部分初始化，其余为 0 */
int c[5] = {0};                   /* 全部为 0 的常用写法 */
int d[]  = {1, 2, 3};             /* 省略长度，编译器数出 3 */
```

- 部分初始化时，剩下的元素自动补 0；
- `{0}` 是“全部清零”的惯用写法；
- 省略长度时，编译器根据初值个数确定大小。

局部数组不会自动清零

`int a[10];` 只是分配了空间，里面的值是随机的“垃圾值”。如果要用，必须先赋值或初始化。只有全局数组和静态数组才会自动清零。

6.2.4 下标访问与遍历

traverse.c ——数组的遍历

```c
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int i;

    for (i = 0; i < 5; i++) {
        printf("a[%d] = %d\n", i, a[i]);
    }

    return 0;
}
```

运行结果

```
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
```

遍历的标准写法是：

traverse-template.c ——遍历模板

```c
int i;
for (i = 0; i < 5; i++) {
    /* 使用 a[i] */
}
```

注意条件是 `i < 5`，不是 `i <= 5`。`a[5]` 已经越界了！

6.2.5 数组大小与元素个数

array-size.c ——数组占多少字节

```c
#include <stdio.h>

int main(void)
{
    int a[10];

    printf("sizeof(a)    = %d\n", (int)sizeof(a));
    printf("sizeof(a[0]) = %d\n", (int)sizeof(a[0]));
    printf("元素个数     = %d\n", (int)(sizeof(a) / sizeof(a[0])));
```

```
10
11      return 0;
12  }
```

运行结果（典型 32 位 int 环境）

```
sizeof(a)    = 40
sizeof(a[0]) = 4
元素个数     = 10
```

`sizeof(a) / sizeof(a[0])` 是求数组元素个数的经典写法。它只在“数组名还是数组”的时候有效，一旦数组名退化成指针（传给函数后），这个写法就会失效——第 8 章会详细解释。

6.3 数组的常见操作

6.3.1 例：求最大值及其下标

例题 6.1 读入 10 个整数，输出最大值以及它在数组中的下标。

解思路如下：

max-index.c ——最大值与下标

```c
#include <stdio.h>

int main(void)
{
    int a[10];
    int i;
    int max;
    int max_index = 0;

    printf("请输入 10 个整数：");
    for (i = 0; i < 10; i++) {
        scanf("%d", &a[i]);
    }

    max = a[0];
    for (i = 1; i < 10; i++) {
        if (a[i] > max) {
            max = a[i];
            max_index = i;
        }
    }

    printf("最大值 = %d，下标 = %d\n", max, max_index);
    return 0;
}
```

运行示例

```
请输入 10 个整数：3 9 1 7 9 5 2 8 6 4
最大值 = 9，下标 = 1
```

如果有多个相同的最大值，这个程序返回第一个。想返回最后一个，把 > 改成 >= 即可。

6.3.2 例：数组逆序

例题 6.2 把数组中的元素原地逆序，例如 {1,2,3,4,5} 变成 {5,4,3,2,1}。

解 用两个下标，一个从头往后，一个从尾往前，交换后向中间靠拢：

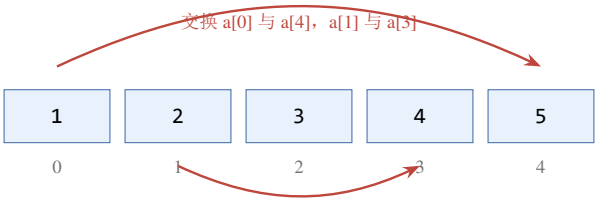


图 6.2: 双下标向中间靠拢，完成原地逆序

reverse-array.c ——原地逆序

```c
#include <stdio.h>

int main(void)
{
    int a[10];
    int i, j, temp;
    int n = 10;

    printf("请输入 10 个整数：");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (i = 0, j = n - 1; i < j; i++, j--) {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }

    printf("逆序后：");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```

运行示例

```
请输入 10 个整数：1 2 3 4 5 6 7 8 9 10
逆序后：10 9 8 7 6 5 4 3 2 1
```

循环条件 `i < j` 保证中间的元素不会被重复交换。

6.3.3 例：统计每个数字出现的次数

例题 6.3 输入一个非负整数，统计它各个数字（0~9）出现的次数。

解 用数组 `count[10]` 当下标为 0~9 的“计数器”：

digit-count.c ——数字频次统计

```c
#include <stdio.h>

int main(void)
{
    int n;
    int digit;
    int count[10] = {0};
    int i;

    printf("请输入一个非负整数：");
    scanf("%d", &n);

    if (n == 0) {
        count[0] = 1;
    }
    while (n > 0) {
        digit = n % 10;
        count[digit]++;
        n = n / 10;
    }

    for (i = 0; i < 10; i++) {
        if (count[i] > 0) {
            printf("数字 %d 出现了 %d 次\n", i, count[i]);
        }
    }

    return 0;
}
```

运行示例

```
请输入一个非负整数：20260226
数字 0 出现了 2 次
数字 2 出现了 4 次
数字 6 出现了 2 次
```

用“值当下标”是数组最常用的技巧之一。统计、计数、查表都靠它。

6.3.4 例：线性查找

例题 6.4 在数组中查找某个值，找到则输出下标，找不到输出提示。

解思路如下：

linear-search.c ——线性查找

```c
#include <stdio.h>

int main(void)
{
    int a[10] = {3, 9, 1, 7, 5, 2, 8, 6, 4, 0};
    int key;
    int i;
    int found = -1;

    printf("请输入要查找的数：");
    scanf("%d", &key);

    for (i = 0; i < 10; i++) {
        if (a[i] == key) {
            found = i;
            break;
        }
    }

    if (found >= 0) {
        printf("找到了，下标是 %d\n", found);
    } else {
        printf("没有找到\n");
    }

    return 0;
}
```

线性查找从头到尾依次比较，最坏要比较 n 次。

6.3.5 例：二分查找

例题 6.5 在一个**已排序**的数组中查找某个值。

解二分查找每次比较中间元素，根据大小关系把查找范围缩小一半：

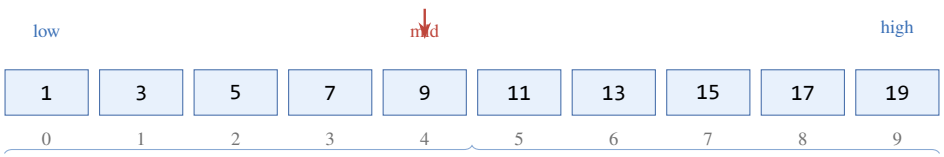


图 6.3: 二分查找：比较 mid，然后丢掉一半

binary-search.c ——二分查找

```c
#include <stdio.h>

int main(void)
{
    int a[10] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
    int key;
    int low = 0;
    int high = 9;
    int mid;
    int found = -1;

    printf("请输入要查找的数：");
    scanf("%d", &key);

    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) {
            found = mid;
            break;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (found >= 0) {
        printf("找到了，下标是 %d\n", found);
    } else {
        printf("没有找到\n");
    }

    return 0;
}
```

二分查找的前提是数组已经有序。它的比较次数是 $O(\log n)$，比线性查找快得多：100 万个元素最多比较约 20 次。

6.4 二维数组

6.4.1 定义与初始化

二维数组可以看成“数组的数组”，例如 `int a[3][4]` 表示 3 行 4 列，共 12 个元素。

define-2d.c ——二维数组的定义与初始化

```c
int a[3][4];                    /* 3 行 4 列，未初始化 */
int b[2][3] = {                 /* 按行初始化 */
    {1, 2, 3},
    {4, 5, 6}
};
int c[2][3] = {1, 2, 3, 4, 5, 6};   /* 也可以写成一维形式，按行填充 */
int d[3][4] = {{0}};                /* 全部清零 */
```

访问元素用两个下标：`a[i][j]`，其中 `i` 是行，`j` 是列，都从 0 开始。

6.4.2 内存布局：按行连续

二维数组在内存里仍然是**连续**的，按“先行后列”排列：

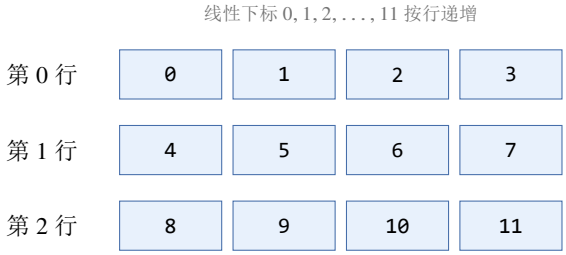


图 6.4: `int a[3][4]` 按行存放：第 0 行 4 个，接着第 1 行 4 个……

元素 `a[i][j]` 的地址是：

$$\&a[i][j] = \&a[0][0] + (i \times 4 + j) \times \texttt{sizeof(int)}.$$

所以“按行遍历”通常比“按列遍历”更快——因为前者访问的内存是连续的，更符合缓存的工作方式。

6.4.3 二维数组的遍历

traverse-2d.c ——遍历二维数组

```c
#include <stdio.h>

int main(void)
{
    int a[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    int i, j;

    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            printf("%4d", a[i][j]);
        }
        printf("\n");
```

```
    }

    return 0;
}
```

运行结果

```
  1   2   3   4
  5   6   7   8
  9  10  11  12
```

6.4.4 例：矩阵加法

例题 6.6 输入两个 3×4 的矩阵，输出它们的和。

解思路如下：

matrix-add.c ——矩阵加法

```c
#include <stdio.h>

int main(void)
{
    int a[3][4];
    int b[3][4];
    int c[3][4];
    int i, j;

    printf("请输入矩阵 A（3x4）：\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("请输入矩阵 B（3x4）：\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            scanf("%d", &b[i][j]);
        }
    }

    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("A + B =\n");
```

```
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            printf("%4d", c[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

6.4.5 例：矩阵转置

例题 6.7 输入一个 3×3 矩阵，输出它的转置（行列互换）。

解思路如下：

transpose.c ——矩阵转置

```
#include <stdio.h>

int main(void)
{
    int a[3][3];
    int i, j;

    printf("请输入 3x3 矩阵：\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("转置后：\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%4d", a[j][i]);
        }
        printf("\n");
    }

    return 0;
}
```

注意输出时用的是 a[j][i]，而不是 a[i][j]。

6.5 数组的常见错误

错误	后果
下标越界（`a[10]` 用于 `int a[10]`）	未定义行为，可能崩溃或悄悄破坏其他数据
数组未初始化就读取	得到随机值
用 `=` 拷贝数组	编译错误；数组不能整体赋值
数组长度用变量	C89 编译错误
遍历条件写成 `i <= n`	多访问一个元素，越界
`sizeof(a)` 用在函数参数上	得到的是指针大小，不是数组大小

数组越界不会报错

C 语言不会检查数组下标是否越界。`a[10]` 会去读写数组后面的内存，可能：

- 什么都没发生（最危险，掩盖了 bug）；
- 覆盖了别的变量，导致诡异的结果；
- 触发段错误，程序崩溃。

写循环时一定要反复确认边界：`i < n`，不是 `i <= n`。

数组不能整体赋值

array-assign.c ——错误：数组整体赋值

```c
int a[5] = {1, 2, 3, 4, 5};
int b[5];

b = a;      /* 错误：数组名不能作为赋值目标 */
```

正确做法是用循环逐个复制：

array-copy.c ——逐个复制

```c
int i;
for (i = 0; i < 5; i++) {
    b[i] = a[i];
}
```

6.6 拔高：数组名、退化与栈空间

6.6.1 数组名不是普通的变量

`sizeof(a)` 能得到整个数组的大小，但 `a` 在大多数表达式里会“退化”成指向首元素的指针。这就是为什么：

- `sizeof(a)` 是 40（整个数组）；
- 把 `a` 传给函数后，函数里的 `sizeof(a)` 变成 8（指针大小，64 位环境）。

array-decay.c ——数组名退化

```c
#include <stdio.h>

void show(int arr[])
```

```
{
    printf("函数内 sizeof(arr) = %d\n", (int)sizeof(arr));
}

int main(void)
{
    int a[10];

    printf("main 内 sizeof(a) = %d\n", (int)sizeof(a));
    show(a);
    return 0;
}
```

运行结果（64 位环境）

```
main 内 sizeof(a) = 40
函数内 sizeof(arr) = 8
```

所以“在函数里用 `sizeof(arr) / sizeof(arr[0])` 求长度”是错的。正确做法是把长度作为参数一起传进去。

6.6.2 数组在栈上

局部数组分配在栈上，大小在编译时确定。栈空间通常只有几 MB，不要定义过大的局部数组：

big-array.c ——栈溢出的风险

```
void f(void)
{
    int a[1000000];   /* 4 MB，可能直接栈溢出 */
    ...
}
```

需要大数组时，可以：

- 定义为全局数组或静态数组（放在数据区，不在栈上）；
- 用 `malloc` 动态分配（第 8 章）。

6.6.3 变长数组（VLA）

C99 支持用变量定义数组长度（`int a[n];`），但 C89 不支持，而且 VLA 在栈上分配、大小不可控，不建议在嵌段式系统里使用。本书坚持 C89 写法。

6.7 综合例题

例题 6.8 读入 10 个整数，统计其中正数、负数和零的个数。

解思路如下：

count-sign.c ——统计正负零

```c
#include <stdio.h>

int main(void)
{
    int a[10];
    int i;
    int positive = 0;
    int negative = 0;
    int zero = 0;

    printf("请输入 10 个整数：");
    for (i = 0; i < 10; i++) {
        scanf("%d", &a[i]);
    }

    for (i = 0; i < 10; i++) {
        if (a[i] > 0) {
            positive++;
        } else if (a[i] < 0) {
            negative++;
        } else {
            zero++;
        }
    }

    printf("正数 %d 个，负数 %d 个，零 %d 个\n",
           positive, negative, zero);
    return 0;
}
```

例题 6.9 用数组求斐波那契数列的前 20 项。

解思路如下：

fib-array.c ——用数组存斐波那契

```c
#include <stdio.h>

int main(void)
{
    int f[20];
    int i;

    f[0] = 1;
    f[1] = 1;
    for (i = 2; i < 20; i++) {
        f[i] = f[i - 1] + f[i - 2];
    }

```

```c
    for (i = 0; i < 20; i++) {
        printf("%d ", f[i]);
    }
    printf("\n");

    return 0;
}
```

用数组的好处是：所有项都保留下来，可以随时回头访问 f[k]。

例题 6.10 输入 10 个学生的成绩，输出高于平均分的成绩。

解思路如下：

above-average.c ——高于平均分

```c
#include <stdio.h>

int main(void)
{
    int scores[10];
    int i;
    int sum = 0;
    double average;

    printf("请输入 10 个成绩: ");
    for (i = 0; i < 10; i++) {
        scanf("%d", &scores[i]);
        sum = sum + scores[i];
    }

    average = sum / 10.0;
    printf("平均分 = %.2f\n", average);
    printf("高于平均分的成绩: ");
    for (i = 0; i < 10; i++) {
        if (scores[i] > average) {
            printf("%d ", scores[i]);
        }
    }
    printf("\n");

    return 0;
}
```

注意 scores[i] > average 比较的是 int 和 double，scores[i] 会自动转成 double，没有问题。

6.8 本章小结

主题	要点
数组	一段连续内存，元素类型相同，下标从 0 开始
定义	`类型名字 [长度];`，长度必须是编译期常量
初始化	`{...}`：部分初始化补 0；`{0}` 清零
遍历	`for (i = 0; i < n; i++)`，注意边界
内存	元素连续存放；下标可换算成地址偏移
二维数组	`a[i][j]`，按行连续存放
常见算法	最值、逆序、统计、线性查找、二分查找、矩阵
常见错误	越界、未初始化、整体赋值、长度用变量

下一章预告

数组只能装同一种类型的数据。如果要描述一个“学生”，需要同时存姓名、学号、成绩，类型各不相同，数组就不够用了。下一章学习**结构体**——把不同类型的数据打包成一个整体。

第六章 练习

1. 定义 `int a[10]`，读入 10 个数并输出，要求每行 5 个。
2. 读入 10 个数，求最大值、最小值、平均值。
3. 读入 10 个数，原地逆序后输出。
4. 读入 10 个数，统计其中偶数和奇数的个数。
5. 读入一个整数，统计每个数字出现的次数（0~9）。
6. 在有序数组中用二分查找查找某个数，输出比较次数。
7. 输入两个 3×3 矩阵，输出它们的和与转置。
8. 输入 10 个成绩，输出高于平均分的成绩及其下标。
9. 用数组求斐波那契数列前 30 项（注意是否会溢出）。
10. 思考题：为什么 `sizeof(a) / sizeof(a[0])` 在函数内部求数组长度会失效？应该怎么传长度？

第七章 结构体：古希腊掌管分类的神

内容提要

- 为什么需要结构体：把不同类型的数据打包成一个整体
- 结构体的定义、声明、初始化与成员访问
- typedef 与嵌套结构体
- 结构体数组与结构体作为函数参数
- 内存对齐：结构体到底占多少字节
- 位域与联合体（拔高）

「世界不是由同一种东西组成的，程序也不该只用一种类型描述世界。」

数组很优雅，但它有一个前提：所有元素类型相同。可现实中的一个“学生”包含学号（整数）、姓名（字符串）、成绩（小数），类型各不相同。你当然可以用三个平行数组分别存，但数据一旦多了，维护起来就是灾难。

结构体（structure）就是为此而生：它把若干个不同类型的数据打包成一个整体，给这个整体起一个名字。学会结构体，你就从“处理数据”迈向了“描述对象”。

7.1 为什么需要结构体

假设要管理 3 个学生的信息。用平行数组的写法是这样的：

parallel-arrays.c ——用平行数组存学生信息

```c
#include <stdio.h>

int main(void)
{
    int ids[3] = {1001, 1002, 1003};
    char names[3][20] = {"Alice", "Bob", "Carol"};
    double scores[3] = {92.5, 78.0, 85.5};
    int i;

    for (i = 0; i < 3; i++) {
        printf("%d %s %.1f\n", ids[i], names[i], scores[i]);
    }

    return 0;
}
```

这个写法的问题在于：三个数组必须**一一对应**。如果排序时只交换了 `scores`，忘了同步交换 `ids` 和 `names`，数据就全乱了。新增一个字段（比如年龄）还要再开一个数组，处处要同步。

结构体把“一个学生的所有信息”绑在一起：

struct-student.c ——用结构体存学生信息

```c
#include <stdio.h>

struct Student {
    int id;
    char name[20];
```

```c
    double score;
};

int main(void)
{
    struct Student s;

    s.id = 1001;
    s.score = 92.5;
    /* s.name 是字符数组，赋值方式第 9 章讲，这里先用初始化 */

    printf("%d %.1f\n", s.id, s.score);
    return 0;
}
```

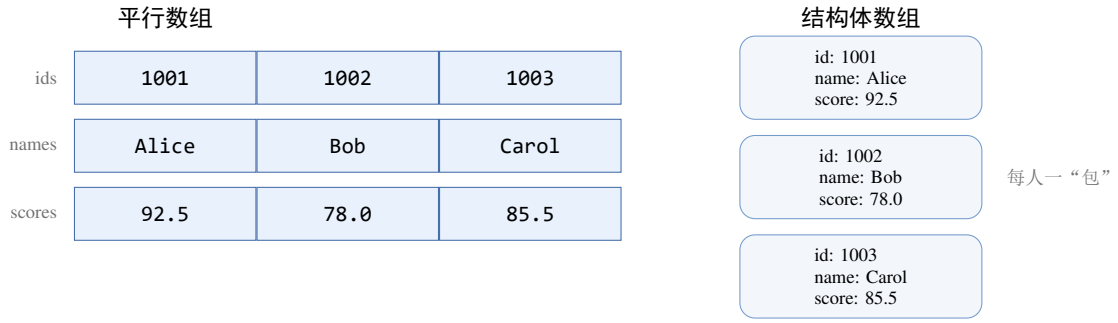


图 7.1: 平行数组 vs. 结构体数组：后者把一个人的信息绑在一起

7.2 结构体的定义与使用

7.2.1 定义

```
struct 结构体名 {
    类型成员 1;
    类型成员 2;
    ...
};   /* 注意这个分号 */
```

struct-def.c ——定义一个学生结构体

```c
struct Student {
    int id;             /* 学号 */
    char name[20];      /* 姓名 */
    double score;       /* 成绩 */
};
```

结构体定义末尾的分号不能少

`struct Student { ... };` 最后的分号是定义的一部分，漏掉会导致一连串奇怪的编译错误。这个分号很容易忘，因为函数定义后面不写分号。

`struct Student` 是一个**类型名**，就像 `int` 一样。注意：`Student` 单独使用不合法，必须写 `struct Student`，除非用 `typedef` 起别名（见后文）。

7.2.2 声明变量与访问成员

struct-use.c ——声明、初始化、访问成员

```c
#include <stdio.h>

struct Student {
    int id;
    char name[20];
    double score;
};

int main(void)
{
    struct Student s1;
    struct Student s2 = {1002, "Bob", 78.0};

    s1.id = 1001;
    s1.score = 92.5;

    printf("s1: %d, %.1f\n", s1.id, s1.score);
    printf("s2: %d, %s, %.1f\n", s2.id, s2.name, s2.score);

    return 0;
}
```

运行结果

```
s1: 1001, 92.5
s2: 1002, Bob, 78.0
```

访问成员用点号 `.`，读作“的”：`s1.id` 就是“s1 的 id”。

7.2.3 整体赋值

和数组不同，**同类型的结构体变量可以整体赋值**，编译器会把所有成员逐字节复制过去：

struct-assign.c ——结构体整体赋值

```c
#include <stdio.h>

struct Point {
    int x;
```

```c
    int y;
};

int main(void)
{
    struct Point p1 = {3, 4};
    struct Point p2;

    p2 = p1;        /* 合法：把 p1 的所有成员复制给 p2 */

    printf("p2 = (%d, %d)\n", p2.x, p2.y);
    return 0;
}
```

运行结果

```
p2 = (3, 4)
```

7.2.4 typedef：给类型起别名

每次写 `struct Student` 有点啰嗦。可以用 `typedef` 起一个别名：

typedef-struct.c ——typedef 的两种写法

```c
/* 写法一：先定义，再起别名 */
struct Student {
    int id;
    char name[20];
    double score;
};
typedef struct Student Student;

/* 写法二：定义的同时起别名（更常用） */
typedef struct {
    int x;
    int y;
} Point;
```

之后就可以直接写 `Student s;`、`Point p;`，和 `int a;` 一样自然。

本书的风格

本书在需要频繁使用某个结构体时，会用 `typedef` 起一个首字母大写的别名；只在定义处出现一次的结构体，可以直接用 `struct` 写法。

7.2.5 嵌套结构体

结构体的成员也可以是另一个结构体：

nested-struct.c ——嵌套结构体

```c
#include <stdio.h>

typedef struct {
    int year;
    int month;
    int day;
} Date;

typedef struct {
    int id;
    char name[20];
    Date birthday;      /* 成员是另一个结构体 */
    double score;
} Student;

int main(void)
{
    Student s = {1001, "Alice", {2005, 6, 18}, 92.5};

    printf("%s 出生于 %d 年 %d 月 %d 日\n",
           s.name, s.birthday.year, s.birthday.month, s.birthday.day);

    return 0;
}
```

运行结果

```
Alice 出生于 2005 年 6 月 18 日
```

访问嵌套成员就一层层点下去：`s.birthday.year`。

7.3 结构体在内存里长什么样

结构体的成员按声明顺序依次存放，但**不一定紧挨着**——编译器会在成员之间插入“填充字节”（padding），让每个成员的地址满足对齐要求。

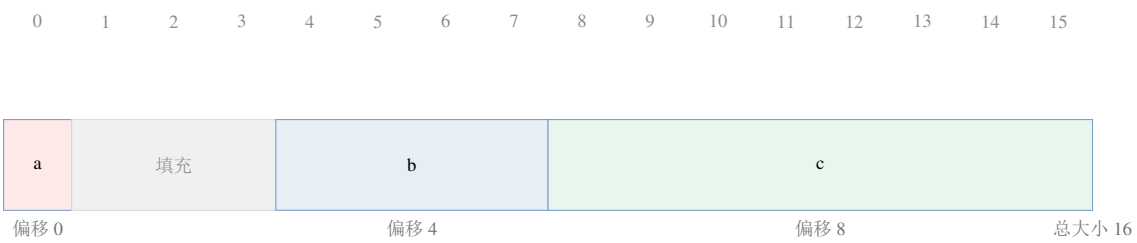


图 7.2: `struct { char a; int b; double c; }` 的内存布局（典型 8 字节对齐）

为什么需要对齐？因为 CPU 读取内存时，按“字”对齐的访问速度最快，某些架构甚至不允许非对齐访问。编译器为了性能，会在成员之间补空。

sizeof-struct.c ——结构体的大小

```c
#include <stdio.h>

struct A {
    char a;
    int b;
    double c;
};

struct B {
    char a;
    char b;
    int c;
};

int main(void)
{
    printf("sizeof(struct A) = %d\n", (int)sizeof(struct A));
    printf("sizeof(struct B) = %d\n", (int)sizeof(struct B));
    return 0;
}
```

运行结果（典型 64 位环境）

```
sizeof(struct A) = 16
sizeof(struct B) = 8
```

`struct A` 的成员一共 1 + 4 + 8 = 13 字节，但因为对齐，实际占 16 字节。`struct B` 的两个 `char` 挤在一起，后面补 2 字节再放 `int`，共 8 字节。

省内存的小技巧

把成员按“从大到小”排列，通常能减少填充。例如 `struct {double c; int b; char a;}` 在很多平台上只占 16 字节，而 `{char a; int b; double c;}` 也是 16——但当成员更多时，差别会明显。

不要把结构体的内存布局当成协议

虽然你可以用 `memcpy` 把结构体直接写进文件或发到网络，但填充字节的内容是不确定的，不同编译器/平台的对齐规则也不同。跨平台传输数据时，应该逐字段序列化，而不是直接发结构体。

7.4 结构体数组

结构体数组就是“每个元素都是一个结构体”的数组：

struct-array.c ——结构体数组

```c
#include <stdio.h>

typedef struct {
    int id;
```

```
    char name[20];
    double score;
} Student;

int main(void)
{
    Student cls[3] = {
        {1001, "Alice", 92.5},
        {1002, "Bob",   78.0},
        {1003, "Carol", 85.5}
    };
    int i;
    double sum = 0.0;

    for (i = 0; i < 3; i++) {
        printf("%d %-8s %.1f\n", cls[i].id, cls[i].name, cls[i].score);
        sum = sum + cls[i].score;
    }

    printf("平均分 = %.2f\n", sum / 3.0);
    return 0;
}
```

运行结果

```
1001 Alice    92.5
1002 Bob      78.0
1003 Carol    85.5
平均分 = 85.33
```

笔记 %-8s 表示字符串左对齐、占 8 格，用来让姓名列对齐。这个格式符第 9 章会详细讲。

7.5 结构体与函数

结构体可以像普通变量一样传给函数、从函数返回。这里先讲最基本的用法，函数的完整语法见附录 E。

7.5.1 按值传递：函数拿到一份副本

struct-param.c ——结构体作为参数

```
#include <stdio.h>

typedef struct {
    int id;
    char name[20];
    double score;
} Student;

```

```c
void print_student(Student s)
{
    printf("%d %s %.1f\n", s.id, s.name, s.score);
}

int main(void)
{
    Student s = {1001, "Alice", 92.5};

    print_student(s);       /* 传递的是一份副本 */
    return 0;
}
```

按值传递时，函数里修改 `s` 不会影响 `main` 里的原变量。这很安全，但如果结构体很大，复制会浪费时间。

7.5.2 按地址传递：函数可以修改原变量

如果想让函数修改原结构体，或者结构体很大想避免复制，就传地址：

struct-pointer.c ——传地址修改结构体

```c
#include <stdio.h>

typedef struct {
    int id;
    double score;
} Student;

void set_score(Student *p, double new_score)
{
    p->score = new_score;       /* 等价于 (*p).score */
}

int main(void)
{
    Student s = {1001, 92.5};

    set_score(&s, 95.0);        /* 传地址 */
    printf("%.1f\n", s.score);
    return 0;
}
```

运行结果

```
95.0
```

`p->score` 是“指针访问成员”的写法，等价于 `(*p).score`。下一章学指针时会详细解释，现在先记住这个符号。

7.6 综合例题

7.6.1 例：学生成绩统计

例题 7.1 有 5 个学生，每个学生有学号、姓名、成绩。输出平均分、最高分学生，以及高于平均分的人数。

解思路如下：

student-stats.c ——学生成绩统计

```c
#include <stdio.h>

typedef struct {
    int id;
    char name[20];
    double score;
} Student;

int main(void)
{
    Student cls[5] = {
        {1001, "Alice", 92.5},
        {1002, "Bob",   78.0},
        {1003, "Carol", 85.5},
        {1004, "Dave",  66.0},
        {1005, "Eve",   95.0}
    };
    int i;
    double sum = 0.0;
    double average;
    int max_index = 0;
    int above = 0;

    for (i = 0; i < 5; i++) {
        sum = sum + cls[i].score;
    }
    average = sum / 5.0;

    for (i = 1; i < 5; i++) {
        if (cls[i].score > cls[max_index].score) {
            max_index = i;
        }
    }

    for (i = 0; i < 5; i++) {
        if (cls[i].score > average) {
            above++;
        }
    }

```

```
    printf("平均分 = %.2f\n", average);
    printf("最高分: %s (%.1f)\n",
           cls[max_index].name, cls[max_index].score);
    printf("高于平均分的人数: %d\n", above);

    return 0;
}
```

运行结果

```
平均分 = 83.40
最高分: Eve (95.0)
高于平均分的人数: 2
```

7.6.2 例：复数运算

例题 7.2 定义复数结构体，输入两个复数，输出它们的和与积。

解 复数 $a+bi$ 与 $c+di$ 相加为 $(a+c)+(b+d)i$，相乘为 $(ac-bd)+(ad+bc)i$。

complex.c ——复数运算

```c
#include <stdio.h>

typedef struct {
    double re;      /* 实部 */
    double im;      /* 虚部 */
} Complex;

int main(void)
{
    Complex x, y, sum, product;

    printf("请输入第一个复数（实部 虚部）: ");
    scanf("%lf%lf", &x.re, &x.im);
    printf("请输入第二个复数（实部 虚部）: ");
    scanf("%lf%lf", &y.re, &y.im);

    sum.re = x.re + y.re;
    sum.im = x.im + y.im;

    product.re = x.re * y.re - x.im * y.im;
    product.im = x.re * y.im + x.im * y.re;

    printf("和 = %.2f + %.2fi\n", sum.re, sum.im);
    printf("积 = %.2f + %.2fi\n", product.re, product.im);

    return 0;
}
```

运行示例

```
请输入第一个复数（实部 虚部）：1 2
请输入第二个复数（实部 虚部）：3 4
和 = 4.00 + 6.00i
积 = -5.00 + 10.00i
```

7.6.3 例：日期与下一天

例题 7.3 定义日期结构体，输入一个日期，输出它的下一天。要考虑闰年和月份天数。

解思路如下：

next-day.c ——下一天

```c
#include <stdio.h>

typedef struct {
    int year;
    int month;
    int day;
} Date;

int is_leap(int year)
{
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

int main(void)
{
    Date d;
    int days[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    printf("请输入日期（年 月 日）：");
    scanf("%d%d%d", &d.year, &d.month, &d.day);

    if (is_leap(d.year)) {
        days[2] = 29;
    }

    d.day++;
    if (d.day > days[d.month]) {
        d.day = 1;
        d.month++;
        if (d.month > 12) {
            d.month = 1;
            d.year++;
        }
    }

    printf("下一天：%d-%02d-%02d\n", d.year, d.month, d.day);
```

```
    return 0;
}
```

运行示例

```
请输入日期（年 月 日）：2024 2 28
下一天：2024-02-29
```

这里用数组 days 存每月天数，下标 1~12 对应 1~12 月，days[0] 占位不用。

7.6.4 例：电机参数

例题 7.4 定义一个电机参数结构体，包含编号、PID 三个参数和输出限幅；输入一组参数并打印。

解思路如下：

motor.c ——电机参数结构体

```c
#include <stdio.h>

typedef struct {
    int id;
    double kp;
    double ki;
    double kd;
    double limit;
} MotorParam;

int main(void)
{
    MotorParam m;

    printf("请输入电机编号与 PID 参数：");
    scanf("%d%lf%lf%lf", &m.id, &m.kp, &m.ki, &m.kd);

    printf("请输入输出限幅：");
    scanf("%lf", &m.limit);

    printf("电机 %d：kp=%.3f, ki=%.3f, kd=%.3f, limit=%.2f\n",
           m.id, m.kp, m.ki, m.kd, m.limit);

    return 0;
}
```

结构体让一组相关参数“打包”在一起，后面写 PID 函数时，传一个 MotorParam 就够了，不用传五个参数。

7.7 常见错误

错误	问题
结构体定义后漏分号	后续代码出现莫名其妙的错误
`Student s;` 但没用 `typedef`	应写 `struct Student s;`
`s == t`	结构体不能直接用 `==` 比较
`s.name = "Bob"`	数组不能整体赋值，要用 `strcpy`
`scanf("%s", &s.name)`	`s.name` 本身就是地址，不要加 `&`
忘记给数值成员加 `&`	`scanf` 写不进去

结构体比较要逐成员

`if (s1 == s2)` 在 C 里不合法。正确做法是逐成员比较，或者用 `memcmp`——但 `memcmp` 会把填充字节也比进去，不可靠。最稳妥的是逐成员比较：

struct-compare.c ——逐成员比较

```c
int equal(Student a, Student b)
{
    return a.id == b.id &&
           a.score == b.score;   /* 字符串比较需用 strcmp */
}
```

字符串成员不能直接赋值

`s.name = "Bob";` 是错的，因为 `name` 是数组，数组名不能做赋值目标。正确写法是 `strcpy(s.name, "Bob");`，第 9 章会讲。

7.8 拔高：位域与联合体

7.8.1 位域

有时结构体成员只需要几个二进制位（例如标志位），用 `int` 会浪费空间。C 语言提供了**位域**：

bitfield.c ——位域

```c
#include <stdio.h>

struct Flags {
    unsigned int enable : 1;    /* 占 1 位 */
    unsigned int mode   : 2;    /* 占 2 位，取值 0-3 */
    unsigned int speed  : 5;    /* 占 5 位，取值 0-31 */
};

int main(void)
{
    struct Flags f;

    f.enable = 1;
```

```
    f.mode = 3;
    f.speed = 20;

    printf("enable=%u mode=%u speed=%u\n", f.enable, f.mode, f.speed);
    printf("sizeof(struct Flags) = %d\n", (int)sizeof(struct Flags));
    return 0;
}
```

位域在嵌入式寄存器映射中很常见，但它的具体布局（谁在高位、跨字节怎么排）由编译器决定，跨平台代码要谨慎。

7.8.2 联合体

union 让多个成员**共用同一块内存**，同一时刻只有一个成员有意义：

union.c ——联合体

```c
#include <stdio.h>

union Data {
    int i;
    float f;
    unsigned char bytes[4];
};

int main(void)
{
    union Data d;

    d.i = 0x12345678;
    printf("i = %x\n", d.i);
    printf("bytes = %02x %02x %02x %02x\n",
           d.bytes[0], d.bytes[1], d.bytes[2], d.bytes[3]);
    printf("sizeof(union Data) = %d\n", (int)sizeof(union Data));

    return 0;
}
```

联合体常用于：

- 查看一个数据的字节表示（例如上面的小端序演示）；
- 节省内存：同一块空间在不同时刻存不同类型；
- 协议解析：把一帧数据按不同格式解释。

笔记 上面代码里 d.i = 0x12345678 后读 d.bytes，属于“用另一个成员读取”，在 C 标准里是允许的（类型双关），但不同电脑的存放顺序可能不同。这里只用于演示，不要依赖它做可移植代码。

7.9 本章小结

主题	要点
结构体	把不同类型的成员打包成一个整体
定义	`struct 名字 { ... };`，末尾分号不能少
访问	用 `.` 访问成员；整体赋值是逐字节复制
`typedef`	给类型起别名，少写 `struct`
嵌套	成员可以是另一个结构体，逐层点访问
数组	结构体数组适合管理一批对象
函数	按值传递复制一份；传地址用 `->` 修改原值
内存	成员按声明顺序排列，可能有填充字节
拔高	位域省空间；联合体共用内存

下一章预告

结构体让我们能“描述对象”，但还有一个问题没解决：如何让函数直接修改调用者的变量？如何让一个变量“指向”另一个变量？如何高效地传递大块数据？答案就是**指针**——C 语言最锋利、也最容易伤到自己的工具。

第七章 练习

1. 定义一个 `Point` 结构体（`x`、`y`），输入两个点，输出它们的中点。
2. 定义一个 `Rect` 结构体（左上角、右下角），计算面积和周长。
3. 定义 `Student` 结构体，输入 5 个学生信息，输出平均分和最高分学生。
4. 用结构体数组实现：输入 5 个学生的成绩，按输入顺序输出，并统计及格人数。
5. 定义 `Date` 结构体，判断两个日期哪个更早。
6. 定义 `Complex` 结构体，实现复数的加法、减法、乘法。
7. 定义一个包含 `char`、`int`、`double` 的结构体，用 `sizeof` 观察它的大小，并解释为什么不是各成员大小之和。
8. 写一个函数，接收 `Student` 的地址，把成绩加 5 分（不超过 100）。
9. 用位域定义一个 8 位的状态字，包含 1 位使能、2 位模式、5 位速度，并打印各字段。
10. 思考题：为什么结构体可以整体赋值，而数组不可以？

第八章 指针：会者不难

内容提要

- 地址、指针、取地址与解引用
- 指针与变量、数组、结构体的关系
- 指针作为函数参数：让函数修改调用者的数据
- 指针运算与数组遍历
- 动态内存：malloc 与 free
- 二级指针、函数指针与常见陷阱

「指针不是魔法，它只是“存地址的变量”。」

很多同学一听到“指针”就紧张。其实指针的概念非常朴素：**一个变量，里面存的是另一个变量的地址**。你已经在第 1 章见过地址，在第 2 章用 scanf 时用过 &，在第 6 章知道数组元素连续存放。指针只是把这些知识串起来。

这一章会慢一点讲：先把“地址—指针—解引用”这三件事用图讲清楚，再谈指针和数组、函数、结构体的配合，最后讲动态内存和常见陷阱。请务必亲手画几遍内存图。

8.1 地址与指针

8.1.1 变量住在哪里

第 1 章说过：每个变量都占一块内存，每块内存有地址。例如：

address.c ——打印变量的地址

```c
#include <stdio.h>

int main(void)
{
    int a = 42;

    printf("a 的值    = %d\n", a);
    printf("a 的地址 = %p\n", (void *)&a);

    return 0;
}
```

运行结果（地址每次可能不同）

```
a 的值    = 42
a 的地址 = 000000000061FE1C
```

&a 表示“取 a 的地址”。你可以把地址理解成酒店房间号：房间号不是房间本身，但拿着房间号就能找到房间。

8.1.2 指针变量

指针就是一个专门用来存地址的变量。定义方式：

```
类型 * 指针名;
```

例如 `int *p;` 表示“p 是一个指针，它指向一个 int 类型的数据”。

pointer-basic.c ——指针的基本用法

```c
#include <stdio.h>

int main(void)
{
    int a = 42;
    int *p;         /* p 是指向 int 的指针 */

    p = &a;         /* 把 a 的地址存进 p */

    printf("a 的地址 = %p\n", (void *)&a);
    printf("p 的值   = %p\n", (void *)p);
    printf("p 指向的值 = %d\n", *p);

    return 0;
}
```

运行结果（地址每次可能不同）

```
a 的地址 = 000000000061FE1C
p 的值   = 000000000061FE1C
p 指向的值 = 42
```

这里出现了两个新符号：

- `&`：取地址。`&a` 得到 a 的地址；
- `*`：解引用（间接访问）。`*p` 表示“p 指向的那个变量”。



图 8.1: 指针 p 里存的是变量 a 的地址；通过 *p 可以访问 a

一句话记住

p 是地址，*p 是地址里的东西。就像 p 是房间号，*p 是房间里的行李。

8.1.3 通过指针修改变量

有了指针，就可以“隔着地址”修改原变量：

pointer-modify.c ——通过指针修改

```c
#include <stdio.h>

int main(void)
```

```c
{
    int a = 42;
    int *p = &a;

    *p = 100;       /* 等价于 a = 100 */
    printf("a = %d\n", a);
    printf("*p = %d\n", *p);

    return 0;
}
```

运行结果

```
a = 100
*p = 100
```

这看起来多此一举，但它是很多高级技巧的基础：函数可以通过指针修改调用者的变量，数据结构可以用指针连接起来。

8.1.4 空指针与野指针

- **空指针**：不指向任何有效地址，用 NULL 表示；
- **野指针**：指向一个不确定/已失效地址的指针，极其危险。

null-pointer.c ——空指针

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = NULL;

    if (p != NULL) {
        printf("%d\n", *p);
    } else {
        printf("p 是空指针，不能解引用\n");
    }

    return 0;
}
```

野指针

wild-pointer.c ——野指针

```c
int *p;         /* 未初始化，p 里是垃圾值 */
*p = 10;        /* 往一个随机地址写数据：崩溃或更糟 */
```

指针在使用前必须初始化：要么指向一个有效变量，要么赋值为 NULL。

8.2 指针与函数

8.2.1 让函数修改调用者的变量

C 语言的函数参数是“按值传递”的：函数拿到的是实参的副本。所以下面这个交换函数是**错的**：

swap-wrong.c ——交换失败

```c
#include <stdio.h>

void swap(int x, int y)
{
    int t;
    t = x;
    x = y;
    y = t;
}

int main(void)
{
    int a = 3;
    int b = 5;

    swap(a, b);
    printf("a = %d, b = %d\n", a, b);   /* 还是 3 和 5 */
    return 0;
}
```

运行结果

```
a = 3, b = 5
```

因为 swap 里的 x、y 只是 a、b 的副本，交换副本不影响原件。要修改原件，就得把地址传进去：

swap-pointer.c ——用指针交换

```c
#include <stdio.h>

void swap(int *px, int *py)
{
    int t;

    t = *px;
    *px = *py;
    *py = t;
}

```

```c
int main(void)
{
    int a = 3;
    int b = 5;

    swap(&a, &b);       /* 传地址 */
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

运行结果

```
a = 5, b = 3
```



图 8.2: 传地址后，函数通过指针直接修改 a 和 b

8.2.2 指针参数与数组参数

数组作为函数参数时，会“退化”成指向首元素的指针。所以下面两种写法等价：

array-param.c ——数组参数就是指针

```c
void print_array(int a[], int n);   /* 写法一 */
void print_array(int *a, int n);    /* 写法二，完全等价 */
```

在函数内部，`sizeof(a)` 得到的是指针大小，不是数组大小。所以长度必须额外传进来：

print-array.c ——打印数组

```c
#include <stdio.h>

void print_array(int *a, int n)
{
    int i;
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

```

```c
int main(void)
{
    int arr[5] = {1, 2, 3, 4, 5};

    print_array(arr, 5);        /* 数组名就是首元素地址 */
    return 0;
}
```

运行结果

```
1 2 3 4 5
```

8.3 指针与数组

8.3.1 数组名的含义

数组名在大多数表达式里会转换成“指向首元素的指针”，即：

`a` 等价于 `&a[0]`.

所以下面四种写法访问的是同一个元素：

写法	含义
`a[i]`	下标访问
`*(a + i)`	先算地址，再解引用
`*(p + i)`	`p` 指向 `a[0]` 时
`p[i]`	指针也可以下标访问

pointer-array.c ——指针遍历数组

```c
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;         /* p 指向 a[0] */
    int i;

    for (i = 0; i < 5; i++) {
        printf("a[%d] = %d, *(p+%d) = %d\n",
               i, a[i], i, *(p + i));
    }

    return 0;
}
```

运行结果

```
a[0] = 10, *(p+0) = 10
a[1] = 20, *(p+1) = 20
a[2] = 30, *(p+2) = 30
a[3] = 40, *(p+3) = 40
a[4] = 50, *(p+4) = 50
```

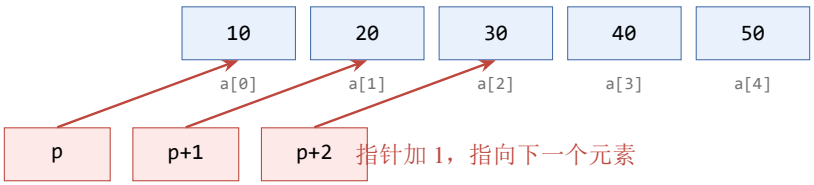


图 8.3: 指针算术：p+i 指向第 i 个元素

8.3.2 指针算术

指针加 1，不是地址加 1 字节，而是加“一个元素的大小”：

- int *p：p + 1 的地址增加 sizeof(int)（通常 4 字节）；
- double *p：p + 1 增加 8 字节；
- char *p：p + 1 增加 1 字节。

两个同类型指针相减，得到它们之间相隔的元素个数：

pointer-diff.c ——指针相减

```c
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = &a[1];
    int *q = &a[4];

    printf("q - p = %d\n", (int)(q - p));   /* 3 个元素 */

    return 0;
}
```

运行结果

```
q - p = 3
```

8.3.3 数组名、首元素地址与整个数组的地址

a、&a[0]、&a 的值通常相同，但类型不同：

- a 和 &a[0]：类型是 int *；
- &a：类型是“指向整个数组的指针”，&a + 1 会跳过整个数组。

array-address.c ——三种写法的区别

```c
#include <stdio.h>

int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};

    printf("a     = %p\n", (void *)a);
    printf("&a[0] = %p\n", (void *)&a[0]);
    printf("&a    = %p\n", (void *)&a);
    printf("a + 1   = %p\n", (void *)(a + 1));
    printf("&a + 1  = %p\n", (void *)(&a + 1));

    return 0;
}
```

运行结果（地址示意）

```
a     = 000000000061FE00
&a[0] = 000000000061FE00
&a    = 000000000061FE00
a + 1   = 000000000061FE04
&a + 1  = 000000000061FE14
```

`a + 1` 前进了 4 字节（一个 `int`），`&a + 1` 前进了 20 字节（整个数组）。初学阶段不必深究，知道有区别即可。

8.4 指针与结构体

指向结构体的指针用 `->` 访问成员：

struct-pointer2.c ——结构体指针

```c
#include <stdio.h>

typedef struct {
    int id;
    double score;
} Student;

int main(void)
{
    Student s = {1001, 92.5};
    Student *p = &s;

    printf("%d\n", p->id);          /* 等价于 (*p).id */
    p->score = 95.0;
    printf("%.1f\n", s.score);
```

```
    return 0;
}
```

运行结果

```
1001
95.0
```

结构体指针在链表、树、函数参数里非常常用。传结构体指针给函数，既避免复制，又能修改原结构体。

8.5 二级指针与函数指针（拔高）

8.5.1 二级指针

指针本身也有地址，所以可以有“指向指针的指针”：

double-pointer.c ——二级指针

```c
#include <stdio.h>

int main(void)
{
    int a = 5;
    int *p = &a;
    int **pp = &p;

    **pp = 10;          /* 通过二级指针修改 a */
    printf("a = %d\n", a);

    return 0;
}
```

运行结果

```
a = 10
```

二级指针常用于：函数需要修改“指针本身”（例如在函数里 `malloc` 并让调用者的指针指向新内存），或者二维数组、字符串数组。

8.5.2 函数指针

函数也有地址，可以定义指向函数的指针：

function-pointer.c ——函数指针

```c
#include <stdio.h>

int add(int a, int b)
```

```
{
    return a + b;
}

int main(void)
{
    int (*fp)(int, int);    /* fp 是指向“两个 int 参数、返回 int”的函数的指针 */

    fp = add;
    printf("%d\n", fp(2, 3));   /* 通过函数指针调用 */
    return 0;
}
```

运行结果

```
5
```

函数指针在回调、状态机、驱动表里很常见。初学阶段知道语法即可，不必急着用。

8.6 动态内存

8.6.1 为什么需要动态内存

数组的长度必须在编译时确定，如果运行时才知道要多少数据怎么办？用 `malloc` 在堆上动态申请内存。



图 8.4: 堆与栈：`malloc` 在堆上分配，`free` 释放

8.6.2 malloc 与 free

malloc.c ——动态数组

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
```

```c
    int i;
    int *a;

    printf("请输入元素个数：");
    scanf("%d", &n);

    a = (int *)malloc(n * sizeof(int));
    if (a == NULL) {
        printf("内存分配失败\n");
        return 1;
    }

    printf("请输入 %d 个整数：", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    free(a);        /* 用完必须释放 */
    a = NULL;       /* 防止野指针 */

    return 0;
}
```

要点：

- `malloc(n * sizeof(int))` 申请 n 个 `int` 的空间，返回首地址（`void *`）；
- 强制转换成 `(int *)` 是为了类型匹配；
- 申请可能失败，返回 `NULL`，必须检查；
- 用完调用 `free(a)` 释放；
- 释放后把指针置 `NULL`，避免“悬空指针”。

动态内存的四个大坑

1. **忘记 free**：内存泄漏，程序越跑越占内存；
2. **重复 free**：未定义行为，可能崩溃；
3. **free 后继续使用**：悬空指针，数据已被回收；
4. **越界写**：`a[n]` 越界，破坏堆结构。

8.6.3 calloc 与 realloc

calloc-realloc.c ——calloc 与 realloc

```c
/* calloc: 申请 n 个元素，并自动清零 */
int *a = (int *)calloc(n, sizeof(int));

/* realloc: 把已申请的空间调整为 new_n 个元素 */
int *b = (int *)realloc(a, new_n * sizeof(int));
if (b == NULL) {
    /* 调整失败，原来的 a 仍然有效 */
}
```

8.7 指针的常见错误

错误	后果
使用未初始化的指针	野指针，崩溃或破坏数据
`*p = 10;` 但 p 未指向有效内存	同上
数组越界后继续用指针	未定义行为
`free` 后继续使用	悬空指针
重复 `free`	崩溃
忘记 `free`	内存泄漏
`p = 10;`	类型错误：不能把整数赋给指针
返回局部变量的地址	函数结束后地址失效

不要返回局部变量的地址

return-local.c ——错误：返回局部变量地址

```c
int *bad(void)
{
    int x = 10;
    return &x;      /* x 在函数返回后就失效了 */
}
```

函数返回后，局部变量所在的栈空间会被回收，返回的地址指向一块不再有效的地方。

8.8 指针与 const

`const` 和指针结合时，位置不同含义不同：

写法	含义
`const int *p`	指向的内容不能改，指针本身可以改
`int * const p`	指针本身不能改，指向的内容可以改
`const int * const p`	两者都不能改

const-pointer.c ——const 指针

```c
void print(const int *p)
{
    /* *p = 10; 错误：不能通过 p 修改内容 */
    printf("%d\n", *p);
}
```

函数参数里用 `const int *` 表示“我只读，不改”，既清晰又安全。

8.9 综合例题

例题 8.1 用指针实现数组逆序。

解思路如下：

reverse-pointer.c ——指针逆序

```c
#include <stdio.h>

int main(void)
{
    int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int *left = a;
    int *right = a + 9;
    int temp;

    while (left < right) {
        temp = *left;
        *left = *right;
        *right = temp;

        left++;
        right--;
    }

    {
        int i;
        for (i = 0; i < 10; i++) {
            printf("%d ", a[i]);
        }
        printf("\n");
    }

    return 0;
}
```

运行结果

```
10 9 8 7 6 5 4 3 2 1
```

在 C89 里不能在 while 后面临时声明 i，所以代码里用了一个花括号块，把 i 的声明放在块开头。

例题 8.2 写一个函数，用指针求数组的最大值。

解思路如下：

max-pointer.c ——指针求最大值

```c
#include <stdio.h>

int max_array(const int *a, int n)
{
    int i;
    int max = a[0];

    for (i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
    }
    return max;
}

int main(void)
{
    int a[5] = {3, 9, 1, 7, 5};

    printf("最大值 = %d\n", max_array(a, 5));
    return 0;
}
```

例题 8.3 用 malloc 申请一个数组，读入 n 个数，求平均值后释放。

解思路如下：

dynamic-average.c ——动态数组求平均

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    int i;
    int *a;
    double sum = 0.0;

    printf("请输入个数: ");
    scanf("%d", &n);

    a = (int *)malloc(n * sizeof(int));
    if (a == NULL) {
        printf("分配失败\n");
```

```c
        return 1;
    }

    printf("请输入 %d 个数: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        sum = sum + a[i];
    }

    printf("平均值 = %.2f\n", sum / n);

    free(a);
    a = NULL;
    return 0;
}
```

8.10 本章小结

主题	要点
指针	存地址的变量；`&` 取地址，`*` 解引用
类型	指针有类型，`int *` 和 `double *` 不同
空指针	`NULL`，不能解引用
函数	传指针才能修改调用者的变量；数组参数退化为指针
数组	`a[i]` 等价于 `*(a+i)`；指针加 1 走一个元素
结构体	`p->member` 等价于 `(*p).member`
动态内存	`malloc/free`；检查 `NULL`；释放后置空
常见错误	野指针、悬空指针、内存泄漏、越界、返回局部地址

下一章预告

指针和字符数组结合，就是 C 语言的字符串。字符串是电控里处理命令、日志、协议时最常用的数据形式。下一章我们会把字符、字符数组、字符串常量、指针之间的关系彻底理清。

第八章 练习

1. 定义变量 `int a = 10`，用指针输出 `a` 的值和地址，并通过指针把 `a` 改成 20。
2. 写函数 `swap(int *a, int *b)` 交换两个整数，并在 `main` 中验证。
3. 用指针遍历数组，输出所有元素。
4. 用指针实现数组逆序。
5. 用指针求数组的最大值、最小值和平均值。
6. 写函数 `void add_score(Student *s, double x)`，给学生成绩加 `x` 分（不超过 100）。
7. 用 `malloc` 申请 n 个整数，读入并排序（可以用冒泡），然后释放。
8. 解释 `a`、`&a[0]`、`&a` 的区别。
9. 解释 `const int *p` 和 `int * const p` 的区别。
10. 思考题：为什么函数参数用数组形式 `int a[]` 时，函数内部无法用 `sizeof` 求长度？

第九章 字符串

内容提要

- 字符、字符数组与字符串常量
- 字符串的结束标志 `\0`
- `scanf`、`fgets`、`puts` 的输入输出
- `<string.h>` 常用函数：求长、复制、连接、比较、查找
- 用指针和数组处理字符串
- 中文字符编码与常见错误

「字符串是字符排成的队伍，`\0` 是队伍的终点线。」

C 语言没有专门的“字符串类型”。字符串本质上是一个**以 `\0` 结尾的字符数组**，或者一个指向这种数组的指针。理解这一点，字符串的所有“怪现象”都能解释清楚。

电控程序里到处都是字符串：串口命令、日志、文件名、协议报文、调试信息。这一章会把你从“会打印 hello”带到“能自己写字符串处理函数”。

9.1 字符与字符串

9.1.1 字符类型回顾

`char` 存的是一个字符的编码（ASCII 码），本质是小整数：

char-basic.c ——字符与编码

```c
#include <stdio.h>

int main(void)
{
    char c = 'A';

    printf("字符：%c\n", c);
    printf("编码：%d\n", c);
    return 0;
}
```

运行结果

```
字符：A
编码：65
```

常用字符的 ASCII 码：

字符	编码	字符	编码
`'0'` ~ `'9'`	48 ~57	`'A'` ~ `'Z'`	65 ~90
`'a'` ~ `'z'`	97 ~122	空格	32
`'\n'`	10	`'\0'`	0

记住三组关系：

- 数字字符'0'的编码是 48，所以'7' - '0'等于整数 7；
- 大写字母'A'是 65，小写字母'a'是 97，相差 32；
- 字符'0'和整数 0 完全不是一回事。

9.1.2 字符串就是字符数组

string-array.c ——字符串的内存

```c
#include <stdio.h>

int main(void)
{
    char s[6] = "hello";

    printf("%s\n", s);
    printf("sizeof(s) = %d\n", (int)sizeof(s));

    return 0;
}
```

运行结果

```
hello
sizeof(s) = 6
```

字符串"hello"有 5 个字符，但数组要 6 个字节，因为最后还有一个 **\0**（空字符，编码 0）作为结束标志：

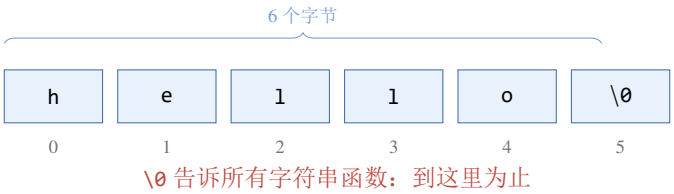


图 9.1: char s[6] = "hello" 的内存布局：5 个字符 + 1 个结束符

没有 \0 就不是字符串

no-null.c ——缺少结束符

```c
char s[5] = {'h', 'e', 'l', 'l', 'o'};   /* 没有 '\0' */
printf("%s\n", s);                       /* 会一直往后读，直到碰巧遇到 0 */
```

打印时 printf 不知道什么时候停，会读到数组后面的内存，输出乱码甚至崩溃。**字符串必须以 \0 结尾**。

9.1.3 字符串常量

双引号括起来的是字符串常量，例如"hello"。编译器会在末尾自动加 \0：

string-literal.c ——字符串常量

```c
#include <stdio.h>

int main(void)
{
    printf("%s\n", "hello");        /* 直接使用字符串常量 */
    printf("sizeof(\"hello\") = %d\n", (int)sizeof("hello"));

    return 0;
}
```

运行结果

```
hello
sizeof("hello") = 6
```

注意 `sizeof("hello")` 是 6，不是 5。

9.2 字符数组与字符指针

这是字符串最容易混淆的地方。两种写法看起来都能用，但含义不同：

array-vs-pointer.c ——两种“字符串”

```c
char s1[] = "hello";    /* 字符数组：在栈上复制一份，可修改 */
char *s2 = "hello";     /* 字符指针：指向只读的字符串常量 */
```

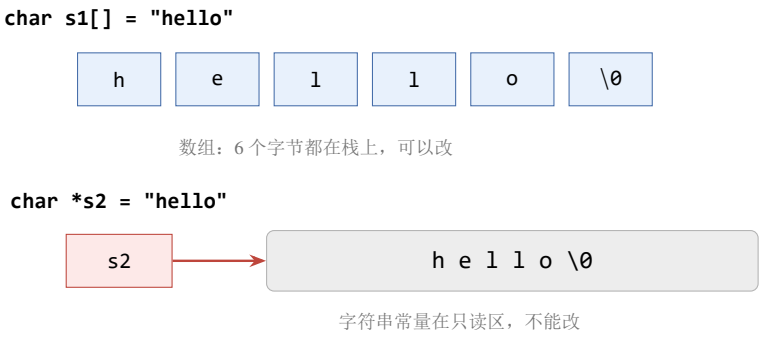


图 9.2: 字符数组会在栈上复制一份；字符指针只指向只读的字符串常量

比较	`char s[] = "hello";`	`char *p = "hello";`
存储	数组，6 字节在栈上	指针，指向只读常量区
能否修改	可以 `s[0] = 'H';`	**不可以**，未定义行为
`sizeof`	6（整个数组）	8（指针大小，64 位）
重新赋值	不可以 `s = ...`	可以 `p = ...`

modify-string.c ——修改字符串

```c
#include <stdio.h>

int main(void)
{
    char s[] = "hello";

    s[0] = 'H';
    printf("%s\n", s);      /* Hello */

    return 0;
}
```

不要修改字符串常量

modify-literal.c ——错误：修改只读常量

```c
char *p = "hello";
p[0] = 'H';     /* 未定义行为，很多平台上直接崩溃 */
```

如果需要修改，用字符数组；如果只读，用 `const char *p = "hello";` 让编译器帮你检查。

9.3 字符串的输入与输出

9.3.1 输出：printf 与 puts

string-output.c ——输出字符串

```c
#include <stdio.h>

int main(void)
{
    char s[] = "hello";

    printf("%s\n", s);      /* printf 输出，不自动换行 */
    puts(s);                /* puts 输出并自动换行 */
    return 0;
}
```

运行结果

```
hello
hello
```

9.3.2 输入：scanf("%s") 的局限

string-input.c ——读入一个单词

```c
#include <stdio.h>

int main(void)
{
    char name[20];

    printf("请输入姓名：");
    scanf("%19s", name);    /* 19 是最大宽度，防止溢出 */

    printf("你好，%s\n", name);
    return 0;
}
```

scanf("%s", name) 会跳过前导空白，然后读到下一个空白（空格、制表符、换行）为止。所以它只能读“一个单词”，不能读带空格的句子。

scanf("%s") 的缓冲区溢出

scanf-overflow.c ——危险写法

```c
char name[20];
scanf("%s", name);      /* 如果输入超过 19 个字符，就会越界写 */
```

务必写成 scanf("%19s", name)，或者改用 fgets。

9.3.3 读入一整行：fgets

fgets.c ——读入一行

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char line[100];
    int i;

    printf("请输入一行文字：");
    fgets(line, sizeof(line), stdin);

    /* fgets 会把换行符也读进来，去掉它 */
    for (i = 0; line[i] != '\0'; i++) {
        if (line[i] == '\n') {
            line[i] = '\0';
            break;
        }
```

```
    }

    printf("你输入的是：%s\n", line);
    return 0;
}
```

运行示例

```
请输入一行文字：hello world
你输入的是：hello world
```

fgets 最多读 size - 1 个字符，并自动补 \0，比 scanf 安全。它的缺点是会把换行符也存进来，需要手动去掉。

输入方式怎么选

- 读一个单词（无空格）：scanf("%19s", s);
- 读一整行：fgets(s, sizeof(s), stdin)，再去掉换行；
- 读单个字符：getchar()。

9.4 常用字符串函数

C 标准库 string.h 提供了一组字符串函数。使用前要 ##include <string.h>。

函数	作用	注意
strlen(s)	求长度（不含 \0）	返回值是 size_t
strcpy(dst, src)	复制字符串	目标空间必须够大
strncpy(dst, src, n)	最多复制 n 个字符	可能不补 \0
strcat(dst, src)	连接字符串	目标空间必须够大
strncat(dst, src, n)	最多连接 n 个字符	会自动补 \0
strcmp(a, b)	比较字符串	返回 0 表示相等
strncmp(a, b, n)	比较前 n 个字符	
strchr(s, c)	查找字符	返回指针或 NULL
strstr(s, sub)	查找子串	返回指针或 NULL

9.4.1 strlen：求长度

strlen.c ——strlen 与 sizeof

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "hello";

    printf("strlen(s) = %d\n", (int)strlen(s));
```

```
    printf("sizeof(s) = %d\n", (int)sizeof(s));
    return 0;
}
```

运行结果

```
strlen(s) = 5
sizeof(s) = 6
```

strlen 数到 \0 为止，不包含它；sizeof 是数组占的总字节数，包含 \0。

9.4.2 strcpy 与 strcat

strcpy.c ——复制与连接

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char dst[50];

    strcpy(dst, "hello");
    strcat(dst, ", world");
    printf("%s\n", dst);

    return 0;
}
```

运行结果

```
hello, world
```

strcpy 不检查目标空间

strcpy-overflow.c ——溢出

```c
char dst[5];
strcpy(dst, "hello world");   /* 目标只有 5 字节，严重越界 */
```

使用前必须确认 dst 足够大：至少 strlen(src) + 1 字节。

9.4.3 strcmp：比较字符串

strcmp.c ——比较字符串

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char a[] = "apple";
    char b[] = "banana";

    if (strcmp(a, b) == 0) {
        printf("相等\n");
    } else if (strcmp(a, b) < 0) {
        printf("a 在 b 前面\n");
    } else {
        printf("a 在 b 后面\n");
    }

    return 0;
}
```

运行结果

```
a 在 b 前面
```

字符串不能用 == 比较

`if (a == b)` 比较的是两个数组的地址，不是内容。比较内容必须用 `strcmp(a, b) == 0`。

9.4.4 查找：strchr 与 strstr

strchr.c ——查找字符与子串

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "hello world";
    char *p;

    p = strchr(s, 'w');
    if (p != NULL) {
        printf("找到 w，位置：%d\n", (int)(p - s));
        printf("从 w 开始：%s\n", p);
    }

    p = strstr(s, "world");
```

```
    if (p != NULL) {
        printf("找到子串，位置：%d\n", (int)(p - s));
    }

    return 0;
}
```

运行结果

```
找到 w，位置：6
从 w 开始：world
找到子串，位置：6
```

9.4.5 自己实现 strlen 和 strcpy

理解原理的最好方式是亲手写一遍：

my-string.c ——自己实现字符串函数

```c
#include <stdio.h>

int my_strlen(const char *s)
{
    int len = 0;

    while (s[len] != '\0') {
        len++;
    }
    return len;
}

void my_strcpy(char *dst, const char *src)
{
    int i = 0;

    while (src[i] != '\0') {
        dst[i] = src[i];
        i++;
    }
    dst[i] = '\0';      /* 别忘了结束符 */
}

int main(void)
{
    char buf[20];

    my_strcpy(buf, "hello");
    printf("%s, len = %d\n", buf, my_strlen(buf));
```

```
    return 0;
}
```

用指针写会更简洁：

my-string-pointer.c ——指针版

```
int my_strlen(const char *s)
{
    const char *p = s;

    while (*p != '\0') {
        p++;
    }
    return (int)(p - s);
}

void my_strcpy(char *dst, const char *src)
{
    while ((*dst = *src) != '\0') {
        dst++;
        src++;
    }
}
```

9.5 字符串的处理

9.5.1 统计字符

count-chars.c ——统计字母、数字、其他字符

```
#include <stdio.h>

int main(void)
{
    char s[100];
    int i;
    int letters = 0;
    int digits = 0;
    int others = 0;

    printf("请输入一行文字：");
    fgets(s, sizeof(s), stdin);

    for (i = 0; s[i] != '\0'; i++) {
        if ((s[i] >= 'a' && s[i] <= 'z') || (s[i] >= 'A' && s[i] <= 'Z')) {
            letters++;
        } else if (s[i] >= '0' && s[i] <= '9') {
```

```
            digits++;
        } else if (s[i] != '\n') {
            others++;
        }
    }

    printf("字母 %d 个，数字 %d 个，其他 %d 个\n",
           letters, digits, others);
    return 0;
}
```

运行示例

```
请输入一行文字：abc123 !!
字母 3 个，数字 3 个，其他 3 个
```

9.5.2 大小写转换

case-convert.c ——大小写转换

```
#include <stdio.h>

int main(void)
{
    char s[100];
    int i;

    printf("请输入一行文字：");
    fgets(s, sizeof(s), stdin);

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] >= 'a' && s[i] <= 'z') {
            s[i] = s[i] - 32;       /* 小写转大写 */
        } else if (s[i] >= 'A' && s[i] <= 'Z') {
            s[i] = s[i] + 32;       /* 大写转小写 */
        }
    }

    printf("转换后：%s\n", s);
    return 0;
}
```

也可以用 ctype.h 里的 toupper、tolower、isdigit、isalpha，它们更可移植。

9.5.3 判断回文

palindrome.c ——判断回文

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[100];
    int i;
    int j;
    int is_pal = 1;

    printf("请输入一个字符串：");
    scanf("%99s", s);

    i = 0;
    j = (int)strlen(s) - 1;

    while (i < j) {
        if (s[i] != s[j]) {
            is_pal = 0;
            break;
        }
        i++;
        j--;
    }

    if (is_pal) {
        printf("是回文\n");
    } else {
        printf("不是回文\n");
    }

    return 0;
}
```

运行示例

```
请输入一个字符串：level
是回文
```

9.5.4 统计单词个数

word-count.c ——统计单词个数

```c
#include <stdio.h>

int main(void)
{
    char s[200];
    int i;
    int count = 0;
    int in_word = 0;

    printf("请输入一行文字：");
    fgets(s, sizeof(s), stdin);

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == ' ' || s[i] == '\n' || s[i] == '\t') {
            in_word = 0;
        } else {
            if (!in_word) {
                count++;
                in_word = 1;
            }
        }
    }

    printf("单词个数：%d\n", count);
    return 0;
}
```

思路：用一个标志 `in_word` 记录“当前是否在单词里”。从空白进入非空白时，单词数加 1。

9.6 字符串与指针

9.6.1 用指针遍历字符串

string-pointer.c ——指针遍历

```c
#include <stdio.h>

int main(void)
{
    char s[] = "hello";
    char *p;

    for (p = s; *p != '\0'; p++) {
        printf("%c ", *p);
    }
```

```c
    printf("\n");

    return 0;
}
```

运行结果

```
h e l l o
```

`p++` 每次指向下一个字符，`*p != '\0'` 作为结束条件。

9.6.2 字符串数组

如果要存多个字符串，有两种常见方式：

string-list.c ——多个字符串

```c
/* 方式一：二维字符数组，每行一个字符串，可修改 */
char names[3][20] = {"Alice", "Bob", "Carol"};

/* 方式二：字符指针数组，每个指针指向一个字符串常量，只读 */
char *colors[3] = {"red", "green", "blue"};
```

方式一每个字符串固定占 20 字节，浪费空间但可修改；方式二只占指针和常量本身，但不能修改内容。根据需求选择。

9.7 中文与字符编码

一个英文字符在 ASCII 里占 1 字节，但中文不是。以 UTF-8 为例，一个汉字通常占 3 字节：

utf8.c ——中文的长度

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "你好";

    printf("strlen(s) = %d\n", (int)strlen(s));
    printf("sizeof(s) = %d\n", (int)sizeof(s));
    return 0;
}
```

运行结果（**UTF-8** 环境）

```
strlen(s) = 6
sizeof(s) = 7
```

“你好”只有 2 个汉字，`strlen` 却是 6——因为 UTF-8 下每个汉字占 3 字节。`strlen` 数的是**字节数**，不是字符数。

- **GBK**：一个汉字通常 2 字节；
- **UTF-8**：一个汉字通常 3 字节；
- 文件编码、终端编码、程序里的编码不一致，就会出现乱码。

处理中文的原则

- 输入输出用 `char` 数组存 UTF-8 字节，一般能原样显示；
- 不要用 `strlen` 判断“几个汉字”；
- 截断字符串时不能从中间截断一个 UTF-8 字符，否则会乱码；
- 真正需要按“字符”处理时，用宽字符 `wchar_t`（平台相关，初学了解即可）。

9.8 常见错误

错误	后果
字符数组没有 `\0`	打印越界、乱码
`char *p = "abc"; p[0] = 'A';`	修改只读常量，崩溃
`if (s1 == s2)`	比较地址，不是内容
`s = "abc";`（数组）	数组不能整体赋值，用 `strcpy`
`scanf("%s", s)` 不限宽	缓冲区溢出
`strcpy` 目标太小	溢出
`strncpy` 后忘记补 `\0`	字符串没有结束符
把 `strlen` 当字符数（中文）	结果比预期大

`strncpy` 不保证补零

strncpy.c ——strncpy 的坑

```c
char dst[5];
strncpy(dst, "hello", 5);   /* dst 里没有 '\0'! */
dst[4] = '\0';              /* 需要手动补 */
```

`strncpy` 只在源字符串长度小于 `n` 时补 `\0`；如果源字符串更长，它不会补。

9.9 综合例题

例题 9.1 输入一个字符串，删除其中所有的空格，输出结果。

解 用两个下标：`i` 读，`j` 写。遇到空格就跳过，否则写到 `j` 位置。

remove-space.c ——删除空格

```c
#include <stdio.h>

int main(void)
{
    char s[200];
    int i;
```

```
    int j = 0;

    printf("请输入一行文字：");
    fgets(s, sizeof(s), stdin);

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] != ' ') {
            s[j] = s[i];
            j++;
        }
    }
    s[j] = '\0';

    printf("删除空格后：%s\n", s);
    return 0;
}
```

例题 9.2 输入一个字符串，统计每个字母出现的次数（只统计小写字母）。

解思路如下：

letter-count.c ——字母频次

```c
#include <stdio.h>

int main(void)
{
    char s[200];
    int count[26] = {0};
    int i;

    printf("请输入一行文字：");
    fgets(s, sizeof(s), stdin);

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] >= 'a' && s[i] <= 'z') {
            count[s[i] - 'a']++;
        }
    }

    for (i = 0; i < 26; i++) {
        if (count[i] > 0) {
            printf("%c: %d\n", 'a' + i, count[i]);
        }
    }

    return 0;
}
```

例题 9.3 输入两个字符串，判断第二个是不是第一个的子串（不用 strstr）。

解思路如下：

substring.c ——判断子串

```c
#include <stdio.h>

int main(void)
{
    char s[200];
    char sub[100];
    int i, j, k;

    printf("请输入主串：");
    scanf("%199s", s);
    printf("请输入子串：");
    scanf("%99s", sub);

    for (i = 0; s[i] != '\0'; i++) {
        j = 0;
        k = i;
        while (sub[j] != '\0' && s[k] == sub[j]) {
            j++;
            k++;
        }
        if (sub[j] == '\0') {
            printf("是子串，起始位置 %d\n", i);
            return 0;
        }
    }

    printf("不是子串\n");
    return 0;
}
```

9.10 本章小结

主题	要点
字符串	以 \0 结尾的字符数组
字符数组	可修改；sizeof 是数组总大小
字符指针	指向字符串常量；不能修改内容
输入	scanf("%19s") 防溢出；fgets 读整行
输出	printf("%s") 不换行；puts 自动换行
常用函数	strlen、strcpy、strcat、strcmp、strchr、strstr
比较	用 strcmp，不能用 ==
中文	strlen 数字节，不数字符

下一章预告

有了数组和字符串，我们就可以处理一批数据了。但数据往往是乱序的，查找、统计、输出都需要先排序。下一章学习几种经典排序算法，并比较它们的效率。

第九章 练习

1. 输入一个字符串，输出它的长度（分别用 `strlen` 和自己写的函数）。
2. 输入两个字符串，把第二个连接到第一个后面（用 `strcat` 和自己写的函数）。
3. 输入两个字符串，比较它们的大小（用 `strcmp`）。
4. 输入一个字符串，统计其中字母、数字、空格和其他字符的个数。
5. 输入一个字符串，把所有小写字母转成大写。
6. 判断一个字符串是否为回文。
7. 输入一行文字，统计单词个数。
8. 输入一个字符串，删除所有空格。
9. 输入一个字符串，输出其中每个字母出现的次数。
10. 思考题：为什么 `char s[] = "hello";` 可以修改，而 `char *p = "hello";` 不能修改？它们的内存分别在哪里？

第十章 排序算法

内容提要

- 为什么要排序
- 冒泡排序：两两比较，大的往后走
- 选择排序：每次挑最小的
- 插入排序：像整理扑克牌
- 三种排序怎么选

「把乱七八糟的数据排好队，后面的事情就简单了。」

考试成绩要排名，找最大值要先比较，查资料要按顺序找。生活里到处都需要“排好队”。

这一章只学三种最基础、最好懂的排序：冒泡排序、选择排序、插入排序。我们不讲复杂的公式，只用手把过程走一遍，再把过程翻译成 C 语言。

10.1 为什么要排序

先看一组没有排序的分数：

90	65	78	100	82	55

如果老师要问“最高分是谁”，你得从头看一遍。如果数据有 1000 个，就更麻烦了。

排好序以后：

55	65	78	82	90	100

最高分在最后，最低分在最前面，找起来一眼就能看到。后面学查找、统计、排名，也会方便很多。

排序就是把一组数据按照从小到大（或从大到小）的顺序重新排列。

10.2 冒泡排序

10.2.1 先用手做一遍

冒泡排序的规则很简单：**从左到右，两两比较，前面的比后面大就交换。**

先拿 5 个数试一试：

5	2	9	1	7

第一轮：

- 比较 5 和 2：5 比 2 大，交换，变成 2 5 9 1 7；
- 比较 5 和 9：5 比 9 小，不换，还是 2 5 9 1 7；
- 比较 9 和 1：9 比 1 大，交换，变成 2 5 1 9 7；
- 比较 9 和 7：9 比 7 大，交换，变成 2 5 1 7 9。

第一轮结束，最大的 9 已经“冒”到了最后面。就像水里的气泡，大的会往上冒。

第二轮再从左边开始，但最后一个 9 已经排好，不用再比。这样一轮一轮下去，直到所有数都排好。

10.2.2 写成 C 语言

bubble.c ——冒泡排序

```c
#include <stdio.h>

int main(void)
{
    int a[8] = {5, 2, 9, 1, 7, 3, 8, 6};
    int n = 8;
    int i, j, temp;

    /* 排序前 */
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    /* 冒泡排序 */
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    /* 排序后 */
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```

运行结果

```
5 2 9 1 7 3 8 6
1 2 3 5 6 7 8 9
```

10.2.3 这段代码在做什么

- 外层循环 `i` 表示“第几轮”；
- 内层循环 `j` 表示“这一轮比较到第几个”；
- `n - 1 - i` 的意思是：每轮结束后，后面已经有 `i` 个数排好了，不用再比较；
- 如果前面的数比后面的大，就借助 `temp` 交换它们。

交换两个数

交换要用临时变量，就像交换两杯水需要第三个空杯子：

swap-three.c

```
temp = a[j];
a[j] = a[j + 1];
a[j + 1] = temp;
```

笔记 冒泡排序每轮都把当前最大的数送到后面，所以叫“冒泡”。数据少的时候很好用，数据多了会比较慢。初学阶段先把它写熟。

10.3 选择排序

10.3.1 先用手做一遍

选择排序的规则是：**每一轮找出最小的数，放到最前面。**

还是这 5 个数：

5	2	9	1	7

1. 从全部数里找最小的：是 1；
2. 把 1 和第一个数 5 交换：`1 2 9 5 7`；
3. 从剩下的 `2 9 5 7` 里找最小的：是 2，已经在第二位，不用换；
4. 从剩下的 `9 5 7` 里找最小的：是 5，和 9 交换：`1 2 5 9 7`；
5. 从剩下的 `9 7` 里找最小的：是 7，和 9 交换：`1 2 5 7 9`。

排好了。

10.3.2 写成 C 语言

selection.c ——选择排序

```
#include <stdio.h>

int main(void)
{
    int a[8] = {5, 2, 9, 1, 7, 3, 8, 6};
    int n = 8;
    int i, j, min_index, temp;

    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    for (i = 0; i < n - 1; i++) {
        min_index = i;
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[min_index]) {
```

```
                min_index = j;
            }
        }
        if (min_index != i) {
            temp = a[i];
            a[i] = a[min_index];
            a[min_index] = temp;
        }
    }

    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```

运行结果

```
5 2 9 1 7 3 8 6
1 2 3 5 6 7 8 9
```

10.3.3 这段代码在做什么

- i 表示“现在要填第几个位置”；
- 先假设第 i 个位置上的数最小，记下它的下标 min_index；
- 用 j 从后面找一圈，发现更小的就更新 min_index；
- 找完以后，如果最小数不在第 i 个位置，就交换过去。

选择排序的好处是**交换次数少**，每一轮最多交换一次。

10.4 插入排序

10.4.1 先用手做一遍

插入排序就像整理扑克牌：

- 左手拿已经排好的牌；
- 右手每次摸一张新牌；
- 从右往左比，把新牌插到合适的位置。

例如 5 2 9 1 7：

1. 先看第一个数 5，它自己就是有序的；
2. 摸到 2，比 5 小，插到 5 前面：2 5；
3. 摸到 9，比 5 大，放在后面：2 5 9；
4. 摸到 1，比 9、5、2 都小，插到最前面：1 2 5 9；
5. 摸到 7，插到 5 和 9 之间：1 2 5 7 9。

10.4.2 写成 C 语言

insertion.c ——插入排序

```c
#include <stdio.h>

int main(void)
{
    int a[8] = {5, 2, 9, 1, 7, 3, 8, 6};
    int n = 8;
    int i, j, key;

    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    for (i = 1; i < n; i++) {
        key = a[i];             /* 摸到的新牌 */
        j = i - 1;

        /* 从右往左找位置，比 key 大的都往后挪 */
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;         /* 把新牌放进去 */
    }

    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```

运行结果

```
5 2 9 1 7 3 8 6
1 2 3 5 6 7 8 9
```

10.4.3 这段代码在做什么

- 从第二个数开始，把当前数当作“新牌”key；
- 从它前面一个数开始，从右往左看；
- 如果前面的数比 key 大，就把前面的数往后挪一格；
- 挪到合适的位置后，把 key 放进去。

插入排序在数据**原本就比较有序**的时候特别快，因为不需要挪很多次。

10.5 三种排序比一比

排序方法	怎么做的	优点	适合的情况
冒泡排序	相邻两个比较，大的往后换	最好理解	数据很少，练手
选择排序	每轮找最小的放到前面	交换次数少	交换比较麻烦的时候
插入排序	像抓牌，插入已排好的部分	数据基本有序时很快	数据不多、接近有序

初学怎么选

初学阶段，先把**冒泡排序**写熟。它最直观，出错也容易查。等以后数据变多，再学更快的排序方法。



笔记这三种排序在数据很多时都比较慢。真正的大数据排序，计算机会用更聪明的算法。那是后面进阶的内容，现在不用着急。

10.6 小应用：给成绩排序

下面把 5 个成绩从高到低排序，用的是冒泡排序。注意比较符号从 `>` 变成了 `<`：

sort-scores.c ——成绩从高到低

```c
#include <stdio.h>

int main(void)
{
    int score[5] = {78, 92, 65, 88, 70};
    int n = 5;
    int i, j, temp;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (score[j] < score[j + 1]) {
                temp = score[j];
                score[j] = score[j + 1];
                score[j + 1] = temp;
            }
        }
    }

    printf("从高到低：");
    for (i = 0; i < n; i++) {
        printf("%d ", score[i]);
    }
    printf("\n");

    return 0;
}
```

运行结果

```
从高到低：92 88 78 70 65
```

只要把比较的“大于”改成“小于”，就能从升序变成降序。

10.7 本章小结

主题	要点
排序	把数据按从小到大或从大到小的顺序排好
冒泡排序	相邻比较，大的往后换；每轮把最大值送到末尾
选择排序	每轮找最小值，放到前面
插入排序	把当前数插入左边已排好的部分
交换	需要临时变量 `temp`
升序/降序	改一下比较符号即可

下一章预告

最后我们学宏定义：给一段代码起个名字，让写程序更方便。学完这一章，这本书的正文就结束啦。

第十章 练习

1. 用冒泡排序把 `5 3 8 1 2` 从小到大排序，写出每一轮的结果。
2. 用选择排序把 `9 4 7 2 6` 从小到大排序，写出每一轮的结果。
3. 用插入排序把 `6 2 8 4 1` 从小到大排序，写出每一步。
4. 把本章的冒泡排序程序改成从大到小排序。
5. 输入 10 个整数，用冒泡排序从小到大输出。
6. 输入 10 个整数，用选择排序从大到小输出。
7. 输入 10 个学生的成绩，用任意一种排序从高到低输出。
8. 想一想：如果数据本来就已经排好序了，哪种排序最省事？为什么？
9. 思考题：冒泡排序为什么要比较 `n - 1 - i` 次，而不是每次都比较 `n - 1` 次？

第十一章 宏定义

内容提要

- 预处理器在编译之前做了什么
- 对象式宏与函数式宏
- 括号、副作用与 do-while(0)
- # 与 ##：字符串化与连接
- 条件编译与头文件保护
- 预定义宏、调试宏与常见陷阱

「宏是编译之前的文本替换：威力很大，也很容易伤到自己。」

宏定义（macro）由**预处理器**处理，发生在真正的编译之前。它本质上就是“文本替换”：你写一个名字，预处理器把它替换成一段文本。

宏可以用来定义常量、写“函数式”代码、做条件编译、开关调试信息。它没有类型检查，也不遵守作用域规则，所以用得好是利器，用不好就是灾难。这一章会把宏的能力和边界都讲清楚。

11.1 预处理器

11.1.1 编译之前的步骤

第 1 章讲过，C 程序从源码到可执行文件要经过预处理、编译、汇编、链接。预处理器负责处理所有以 ## 开头的行：



图 11.1: 预处理器在编译之前工作，只做文本层面的处理

预处理器的输出仍然是 C 代码，只是所有宏都被替换了、头文件被展开了、条件编译的分支被裁剪了。

11.1.2 预处理指令

指令	作用
`#include`	包含头文件
`#define`	定义宏
`#undef`	取消宏定义
`#if` / `#ifdef` / `#ifndef`	条件编译
`#else` / `#elif` / `#endif`	条件编译分支
`#error`	产生编译错误
`#pragma`	编译器相关指令

11.2 对象式宏

11.2.1 定义常量

define-const.c ——定义常量

```c
#include <stdio.h>

#define PI 3.1415926
#define MAX_SIZE 100
#define VERSION "1.0"

int main(void)
{
    printf("PI = %f\n", PI);
    printf("MAX_SIZE = %d\n", MAX_SIZE);
    printf("VERSION = %s\n", VERSION);

    return 0;
}
```

运行结果

```
PI = 3.141593
MAX_SIZE = 100
VERSION = 1.0
```

预处理器会把代码里所有的 `PI` 替换成 `3.1415926`，就像你在编辑器里用了“全部替换”。

宏定义不要加分号

define-semicolon.c ——错误：多了分号

```c
#define MAX_SIZE 100;      /* 错误 */
int a[MAX_SIZE];            /* 展开成 int a[100;]; */
```

宏不是语句，末尾不加分号。否则替换后会多出一个分号，引发莫名其妙的错误。

11.2.2 宏与常量的区别

macro-vs-const.c ——宏、const、enum

```c
#define MAX 100                 /* 宏：无类型，预处理替换 */
const int max_value = 100;      /* const 变量：有类型，编译器检查 */
enum { MAX_ENUM = 100 };        /* 枚举常量：有类型，可作数组长度 */
```

方式	优点	缺点
`#define`	无类型限制、可用于预处理条件	无类型检查、难调试、可能重复求值
`const`	有类型、可调试	C89 中不能作数组长度
`enum`	有类型、可作数组长度	只能是整数

常量怎么选

整数常量优先用 `enum` 或 `const`；只在需要预处理条件（`#if`）或跨平台配置时用 `#define`。

11.3 函数式宏

11.3.1 基本写法

square-macro.c ——函数式宏

```c
#include <stdio.h>

#define SQUARE(x) ((x) * (x))

int main(void)
{
    int a = 5;

    printf("%d\n", SQUARE(a));
    printf("%d\n", SQUARE(a + 1));

    return 0;
}
```

运行结果

```
25
36
```

`SQUARE(a + 1)` 展开成 `((a + 1) * (a + 1))`，结果是 36。

11.3.2 括号！括号！括号！

macro-paren.c ——不加括号的灾难

```c
#define SQUARE_BAD(x) x * x
#define SQUARE_GOOD(x) ((x) * (x))

SQUARE_BAD(2 + 3)    /* 展开成 2 + 3 * 2 + 3 = 11，而不是 25 */
SQUARE_GOOD(2 + 3)   /* 展开成 ((2 + 3) * (2 + 3)) = 25 */
```

宏的参数和整体都要加括号

macro-paren2.c ——另一个经典错误

```c
#define DOUBLE(x) (x) + (x)
int y = DOUBLE(3) * 2;   /* 展开成 (3) + (3) * 2 = 9，而不是 12 */
```

正确写法：`#define DOUBLE(x) ((x) + (x))`。记住：**每个参数加括号，整个表达式再加一层括号。**

11.3.3 副作用：参数被求值多次

macro-side-effect.c ——宏的副作用

```c
#define SQUARE(x) ((x) * (x))

int i = 5;
int y = SQUARE(i++);    /* 展开成 ((i++) * (i++))，i 被加了两次 */
```

这属于未定义行为：`i` 在同一个表达式中被修改了两次。宏的参数不是“传值”，而是**文本替换**，所以会重复求值。

带副作用的参数不要传给宏

`i++`、`func()`、`*p++` 这类表达式传给函数式宏时，可能被求值多次。要么先用临时变量接住，要么改用函数。

11.3.4 用 do-while(0) 包装多语句宏

do-while-macro.c ——多语句宏

```c
#define SWAP(a, b) do { \
    int temp = (a);     \
    (a) = (b);          \
    (b) = temp;         \
} while (0)
```

为什么要包成 `do { ... } while (0)`？因为这样宏展开后仍然是一条语句，可以安全地放在 `if-else` 里：

swap-macro-use.c ——使用多语句宏

```c
if (x > y)
    SWAP(x, y);
else
    printf("不用交换\n");
```

如果直接写成 `{ ... }`，展开后 `if` 和 `else` 的配对就会出问题。

11.3.5 常用的函数式宏

common-macros.c ——常见宏

```c
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#define ABS(x)    ((x) < 0 ? -(x) : (x))
#define ARRAY_SIZE(a) (sizeof(a) / sizeof((a)[0]))
```

`ARRAY_SIZE` 只能用于真正的数组，不能用于指针参数。

11.4 # 与

11.4.1 字符串化

在宏参数前加 `#`，会把参数变成字符串：

stringize.c ——字符串化

```c
#include <stdio.h>

#define STR(x) #x

int main(void)
{
    printf("%s\n", STR(hello));
    printf("%s\n", STR(1 + 2));
    printf("%s\n", STR(MAX_SIZE));

    return 0;
}
```

运行结果

```
hello
1 + 2
MAX_SIZE
```

注意：`STR(MAX_SIZE)` 得到的是字符串`"MAX_SIZE"`，不会先把宏展开。要展开后再字符串化，需要两层宏（进阶技巧）。

11.4.2 连接

`##` 把两个记号拼成一个：

token-paste.c ——记号连接

```c
#include <stdio.h>

#define MAKE_VAR(n) var_##n
```

```
4
5  int main(void)
6  {
7      int MAKE_VAR(1) = 10;       /* 展开成 int var_1 = 10; */
8      int MAKE_VAR(2) = 20;
9
10     printf("%d %d\n", var_1, var_2);
11     return 0;
12 }
```

运行结果

```
10 20
```

常用于自动生成变量名、函数名，或者 X-Macro 技巧。

11.5 条件编译

11.5.1 #ifdef / #ifndef

ifdef.c ——条件编译

```
1  #include <stdio.h>
2
3  #define DEBUG 1
4
5  int main(void)
6  {
7  #ifdef DEBUG
8      printf("调试模式：程序开始运行\n");
9  #endif
10
11     printf("程序主体\n");
12
13     return 0;
14 }
```

常见用途：

- 用 DEBUG 宏控制调试输出；
- 用 #ifdef _WIN32 区分 Windows 和 Linux；
- 用 #ifndef 做头文件保护。

11.5.2 头文件保护

头文件可能被多次包含，导致重复定义。标准做法是用条件编译保护：

student.h ——头文件保护

```c
#ifndef STUDENT_H
#define STUDENT_H

struct Student {
    int id;
    char name[20];
    double score;
};

#endif
```

第一次包含时，STUDENT_H 未定义，进入内容并定义它；第二次包含时，STUDENT_H 已定义，直接跳过。

头文件保护的名字

宏名通常用文件名大写 + 下划线，例如 STUDENT_H、MOTOR_CTRL_H。确保唯一，避免冲突。

11.5.3 #if 与 defined

if-defined.c ——更灵活的判断

```c
#if defined(DEBUG) && DEBUG > 0
    printf("调试输出\n");
#endif

#if VERSION >= 2
    printf("新版本\n");
#else
    printf("旧版本\n");
#endif
```

#if 后面必须是一个整数常量表达式，不能使用变量。

11.6 预定义宏

预处理器还提供了一些预定义宏，调试时很有用：

宏	含义
__FILE__	当前源文件名（字符串）
__LINE__	当前行号（整数）
__DATE__	编译日期（字符串）
__TIME__	编译时间（字符串）
__STDC__	是否遵循标准 C

predefined.c ——预定义宏

```c
#include <stdio.h>

```

```c
int main(void)
{
    printf("文件：%s\n", __FILE__);
    printf("行号：%d\n", __LINE__);
    printf("编译日期：%s\n", __DATE__);
    printf("编译时间：%s\n", __TIME__);

    return 0;
}
```

运行结果（示例）

```
文件：predefined.c
行号：7
编译日期：Oct 21 2026
编译时间：13:30:00
```

11.7 调试宏

把 __FILE__、__LINE__ 和 ## 结合起来，可以做一个简单的调试输出宏：

debug-macro.c ——调试宏

```c
#include <stdio.h>

#ifdef DEBUG
#define DBG_PRINT(fmt, ...) \
    printf("[%s:%d] " fmt "\n", __FILE__, __LINE__, __VA_ARGS__)
#else
#define DBG_PRINT(fmt, ...) ((void)0)
#endif
```

笔记 上面用了 ... 和 __VA_ARGS__，这是 C99 的可变参数宏，C89 编译器不一定支持。C89 下可以用固定参数或直接写 printf。本书正文代码不使用可变参数宏，这里只作了解。

C89 下更简单的做法：

debug-c89.c ——C89 调试宏

```c
#define DBG(msg) printf("[%s:%d] %s\n", __FILE__, __LINE__, msg)
```

11.8 宏的常见陷阱

错误	后果
`#define MAX 100;`	多出分号
`#define SQUARE(x) x*x`	优先级错误
参数有副作用	被求值多次
宏名与变量名冲突	意外替换
宏里写注释	注释可能影响替换
宏没有类型检查	错误类型也能通过编译
递归宏	展开不终止或行为异常

宏名不要用短小的常见词

`#define max 100` 会把代码里所有叫 `max` 的变量、函数都替换掉。宏名用大写，且尽量具体，例如 `MAX_BUFFER_SIZE`。

11.9 拔高：X-Macro 与代码生成

有时候需要“一份数据，多处使用”。例如定义一组错误码，既要枚举值，又要转成字符串。X-Macro 可以做到：

x-macro.c ——X-Macro 示意

```c
#define ERROR_LIST \
    X(OK, 0)       \
    X(TIMEOUT, 1)  \
    X(OVERFLOW, 2)

#define X(name, code) ERR_##name = code,
enum ErrorCode {
    ERROR_LIST
};
#undef X

#define X(name, code) case ERR_##name: return #name;
const char *error_name(int code)
{
    switch (code) {
        ERROR_LIST
    }
    return "UNKNOWN";
}
#undef X
```

这种写法在大型工程中用于生成“枚举 + 字符串 + 查表”等重复代码。初学阶段知道有这种技巧即可。

11.10 本章小结

主题	要点
预处理	编译前的文本替换
对象式宏	定义常量；不加分号；无类型检查
函数式宏	参数整体加括号；注意副作用
多语句宏	用 `do { ... } while (0)` 包装
`#` / `##`	字符串化 / 记号连接
条件编译	`#ifdef`、`#ifndef`、`#if`
头文件保护	`#ifndef` + `#define` + `#endif`
预定义宏	`__FILE__`、`__LINE__` 等
原则	能不用宏就不用；用就大写、加括号、写清楚

写在最后

到这里，本书的正文就结束了。你从“计算机怎么存数据”出发，走过了顺序、分支、循环、数组、结构体、指针、字符串、排序和宏。接下来最重要的不是继续学新语法，而是**写代码**：找一些小项目（计算器、学生管理、串口协议解析、简单 PID），把学过的知识串起来。遇到问题回来看对应章节，进步会很快。

第十一章 练习

1. 用宏定义 `PI`、`E` 和 `GOLDEN`，输出它们的值。
2. 写一个 `SQUARE` 宏，并测试 `SQUARE(2+3)`。
3. 写一个 `MAX3(a,b,c)` 宏，求三个数的最大值。
4. 写一个 `SWAP` 多语句宏，用 `do-while(0)` 包装。
5. 用 `#` 写一个 `PRINT_VAR(x)` 宏，输出变量名和值。
6. 用 `##` 生成 `var_1`、`var_2` 两个变量。
7. 用条件编译实现：定义 `DEBUG` 时输出调试信息，否则不输出。
8. 写一个头文件保护，防止重复包含。
9. 解释 `#define SQUARE(x) x*x` 的问题，并改正。
10. 思考题：为什么宏参数 `i++` 会导致问题？应该怎么避免？

附录 A C 语言速查表

本附录把全书最常用的语法、类型、格式符和标准库函数集中在一起，方便写代码时查阅。

A.1 关键字（C89）

C89 共有 32 个关键字，不能用作变量名或函数名：

`auto`	`break`	`case`	`char`
`const`	`continue`	`default`	`do`
`double`	`else`	`enum`	`extern`
`float`	`for`	`goto`	`if`
`int`	`long`	`register`	`return`
`short`	`signed`	`sizeof`	`static`
`struct`	`switch`	`typedef`	`union`
`unsigned`	`void`	`volatile`	`while`

C99 新增的 `inline`、`restrict`、`_Bool` 等不属于本书范围。

A.2 基本类型

类型	典型大小	范围/精度	格式符
`char`	1	−128 ~ 127	`%c` 或`%d`
`unsigned char`	1	0 ~ 255	`%u`
`short`	2	−32768 ~ 32767	`%d`
`unsigned short`	2	0 ~ 65535	`%u`
`int`	4	约 ±21 亿	`%d`
`unsigned int`	4	0 ~ 4294967295	`%u`
`long`	4/8	平台相关	`%ld`
`float`	4	约 6~7 位有效数字	`%f`
`double`	8	约 15~16 位有效数字	`%f` / `%lf`
`long double`	8/16	平台相关	`%Lf`

A.3 常用格式符

格式符	含义	例子
%d	有符号十进制整数	printf("%d", -42);
%u	无符号十进制整数	printf("%u", 42u);
%x / %X	十六进制（小/大写）	%x 输出 ff
%o	八进制	
%c	字符	printf("%c", 'A');
%s	字符串	printf("%s", "hi");
%f	浮点，默认 6 位小数	printf("%f", 3.14);
%.2f	保留 2 位小数	3.14
%8.2f	总宽 8、2 位小数	右对齐
%-8d	宽度 8、左对齐	
%05d	宽度 5、补 0	00042
%p	指针地址	(void *)&a
%%	一个百分号	

A.4 转义字符

转义	含义	转义	含义
\n	换行	\t	制表符
\r	回车	\b	退格
\\	反斜杠	\"	双引号
\'	单引号	\0	空字符

A.5 运算符优先级（高到低）

优先级	运算符	含义	结合性
1	() [] -> .	调用、下标、成员	左
2	! \~ ++ -- + - * & (类型) sizeof	单目	右
3	* / %	乘除余	左
4	+ -	加减	左
5	<< >>	移位	左
6	< <= > >=	关系	左
7	== !=	相等	左
8	&	按位与	左
9	\^	按位异或	左
10	|	按位或	左
11	&&	逻辑与	左
12	||	逻辑或	左
13	?:	条件	右
14	赋值类	= += -= ...	右
15	,	逗号	左

A.6 常用标准库函数

A.6.1 stdio.h

函数	用途
`printf(fmt, ...)`	格式化输出
`scanf(fmt, ...)`	格式化输入（变量要加 &）
`getchar()`	读一个字符
`putchar(c)`	输出一个字符
`gets(s)`	读一行（危险，不要用）
`fgets(s, n, stdin)`	安全地读一行
`puts(s)`	输出字符串并换行
`fopen/fclose`	打开/关闭文件

A.6.2 string.h

函数	用途
`strlen(s)`	字符串长度（不含 `\0`）
`strcpy(dst, src)`	复制字符串
`strncpy(dst, src, n)`	最多复制 n 个字符
`strcat(dst, src)`	连接字符串
`strcmp(a, b)`	比较字符串，相等返回 0
`strchr(s, c)`	查找字符
`strstr(s, sub)`	查找子串
`memcpy(dst, src, n)`	复制内存块
`memset(p, c, n)`	填充内存块

A.6.3 stdlib.h

函数	用途
`malloc(size)`	申请内存
`calloc(n, size)`	申请并清零
`realloc(p, size)`	调整内存大小
`free(p)`	释放内存
`abs(n)`	整数绝对值
`atoi(s)`	字符串转整数
`rand()/srand()`	随机数

A.6.4 math.h 与 ctype.h

函数	用途
sqrt(x)	平方根
fabs(x)	浮点绝对值
pow(x, y)	幂
sin/cos/tan	三角函数（弧度）
isalpha(c)	是否字母
isdigit(c)	是否数字
toupper(c)/tolower(c)	大小写转换

A.7 常用代码片段

snippets.c ——常用片段

```c
/* 交换两个整数 */
temp = a; a = b; b = temp;

/* 求数组最大值 */
max = a[0];
for (i = 1; i < n; i++) {
    if (a[i] > max) max = a[i];
}

/* 遍历数组 */
for (i = 0; i < n; i++) {
    printf("%d ", a[i]);
}

/* 字符串长度（自己实现） */
int len = 0;
while (s[len] != '\0') len++;

/* 判断闰年 */
if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
    /* 闰年 */
}

/* 动态数组 */
p = (int *)malloc(n * sizeof(int));
if (p == NULL) { /* 处理失败 */ }
free(p);
p = NULL;
```

附录 B 编译、调试与常见错误

本附录介绍如何安装编译器、编译运行 C 程序、读懂报错，以及用调试器定位问题。不同系统细节不同，但核心命令是相通的。

B.1 选择工具链

系统	推荐编译器	说明
Windows	MinGW-w64（GCC）	免费，命令与 Linux 接近
Linux	GCC / Clang	通常已预装
macOS	Clang	安装 Xcode Command Line Tools

本书推荐 **MinGW-w64 + VS Code**：编辑器轻量，GCC 的报错信息清晰，和后面的嵌入式开发环境也接近。

B.2 Windows：安装 MinGW-w64

最简单的方式是通过 MSYS2：

1. 从 `https://www.msys2.org/` 下载并安装 MSYS2；
2. 打开 MSYS2 UCRT64 终端，执行：

MSYS2 命令

```
pacman -S mingw-w64-ucrt-x86_64-gcc
```

3. 把 `C:\msys64\ucrt64\bin` 加入系统环境变量 `PATH`；
4. 打开新的命令行，执行 `gcc --version`，能显示版本号就成功了。

B.3 第一个程序的编译与运行

新建 `hello.c`：

hello.c

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

在文件所在目录打开终端，执行：

编译与运行

```
gcc hello.c -o hello -std=c89 -Wall -Wextra -pedantic
hello.exe          （Windows）
```

```
./hello          （Linux / macOS）
```

参数	作用
`-o hello`	指定输出文件名
`-std=c89`	按 C89 标准编译（本书要求）
`-Wall`	打开常用警告
`-Wextra`	打开更多警告
`-pedantic`	严格按标准检查
`-g`	生成调试信息（配合 gdb）
`-lm`	链接数学库（用了 `math.h` 时加在命令末尾）

把警告当错误看

初学阶段建议加 `-Wall -Wextra`。很多“能跑但结果不对”的 bug，编译器其实已经用警告提醒你了。更严格可以加 `-Werror`，把警告直接变成错误。

B.4 读懂错误信息

GCC 的报错格式通常是：

错误信息结构

```
文件名:行号:列号: error: 错误描述
    源代码行
    ^
```

例如：

示例

```
hello.c:5:5: error: expected ';' before 'return'
     return 0;
     ^~~~~~
```

意思是第 5 行 `return` 前面少了一个分号。**先看第一条错误**，后面的错误常常是连锁反应。

B.5 常见编译错误对照

错误信息	常见原因
`expected ';' before ...`	上一行漏分号
`expected declaration or statement at end of input`	花括号不配对
`'printf' undeclared`	忘了 `#include <stdio.h>`
`implicit declaration of function`	函数没声明或头文件没包含
`undefined reference to 'main'`	没有 `main` 函数
`undefined reference to 'sqrt'`	链接时忘了 `-lm`
`ISO C90 forbids mixed declarations and code`	声明写在了语句后面
`stray '\357' in program`	用了中文标点
`variable-sized type used in sizeof`	C89 不允许变长数组
`assignment makes pointer from integer`	指针类型不匹配

B.6 用 gdb 调试

如果程序能编译但运行结果不对，或者崩溃，可以用 gdb。

编译时加 -g，然后启动 gdb

```
gcc bug.c -o bug -g -std=c89 -Wall
gdb bug.exe
```

常用命令：

命令	作用
`run`	运行程序
`break main`	在 main 函数处下断点
`break 文件名:行号`	在指定行下断点
`next`	执行下一行（不进入函数）
`step`	执行下一行（进入函数）
`print 变量`	打印变量的值
`continue`	继续运行到下一个断点
`quit`	退出

没有 gdb 怎么办

用 `printf` 打印中间变量，同样能定位大部分问题。第 2 章介绍过这种方法。关键是：**在可疑的位置打印输入、中间结果和边界值**。

B.7 VS Code 配置

安装 VS Code 和 C/C++ 扩展后，在项目目录新建`.vscode/tasks.json`：

tasks.json ——编译任务

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "build",
            "type": "shell",
            "command": "gcc",
            "args": [
                "-std=c89", "-Wall", "-Wextra", "-pedantic", "-g",
                "${file}", "-o", "${fileDirname}\\${fileBasenameNoExtension}.exe"
            ],
            "group": { "kind": "build", "isDefault": true }
        }
    ]
}
```

按 `Ctrl+Shift+B` 即可编译。运行可以在终端手动执行生成的`.exe`。

B.8 多文件项目

编译多个源文件

```
gcc main.c math_utils.c -o app -std=c89 -Wall -Wextra
```

如果文件较多，可以写 Makefile 或使用 CMake。初学阶段先把“一个文件一个程序”练熟，再学分文件。

B.9 编码问题

- 源文件统一保存为 **UTF-8**（无 BOM）；
- Windows 控制台默认可能是 GBK，输出中文可能乱码，可以执行 `chcp 65001` 切换到 UTF-8；
- 不要在代码里混用中文全角标点和英文半角标点，这是编译错误的常见来源。

B.10 推荐的学习流程

1. 先手写代码，不要复制粘贴；
2. 编译，打开所有警告；
3. 有错先读第一条错误信息，按行号定位；
4. 改完再编译，直到零警告；
5. 运行，用边界值测试；
6. 结果不对就打印中间变量或上调试器；
7. 最后把代码整理干净，加上必要注释。

附录 C 习题参考答案与提示

这里给出各章习题的参考答案。编程题只给关键思路或核心代码，完整程序请自己补全；如果和答案不同但能通过测试，也是好答案。

C.1 第 1 章

1. 输入、处理、输出。比如计算器：按键是输入，内部计算是处理，屏幕显示是输出。
2. CPU 像大脑，负责计算和指挥；内存像草稿纸，临时存放正在用的数据；硬盘像书包，长期存放数据。
3. 内存速度快，但一断电内容就没了；硬盘速度慢，但断电后数据还在。
4. 比如只说“把水烧开”，计算机不知道用什么烧、烧到多少度、烧多久。必须把每一步写清楚。
5. $(1010)_2 = 10$，$(1100)_2 = 12$，$(11111)_2 = 31$。
6. 1 个字节等于 8 位；一个字节可以表示 $2^8 = 256$ 种不同的组合。
7. 存的是 65。计算机里只有数字，字符要按 ASCII 码表转换成数字来存。
8. 写代码 → 编译器翻译 → 生成可执行程序 → 双击运行。编译器把 C 代码翻译成机器能执行的指令。
9. 编译错误是代码写错了，编译器看不懂，程序生成不了，比如少分号；运行错误是程序能生成，但运行时做错了事，比如除以 0。
10. 因为电子电路最容易实现两种稳定状态：通电和断电。用 0 和 1 表示最可靠、最简单。

C.2 第 2 章

1. 用两次 `printf`，每句末尾加 `\n` 换行。
2. 读入两个整数，分别输出 `a+b`、`a-b`、`a*b`。
3. 周长 `2 * 3.14159 * r`，面积 `3.14159 * r * r`，用`%.2f` 输出。
4. 面积 `length * width`，周长 `2 * (length + width)`。
5. 先读入三个整数，再算 `(a + b + c) / 3.0`，保留两位小数。
6. 三步交换：`temp = a; a = b; b = temp;`。
7. 错误：第一行 `int a = 1` 少分号；`sum` 没有声明；`printf` 后面少分号。
8. `&` 是取地址，告诉 `scanf` 把读到的数放到变量 `a` 的盒子里。
9. `%d` 要求整数，而 3.14 是小数，应该用 `printf("%f", 3.14);`。
10. 声明变量就是让编译器准备一个盒子，并记住盒子的类型和名字，后面才能安全地往里放数据。

C.3 第 3 章

1. `7/2=3`，`7%2=1`，`-7/2=-3`（向零截断），`-7%2=-1`（常见实现；C89 未强制规定）。
2. `n % 2 != 0`，不要写 `== 1`（负数取余可能为 −1）。
3. `n % 3 == 0 && n % 5 != 0`。
4. 判断第 0 位：`n & 1`；第 2 位置 1：`n |= (1 << 2)`；第 3 位清零：`n &= ~(1 << 3)`。
5. 赋值运算符右结合，等价于 `a = (b = (c + 1))`：先算 `c+1`，赋给 `b`，再赋给 `a`。
6. `a = i++ + ++i;` 在同一个表达式中多次修改 `i`，属于未定义行为，结果不可预测。
7. `c >= 'a' && c <= 'z'`（或 `islower(c)`）。
8. 比较时 `-1` 被转换成 `unsigned int`，变成 4294967295，所以 `-1 < 1U` 为假。
9. `max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);`。

10. 左移超出类型宽度是未定义行为；有符号数左移溢出也可能是未定义行为。例如 `1 << 31` 对 32 位 `int` 是未定义的。

C.4 第 4 章

1. 用 `if (n > 0)`、`else if (n < 0)`、`else` 三种情况。
2. `if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))`。
3. 用字符范围判断：`c >= 'A' && c <= 'Z'` 等；也可以包含 `ctype.h` 用 `isupper`、`islower`、`isdigit`。
4. 用三个 `if` 和临时变量两两比较交换，最后按 `a,b,c` 输出。
5. 先判断 `score < 0 || score > 100`，再用 `else if` 分档。
6. 用 `switch (month)`，把 3~5 归为春、6~8 归为夏、9~11 归为秋、12/1/2 归为冬。
7. 先处理 `a == 0`，再算判别式，分大于 0、等于 0、小于 0 三种情况。
8. 先判断 `a+b>c && a+c>b && b+c>a`，再判断等边、等腰、普通。
9. `if (a = 5)` 是赋值，条件永远为真；`if (a == 5)` 才是比较。
10. `case` 必须是编译期常量，变量不是常量。可以用 `if-else` 链，或用查表/函数。

C.5 第 5 章

1. 三种循环都能写：`for (i=1;i<=100;i++) sum+=i;`、`while (i<=100) {...}`、`do {...} while (i<=100);`。
2. `fact = fact * i;`。10! 能放进 `int`；13! = 6227020800 超出 32 位 `int`，会溢出。
3. 读第一个数初始化 `max`、`min`，循环读其余数并更新，同时累加求平均。
4. `if (i % 3 == 0 || i % 5 == 0)`，每输出一个计数，满 5 个换行。
5. 试除到 `i*i <= n`；外层循环枚举 2~100。
6. `f[0]=1; f[1]=1; for(i=2;i<30;i++) f[i]=f[i-1]+f[i-2];`。
7. 辗转相除得到 gcd，lcm = `a / gcd * b`（先除后乘，避免溢出）。
8. 枚举 100~999，拆出百、十、个位，判断立方和。
9. 用 `do-while` 数位数；用`%10` 求各位和；用 `rev = rev*10 + n%10` 求逆序。
10. 数组下标范围是 `0..n-1`，`i <= n` 会访问 `a[n]`，越界。应改为 `i < n`。

C.6 第 6 章

1. 读入循环 `for (i=0;i<10;i++) scanf("%d", &a[i]);`，输出时用 `if (i%5==4) printf("\n");` 控制换行。
2. 同第 5 章第 3 题，只是数据存在数组里。
3. 双下标 `i=0, j=n-1`，`while (i<j)` 交换后向中间靠拢。
4. 遍历数组，`if (a[i]%2==0) even++; else odd++;`。
5. `count[n%10]++; n/=10;` 直到 `n==0`。
6. 维护 `low`、`high`，每次比较 `a[mid]`，并统计循环次数。
7. 用三重循环读入、相加、输出；转置输出 `a[j][i]`。
8. 先求平均，再遍历输出 `a[i] > average` 的元素及其下标。
9. `f[0]=1; f[1]=1;` 递推；第 30 项约为 832040，仍在 `int` 范围内，但再往后会溢出。
10. 数组作为函数参数会退化成指针，`sizeof(a)` 得到的是指针大小；应把长度作为参数传入。

C.7 第 7 章

1. `mid.x = (p1.x + p2.x) / 2;`，`mid.y` 同理（注意整数除法）。
2. `area = (r.x2 - r.x1) * (r.y2 - r.y1);`，周长 `2 * ((x2-x1) + (y2-y1))`。
3. 遍历结构体数组累加、比较，输出平均分和最高分学生。
4. 遍历数组，`if (s[i].score >= 60) pass++;`。
5. 依次比较年、月、日；可以用 `if` 嵌套或把日期转成“距某天的天数”。
6. 实部、虚部分别按复数公式计算。
7. 因为对齐会产生填充字节，`sizeof` 往往大于各成员大小之和；例如 `char+int+double` 在典型平台上为 16 字节。
8. 函数体里先执行 `p->score += x;`，再用 `if` 把超过 100 的成绩截断为 100。
9. 定义位域后直接给字段赋值，打印各字段和 `sizeof`。
10. 结构体可以整体赋值，编译器按成员逐个复制（包括填充字节）；数组名是常量，不能作为赋值目标。

C.8 第 8 章

1. `int *p = &a;`，用 `*p` 输出值，`*p = 20;` 修改。
2. 函数内用临时变量交换 `*a` 和 `*b`。
3. `int *p; for (p = a; p < a + n; p++) printf("%d ", *p);`。
4. 双指针 `left = a`，`right = a + n - 1`，交换后向中间移动。
5. 遍历一次同时维护 `max`、`min`、`sum`。
6. `p->score += x;`，超过 100 时截断。
7. `malloc` 后用冒泡排序，最后 `free(p); p = NULL;`。
8. `a` 和 `&a[0]` 都是首元素地址（`int *`）；`&a` 是“整个数组的地址”，加 1 会跳过整个数组。
9. `const int *p`：不能通过 `p` 改内容；`int * const p`：`p` 本身不能改指向，但内容可改。
10. 数组参数退化为指针，函数内部无法用 `sizeof` 求原数组长度，所以必须额外传 `n`。

C.9 第 9 章

1. `strlen(s)`：自己实现用循环数到 `\0`。
2. `strcat(dst, src);`，确保 `dst` 足够大。
3. `strcmp(a, b)`，返回 0 相等，小于 0 表示 `a` 在前。
4. 遍历字符，用范围判断分类；注意跳过换行符。
5. `if (c >= 'a' && c <= 'z') c = c - 32;`。
6. 双下标从两端向中间比较，全部相等则是回文。
7. 用 `in_word` 标志，从空白进入非空白时计数加 1。
8. 用读写双下标，遇到空格跳过，最后补 `\0`。
9. `count[s[i] - 'a']++;`，最后输出非零项。
10. 字符数组在栈上复制了一份，可以修改；字符指针指向只读常量区，修改是未定义行为。

C.10 第 10 章

1. 初始 `5 3 8 1 2`；第一轮后 `3 5 1 2 8`；第二轮后 `3 1 2 5 8`；第三轮后 `1 2 3 5 8`；第四轮后不变，排序完成。

2. 初始 9 4 7 2 6；第一轮把 2 换到最前：2 4 7 9 6；第二轮 4 已在第二位；第三轮把 6 换到 7 前面：2 4 6 9 7；第四轮交换 9 和 7：2 4 6 7 9。
3. 初始 6 2 8 4 1；插入 2 得 2 6 8 4 1；插入 8 不变；插入 4 得 2 4 6 8 1；插入 1 得 1 2 4 6 8。
4. 把冒泡排序里的 `a[j] > a[j + 1]` 改成 `a[j] < a[j + 1]`。
5. 读入 10 个数到数组，用本章冒泡排序代码排序，再循环输出。
6. 选择排序中把“找最小”改成“找最大”，比较符号由 `<` 改成 `>`，最后从大到小输出。
7. 读入 10 个成绩，用冒泡或选择排序按从高到低输出即可。
8. 插入排序最省事，因为数据已经有序时，每次只需要比较一次，几乎不用挪动。
9. 因为每轮结束后，最后面已经有 `i` 个数排好了，它们不需要再比较，所以这一轮只比较前 `n - 1 - i` 个。

C.11 第 11 章

1. `#define PI 3.1415926`，`#define E 2.71828`，`#define GOLDEN 1.618`。
2. `#define SQUARE(x) ((x)*(x))`；`SQUARE(2+3)` 展开为 `((2+3)*(2+3))`=25。
3. `#define MAX3(a,b,c) MAX(MAX(a,b),c)`，并保证 `MAX` 的参数都加括号。
4. `#define SWAP(a,b) do { int t=(a); (a)=(b); (b)=t; } while(0)`。
5. `#define PRINT_VAR(x) printf("%s = %d\n", #x, (x))`。
6. `#define MAKE_VAR(n) var_##n`，用 `MAKE_VAR(1)` 生成 `var_1`。
7. 用 `#ifdef DEBUG` 包住调试输出，未定义 `DEBUG` 时编译为空。
8. 头文件写 `#ifndef XXX_H`、`#define XXX_H`、内容、`#endif`。
9. 不加括号时 `SQUARE(2+3)` 会展开成 `2+3*2+3=11`；应写成 `((x)*(x))`。
10. `SQUARE(i++)` 展开为 `((i++)*(i++))`，`i` 被修改两次，属于未定义行为；先用临时变量保存 `i` 的值。

附录 D 术语中英对照

本附录按中文拼音/英文顺序整理书中出现的常用术语，方便查英文资料。

中文	English	说明
地址	address	内存单元的编号，指针里存的就是地址
实参	argument	调用函数时实际传入的值
数组	array	一段连续存放的同类型元素
ASCII 码	ASCII	常用字符的 7 位编码标准
赋值	assignment	把右边的值写进左边的变量
二进制	binary	以 0 和 1 表示数字
位	bit	最小的二进制单位，取值 0/1
字节	byte	8 位，内存编址的基本单位
缓冲区	buffer	临时存放输入输出数据的区域
调用	call	执行一个函数
强制类型转换	cast	显式地把一种类型转成另一种
字符	char	单个字符，本质是小整数
编译器	compiler	把 C 代码翻译成机器指令的程序
常量	constant	程序运行中不改变的值
CPU	central processing unit	中央处理器
悬空指针	dangling pointer	指向已释放内存的指针
调试	debug	定位和修复程序错误的过程
声明	declaration	告诉编译器“有这么个名字和类型”
定义	definition	为变量/函数分配存储或给出实现
解引用	dereference	通过指针访问它指向的数据，用 *
双精度	double	双精度浮点数，通常 8 字节
转义字符	escape sequence	反斜杠加字符表示特殊字符，如 `\n`
表达式	expression	由变量、常量、运算符组成、有值的式子
文件	file	存储在磁盘上的数据集合
浮点数	floating point	带小数点的数，计算机用专门格式存储
函数	function	完成特定任务的一段代码
全局变量	global variable	定义在所有函数外的变量
堆	heap	动态分配的内存区域，用 `malloc/free` 管理
十六进制	hexadecimal	以 16 为基数，前缀 `0x`
标识符	identifier	变量、函数、类型等的名字
初始化	initialization	定义变量时给初值
整数	integer	没有小数部分的数
关键字	keyword	C 语言保留的特殊单词
库	library	预先写好的函数集合
链接器	linker	把目标文件和库组合成可执行文件
局部变量	local variable	定义在函数或块内的变量
循环	loop	重复执行一段代码
宏	macro	预处理阶段的文本替换

中文	English	说明
内存泄漏	memory leak	申请的内存没有释放
空指针	null pointer	值为 `NULL` 的指针
操作数	operand	参与运算的数据
运算符	operator	表示运算的符号
溢出	overflow	数值超出类型能表示的范围
形参	parameter	函数定义中声明的参数
指针	pointer	存放地址的变量
预处理器	preprocessor	编译前处理 `#` 指令的程序
函数原型	prototype	函数的声明，说明返回值和参数类型
递归	recursion	函数调用自身
返回值	return value	函数执行后返回给调用者的值
作用域	scope	名字有效的范围
段错误	segmentation fault	访问非法内存导致的崩溃
栈	stack	存放局部变量和函数调用信息的区域
语句	statement	程序执行的基本单位，以分号结尾
字符串	string	以 `\0` 结尾的字符数组
字符串常量	string literal	双引号括起来的字符序列
结构体	struct	把不同类型成员组合成的类型
语法	syntax	语言的书写规则
补码	two's complement	计算机表示负数的一种编码方法（本书不展开）
未定义行为	undefined behavior	标准没有规定结果的行为，必须避免
联合体	union	多个成员共用同一块内存的类型
变量	variable	有名字、类型、地址的存储空间
`void`	void	表示“无类型/无返回值”
野指针	wild pointer	未初始化或指向无效地址的指针

查英文资料的技巧

遇到不认识的术语，先在本表查中文，再用英文关键词搜索，能找到更准确的结果。例如搜 `dangling pointer` 比搜“悬空指针”得到的资料多得多。

附录 E 函数速查（补充）

本书的章节表里没有单独安排“函数”一章，但后面的结构体、指针、排序都用到了函数。本附录把函数的必要语法补齐，够你读懂和编写本书的代码。

E.1 函数的定义与调用

function-basic.c ——定义一个函数

```c
#include <stdio.h>

int add(int a, int b)
{
    int sum;

    sum = a + b;
    return sum;
}

int main(void)
{
    int result;

    result = add(3, 5);
    printf("%d\n", result);

    return 0;
}
```

运行结果

```
8
```

函数定义由四部分组成：

- **返回类型**：`int` 表示函数返回一个整数；不返回任何值写 `void`；
- **函数名**：`add`，遵循标识符命名规则；
- **参数列表**：`(int a, int b)`，没有参数写 `(void)`；
- **函数体**：花括号里的语句，用 `return` 返回结果。

C89 的参数声明

C89 支持 `int add(int a, int b)` 这种写法，但 `return` 的类型和参数类型都必须写清楚。不要写 `add(a, b)` 这种“隐式 int”的老式写法。

E.2 返回值与 void

function-void.c ——不返回值的函数

```c
#include <stdio.h>

void print_line(int n)
{
    int i;

    for (i = 0; i < n; i++) {
        printf("-");
    }
    printf("\n");
}

int main(void)
{
    print_line(20);
    return 0;
}
```

void 函数可以用 return; 提前结束，但不能返回一个值。

function-early.c ——提前返回

```c
int safe_divide(int a, int b)
{
    if (b == 0) {
        return 0;       /* 简单处理：除数为 0 返回 0 */
    }
    return a / b;
}
```

E.3 函数声明（原型）

如果函数定义在 main 之后，或者跨文件使用，需要先声明：

function-prototype.c ——函数原型

```c
#include <stdio.h>

int add(int a, int b);      /* 声明：告诉编译器有这样的函数 */

int main(void)
{
    printf("%d\n", add(3, 5));
    return 0;
}
```

```
10
11 int add(int a, int b)       /* 定义 */
12 {
13     return a + b;
14 }
```

C89 不要求参数名与声明一致，但类型必须一致。养成“先声明、后使用”的习惯。

E.4 参数传递

E.4.1 按值传递

C 语言的参数默认是**按值传递**：函数拿到的是实参的副本，修改副本不影响原件。

pass-value.c ——按值传递

```c
#include <stdio.h>

void change(int x)
{
    x = 100;
}

int main(void)
{
    int a = 10;

    change(a);
    printf("%d\n", a);      /* 仍然是 10 */
    return 0;
}
```

E.4.2 传地址

要让函数修改调用者的变量，传指针：

pass-pointer.c ——传地址

```c
#include <stdio.h>

void change(int *p)
{
    *p = 100;
}

int main(void)
{
    int a = 10;
```

```
11
12      change(&a);
13      printf("%d\n", a);      /* 100 */
14      return 0;
15  }
```

E.4.3 数组参数

数组作为参数会退化成指针，必须额外传长度：

pass-array.c ——数组参数

```
1   int sum_array(int a[], int n)
2   {
3       int i;
4       int sum = 0;
5
6       for (i = 0; i < n; i++) {
7           sum = sum + a[i];
8       }
9       return sum;
10  }
```

E.5 递归

函数可以直接或间接调用自己，这叫递归。递归必须有**终止条件**，否则会无限递归导致栈溢出。

recursion.c ——递归求阶乘

```
1   int factorial(int n)
2   {
3       if (n <= 1) {
4           return 1;               /* 终止条件 */
5       }
6       return n * factorial(n - 1); /* 递归调用 */
7   }
```

递归适合描述“问题可以分解成同样形式的小问题”，例如阶乘、斐波那契、汉诺塔。递归的缺点是调用层次深时占用栈空间大。

E.6 作用域与存储类别

- **局部变量**：定义在函数或块内，只在块内可见，函数返回后失效；
- **全局变量**：定义在所有函数之外，整个程序可见，默认初始化为 0；
- `static` 局部变量：只初始化一次，函数返回后值仍然保留；
- `static` 全局变量/函数：只在当前源文件可见。

scope.c ——作用域示例

```c
#include <stdio.h>

int global_count = 0;       /* 全局变量 */

void f(void)
{
    int local = 0;          /* 每次调用都重新初始化 */
    static int calls = 0;   /* 只初始化一次，跨调用保留 */

    local++;
    calls++;
    global_count++;

    printf("local=%d, calls=%d, global=%d\n",
           local, calls, global_count);
}
```

E.7 多文件编程

大型程序通常拆成多个源文件和头文件：

math_utils.h ——头文件

```c
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b);
int max_int(int a, int b);

#endif
```

math_utils.c ——实现文件

```c
#include "math_utils.h"

int add(int a, int b)
{
    return a + b;
}

int max_int(int a, int b)
{
    return (a > b) ? a : b;
}
```

main.c ——使用函数

```c
#include <stdio.h>
#include "math_utils.h"

int main(void)
{
    printf("%d\n", add(2, 3));
    return 0;
}
```

编译命令：

命令行

```
gcc main.c math_utils.c -o app -std=c89 -Wall
```

头文件里只放声明

头文件放函数声明、宏、类型定义；函数实现放在.c 文件里。头文件用 #ifndef 保护，避免重复包含。

E.8 函数的常见错误

错误	后果
声明与定义参数类型不一致	编译错误或运行异常
调用函数前没有声明	隐式声明警告，C99 起报错
函数没有 return 却使用返回值	未定义行为
递归没有终止条件	栈溢出崩溃
数组参数忘记传长度	越界访问
返回局部数组的地址	悬空指针

函数设计的三条建议

1. 一个函数只做一件事；
2. 函数名用动词，参数和返回值类型写清楚；
3. 函数尽量短，超过一屏就考虑拆分。